阿里云天池 Python 训练营 - Part1

- 2022 年 02 月 14 日版本，初始版本
- 2024 年 01 月 14 日版本，修改版本
- 本文档是以 CC 开源的模式的发布，你能且将获得本文档的 PDF 版本以及 Jupyter Notebook 版本
- 本文档并不申明自己的版权信息，为了更好的知识传播，我们授权你使用本文档，你可以使用它，进行二次创作，进行分发，进行修改，并可以以此为蓝本进行授课。
- 请保留本文档的原始来源。
- 本文档首先在国际人学校组织的“阿里云天池 Python 训练营课程”中使用。
- 本文档由林肯老师首次组织和编辑。你可以通过网址 数据大咖 找到他。

1. 术语表、中英文对照、注意要点



人类语言是如何产生的

1.1 为什么要学习编程

本次课程的目标是教会你像计算机科学家一样思考。

这种思考方式综合了数学、工程学以及自然科学的一些最优秀的特性。计算机科学家与数学家类似，他们使用形式语言来描述理念(特别是计算）；与工程师类似，他们设计产品，将元件组装成系统，对不同的方案进行评估选择；与自然科学家类似，他们观察复杂系统的行为，构建科学假说，并检验其预测。

作为计算机科学家，最重要的技能就是**问题求解**。

问题求解是发现问题、创造性地思考解决方案以及清晰准确地表达解决方案的能力。

实践证明，学习编程的过程，正是训练问题求解能力的绝佳机会。

一方面，你将学会编程，其本身就是一个非常有用的技能；

另一方面，你可以使用编程作为工具，去达到更高的目标。

1.1.1 什么是程序

程序是指一组定义如何进行计算的指令的集合。这种计算可能是数学计算，如解方程组或者查找多项式的根，也可以是符号运算，如搜索和替换文档中的文本，或者图形相关的操作，如处理图像或播放视频。

在不同的编程语言中，程序的细节有所不同，但几乎所有编程语言中都会出现以下几类基本指令。

- 输入(input)：从键盘、文件或者其他设备中获取数据。
- 输出(output)：将数据显示到屏幕，保存到文件中，或者发送到网络上等。
- 数学(math)：进行基本数学操作，如加法或乘法。
- 条件执行(if)：检查某种条件的状态，并执行相应的代码。
- 重复(for,while)：重复执行某种动作，往往在重复中有一些变化。

信不信由你，这差不多就是全部了😀。

你所遇到过的所有程序，无论多么复杂，都是由类似上面的这些指令组成的。

所以我们可以把编程看作一个将大而复杂的任务分解为更小的子任务的过程，不断分解，直到任务简单到足以由上面的这些基本指令组合完成。

1.1.2 什么是语言， 形式语言和自然语言

自然语言是指人们所说的语言，如英语、西班牙语和法语。它们不是由人设计而来的(虽然人们会尝试加以语法限制），而是自然演化而来的。

$$\sin 2\alpha = 2\sin\alpha\cos\alpha \qquad \log_a \frac{b}{c} = \log_a b - \log_a c \qquad \cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$$

$$\left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)\cdot g(x) - f(x)\cdot g'(x)}{g^2(x)} \qquad tg(\alpha+\beta) = \frac{tg\alpha + tg\beta}{1 - tg\alpha tg\beta}$$

$$\sin^2\alpha + \cos^2\alpha = 1 \qquad \sin(\alpha-\beta) = \sin\alpha\cos\beta - \cos\alpha\sin\beta$$

$$tg^2\alpha + 1 = \frac{1}{\cos^2\alpha} = \sec^2\alpha \qquad \cos 2\alpha = \cos^2\alpha - \sin^2\alpha \qquad f'(x) = \lim_{x\to 0}\frac{f(x+\Delta x) - f(x)}{\Delta x}$$

$$tg\alpha ctg\alpha = 1 \qquad 2\sin^2\alpha = 1 - \cos 2\alpha$$

$$\frac{a}{\sin\alpha} = \frac{b}{\sin\beta} = \frac{c}{\sin\gamma} = 2R \qquad \cos 2\alpha = 2\cos^2\alpha - 1 \qquad \log_{a^n} b = \frac{1}{n}\log_a b$$

$$tg(\alpha-\beta) = \frac{tg\alpha - tg}{1 + t\ \alpha t} \qquad \sin x = a;\ \ x = (-1)^n \arcsin a + \pi n, \qquad a^{\log_b c} = c^{\log_b a} \qquad 2\cos^2\alpha = 1 + \cos 2\alpha$$

$$\log_a b^r = r\log_a b \qquad \cos\alpha - \cos\beta = -2\sin\frac{\alpha+\beta}{2}\sin\frac{\alpha-\beta}{2}$$

$$ctg^2\alpha + 1 = \frac{1}{\sin^2\alpha} = \operatorname{cosec}^2\alpha \qquad \sin\alpha - \sin\beta = 2\sin\frac{\alpha-\beta}{2}\cos\frac{\alpha+\beta}{2} \qquad (\sin x + \cos x)^2 = 1 + \sin 2x$$

$$(\sin x - \cos x)^2 = 1 - \sin 2x \qquad S_\Delta = \sqrt{p(p-a)\cdot(p-b)\cdot(p-c)} = p\cdot r$$

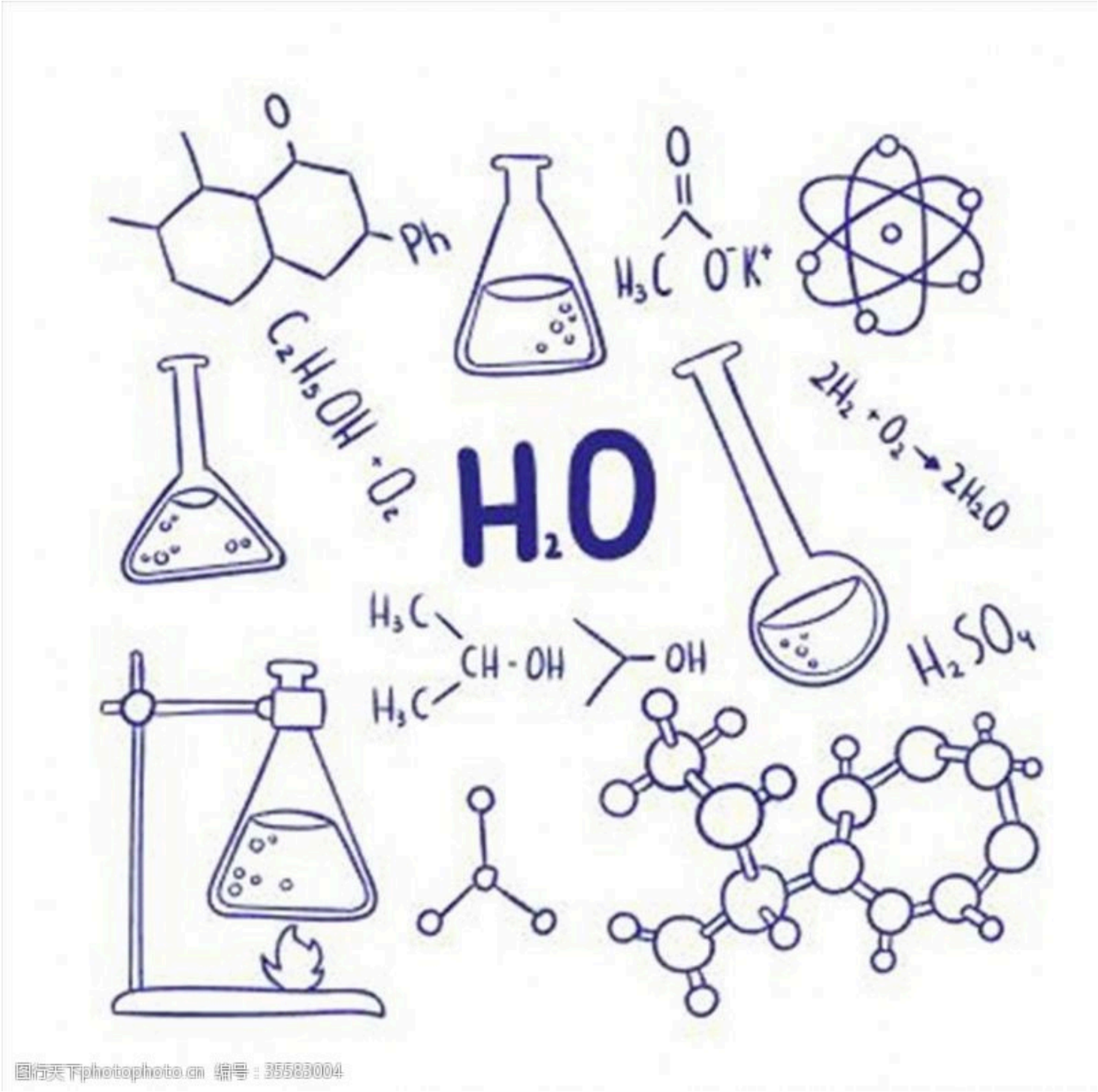
$$tg2\alpha = \frac{2tg\alpha}{1 - tg^2\alpha} \qquad \cos 2\alpha = 1 - 2\sin^2\alpha \qquad tgx = a;\ \ x = \operatorname{arctg} a + \pi n$$

$$\sin\alpha + \sin\beta = 2\sin\frac{\alpha+\beta}{2}\cos\frac{\alpha-\beta}{2} \qquad \operatorname{arctg}(-a) = -\operatorname{arctg} a$$

$$e = \lim_{n\to\infty}\left(1 + \frac{1}{1!} + \frac{1}{2!} + \ldots + \frac{1}{n!}\right) \approx 2{,}71$$

$$\cos\alpha + \cos\beta = 2\cos\frac{\alpha+\beta}{2}\cos\frac{\alpha-\beta}{2} \qquad \log_a b = \frac{\log_c b}{\log_c a}$$

$$\arccos(-a) = \pi - \arccos a \qquad 2\sin\alpha\sin\beta = \cos(\alpha-\beta) - \cos(\alpha+\beta) \qquad \arcsin(-a) = -\arcsin a$$



形式语言 则是人们为了特殊用途设计的语言。

例如，数学上使用的符号体系是一种特别擅于表示数字和符号之间关系的形式语言；化学家则使用另一种形式语言来表示分子的化学结构。而最重要的是：**编程语言是人们为了表达计算过程而设计出来的形式语言。**

形式语言倾向于对语法做出严格的限制。例如，`3+3=6` 是语法正确的数学表达式，但 `3+=3$6` 则不是。

H_2O 是语法正确的化学方程式，而 2Zz 则不是。

语法规则有两种，分别适用于记号(token）和结构(structure）。

记号是语言的基本元素，如词、数字和化学元素。

3 + = 3 $ 6 的一个问题就是 **$** 在数学表达式中(至少就我所知）不是合法记号。相似地，2Zz 不合法是因为并不存在缩写为 Zz 的化学元素。

第二种语法规则指定记号所组合的方式。

数学等式 **3 + = 3** 不合法，因为虽然＋和＝是合法记号，但不能将它们连续放置。

相似地，在化学表达式里，下标数字应该出现在元素名称之后，而不是之前。

"This is@well-structured EngliSh sentence with invalid t∗kens in it." 是一个结构良好，但包含非法记号的英语语句。

"This sentence all valid tokens has，but invalid structure with." 这句话所有的记号都合法，但是语句结构不合法。

当你阅读英语的句子或形式语言的语句时，需要弄清句子的结构是什么(虽然在自然语言中这个过程是下意识完成的）。这个过程称为语法分析。

虽然形式语言和自然语言有很多共同的特点－**记号、结构、语法以及语义**，但它们也有一些区别。

- 歧义性：
    - 自然语言充满了歧义，人们通过上下文线索和其他信息来处理这些歧义。
    - 形式语言通常设计为几乎或者完全没有歧义，即不论上下文环境如何，任何表达式都只有一个含义。
- 冗余性：
    - 为了弥补歧义，减少误解，自然语言采用大量的冗余。因此，自然语言往往很啰唆。形式语言则相对不那么冗余，更加简洁。
- 字面性：
    - 自然语言充满了习惯用语和比喻。例如，有人说，“硬币掉了”(The penny dropped），并不一定是硬币，也不一定是有什么掉了。
    - 形式语言则严格按照它的字面意思表达含义。

形式语言的密度远远大于自然语言，所以阅读起来需要花费更多的时间。还有，结构非常重要，所以直接自顶向下、从左至右的阅读顺序并不一定是最好的。相反，要试着学会在头脑中解析程序，辨别出记号并解析出结构。最后，细节很重要。在自然语言中常常可以忽略的小错误，如拼写错误或者标点符号错误，在形式语言中往往会造成很大的差别。

1.2 术语表、中英文对照

#	英文	中文

1	byte	字节
2	int	整型、整数
3	float	浮点型、小数
4	complex	复数
5	string	字符串
6	char	字符
7	tuple	元组类型
8	list	列表类型
9	dict	字典类型
10	set	集合类型
11	date	日期类型
12	time	时间类型
13	random	随机数
14	row	行
15	column	列
16	table	表格
17	range	范围
18	if	判断(如果)
19	else	其他
20	elif	其他分支(判断)
21	for	循环
22	while	循环
23	continue	跳出循环，继续下一次
24	break	跳出整个循环
25	add,addition	加法
26	sub,subtraction	减法
27	mul,multiple,multiplication	乘法

28	div,division	除法
29	function	函数
30	method	方法
31	return	返回，返回值
32	result	结果
33	def,defination,define	定义
34	max	最大
35	min	最小
35	avg	平均值
36	parameter	参数
37	info,infomation	信息
38	lambda	匿名函数
39	iterator	迭代器
40	generator	生成器
41	True	真，真值
42	False	假，假值
43	next	下一个
44	OOP，Object Oriented Programming	面向对象编程
45	assert	断言
46	class	类
47	del,delete	删除
48	pop	弹出，栈推出
49	try ... except	抓住异常
50	finally	最终
51	global	全局
52	import	引入
53	or	或者

54	not	取反
55	and	并且
56	raise	抛出异常
57	with	获取句柄
58	indent,indentation	缩进
59	unindent	未缩进
60	match	匹配
61	sys,system	系统
62	std,standard	标准的
63	write	写入
64	read	读取
65	path	路径
66	dir,directory	文件夹
67	file	文件
68	instance	实例
69	sub	子
70	reverse	反转，abc 转成 cba
71	eval	计算有效表达式，并返回对象
72	order	排序
73	trace	堆栈
74	call	调用
75	support	支持
76	unsupported	不支持的
77	in	包含
78	not in	未包含
79	is	是某对象
80	is not	不是某对象

81	center	居中
82	count	计数，计算出现次数
83	encode	编码
84	decode	解码
85	endswith	以... 结尾
86	find	查找
87	index	索引位置
88	isnumeric	是否是数字类型
89	isalpha	是否是字符
90	isalnum	是否是字符或数字
91	islower	是否是小写
92	isupper	是否是大写
93	isspace	是否是空格
94	join	合并字符串
95	len,length	字符串长度
96	strip	截掉空格或指定字符
97	replace	替换
98	split	以... 为分隔符截取字符串
99	startswith	是否以... 开头
100	append	追加
101	insert	插入
102	remove	移除
103	sort	排序
104	clear	清空列表
105	copy	复制...
106	items	获取... 中的项
107	keys	获取... 中的键

108	values	获取... 中的值
109	difference,diff	多个集合的差集
110	intersection	多个集合的交集
111	subset	子集
112	superset	超集
113	union	并集
114	symmetric_difference	获取两个结合中不重复的元素集合
115	pass	过
116	module	模块
117	package	包
118	application	应用程序
119	filter	过滤器
120	mkdir	创建目录
121	rename	重命名
122	unlink	删除文件路径
123	error	错误
124	warn	警告
125	init	初始化
126	public	公共的
127	private	私有的
128	protected	受保护的
129	mod	求余数
130	pow	乘方
131	cmp,compare	比较
132	namespace	命名空间
133	math	数学包
134	sin	正弦

135	cos	余弦
136	log	对数
137	request	网络请求包
138	response	响应
139	regex	正则表达式

1.3 注意要点

#	符号	中文符号	英文符号
1	单引号	‘’	''
2	双引号	“”	""
3	小括号	（）	()
4	感叹号	！	!
5	百分号	%	%
6	中括号	【】	[]
7	大括号	{}	{}
8	问号	？	?
9	分号	；	;
10	斜杆	/	/
11	冒号	：	:
12	货币	￥	$
13	井号	#	#
14	反斜杠	\	\
15	竖杠	|	|
16	加号	＋	+
17	减号	－	-
18	星号	＊	*
19	大于	＞	>

20	小于	<	<
21	等号	=	=
22	大于等于	> =	>=
23	小于等于	< =	<=

2. 输出

一般查看输出结果可以使用 `print` 函数

```
print('嗨! python 我来了!')
```

以下是 Python 中 `print()` 函数的一些常见用法和示例，涵盖了基础到高级的使用场景：

2.1 基本用法

输出字符串、数字、变量等。

```
# 输出字符串
print("Hello, World!")

# 输出数字
print(42)

# 输出变量
name = "Alice"
age = 25
print("Name:", name, "Age:", age)
```

2.2 格式化字符串

使用 `f-string`、`str.format()` 和 `%` 格式化输出。

```
# f-string
name = "Bob"
score = 95.6
print(f"{name} scored {score:.1f} points.")

# str.format()
print("{} scored {:.1f} points.".format(name, score))
```

```
8
9  # % 格式化
10 print("%s scored %.1f points." % (name, score))
```

2.3 自定义分隔符

通过 `sep` 参数指定多个值之间的分隔符。

```
1 print("apple", "banana", "cherry", sep=", ")
```

2.4 禁止自动换行

通过 `end` 参数更改默认换行行为。

```
1 # 默认换行
2 print("Hello")
3 print("World")
4
5 # 禁止换行
6 print("Hello", end=" ")
7 print("World")
```

2.5 输出到文件

将内容写入文件而不是控制台。

```
1 with open("output.txt", "w") as file:
2     print("This will be written to a file.", file=file)
```

2.6 转义字符

在字符串中使用特殊字符。

```
1 # 换行和制表符
2 print("Hello\nWorld")  # 换行
3 print("Name\tAge")     # 制表符
4
  # 转义引号
```

```
print("He said, \"Python is awesome!\"")

```

2.7 输出 Unicode 字符

支持输出表情符号或特殊字符。

```
print("Emoji: 😊, Symbol: Ω, Chinese: 你好")
```

2.8 调试时输出变量名和值

在调试时输出变量名和值，可以结合 `f-string` 使用。

```
x = 42
y = 3.14
print(f"x = {x}, y = {y}")
```

2.9 使用 `print` 输出复杂数据结构

输出列表、字典等。

```
my_list = [1, 2, 3]
my_dict = {"name": "Alice", "age": 25}

print("List:", my_list)
print("Dictionary:", my_dict)
```

2.10 彩色输出

利用 `colorama` 或 ANSI 转义序列实现彩色输出。

```
# 使用 ANSI 转义序列
print("\033[31mThis is red text.\033[0m")
print("\033[32mThis is green text.\033[0m")

# 使用 colorama 库
from colorama import Fore, Style
print(Fore.RED + "This is red text." + Style.RESET_ALL)
print(Fore.GREEN + "This is green text." + Style.RESET_ALL)
```

```
8
```

2.11 输出带时间戳的日志

结合 `datetime` 模块输出带时间的消息。

```python
from datetime import datetime

current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
print(f"[{current_time}] Process started...")
```

3. 输入

使用 `print` 函数可以完成数据的输出，那么如何编写输入语句呢？一般使用 `input` 函数。

```python
name = input('请输入你的姓名: ')
print(name)
```

4. Python 的代码注释

代码注释就是为写好的代码片段添加注解。做代码注释有以下两好处：

- 能更好地维护项目，也能让阅读者更快地读懂代码的意思；
- 在做代码调试时，如果需要让一部分代码暂时不运行，也可以使用注释的方法。

4.1 单行注释

单行注释，即注释只作用于一行。在单独的行写注释内容之前，要输入 `#` (井号）。

```python
# 将数据输出到屏幕上
print('嗨! python 我来了! ')
# 在此输入姓名
name = input('请输入你的姓名:')
# 输出输入的数据
print(name)
```

为了减少代码行数，可以将单行注释写到每一行的后面

```python
print('嗨! python 我来了! ') #将数据输出到屏幕上
name = input('请输入你的姓名: ') # 在此输入姓名
print(name) # 将打印出输入的数据**
```

4.2 多行注释

如果有大段的注释文字要写，则可以使用多行注释的方法。多行注释的内容要包含在一对单引号中，6 个单引号为一对。**单引号中的内容不会被运行。**

```
'''
print 函数表示将数据输出
input 函数表示要接收输入的数据
两个函数可以结合使用
'''
name = input('请输入你的姓名: ')
print(name)
```

除使用单引号做注释外，也可以使用双引号来做注释。**双引号中的内容也不会被运行。**

```
"""
print 函数表示将数据输出
input 函数表示要接收输入的数据
两个函数可以结合使用
"""
name = input('请输入你的姓名: ')
print(name)
```

> 到底是使用单引号做注释，还是使用双引号做注释，没有强制规定，完全根据用户的习惯而定。

5. Python 对象详解

真实的世界是由千千万万的对象组成的。

在 Python 的编程世界里，所有的一切也可以看作对象，比如数字、字符，以及后面将会学到的列表、元组、集合、字典、函数等。

用户可以使用这些对象，也可以在 Python 中创建自己的对象。

5.1 类的定义(defination)

类也是一种对象，只不过它是用来创建对象的一种对象。类用来描述具有相同属性和方法的对象集合，它定义了该集合中每个对象所共有的属性和方法，对象是类的实例。也就是说，对象是由类创建的。比如，后面的章节中会讲解通过 list 类来创建或转换一个列表对象。

5.2 对象的身份(id)

在现实生活中，人就是一个类，而每一个具体的人就是对象，具体的人可以靠身份证号来进行识别，也可以定位所在位置。

Python 中的对象也是有身份的，可以通过 id 函数来识别对象在内存中的地址。比如，字符串 **'杨辉'** 就是对象，输入代码

```
print(id('杨辉'))
```

屏幕上的输出结果为 **2673213256496**，这串数字就可以看作该字符串在内存中的地址，并且具有唯一性。但是，这串数字是变化的，因此在测试代码时，每次输出的结果可能不一样，读者不要为此感到困惑。

5.3 对象的类型(type)

虽然万物皆对象，但对象也有类型之分。

比如，猪、狗、牛、马、花、草、树、木等都是对象，但它们却是不同的类型，猪、狗、牛、马是动物类型，花、草、树、木是植物类型。

在 Python 中，99、888、 'abc' 都是对象，9 和 888 是数字类型，而 'abc' 是字符串类型。

不同类型的对象有着不同的属性和方法，遵循不同的规则。

要查看对象的类型，可以使用 `type` 函数。

比如，输入代码 `print(type(99))`，返回结果为 `<class 'int' >`，表示 99 是 int 类型，也就是整数类型。再比如，输入代码 `print(type('abc'))`，返回结果为 `<class 'str' >`，表示 'abc' 是 str 类型，也就是字符串类型。

其他对象的类型就不一一列举了，后文中会涉及。

5.4 对象的值(value)

对象除有身份、类型外，还有值。

人的名字就可以看作值。在 Python 中，有的对象的值是可以改变的，有的对象的值不可以改变。比如，数字、字符、元组都是不可以改变值的对象。

5.5 对象的属性(attribute)

对象的特征也称为属性。比如，字符串 **'abcd'**，它的字符长度为 **4**，这个长度就是该字符串的属性。

5.6 对象的方法(function)

对象所具有的行为也可以称作方法。
比如，对字符串 **'a-b-c-d'** 进行拆分，这个 **拆分**就可以说是方法。

在 Python 中，方法的本质是函数，在类中定义的函数叫作方法，没在类中定义的函数就叫作函数。

在后面的章节中将讲解一些内建的函数或方法，为方便读者阅读，统一都叫作函数。

5.7 对象与变量

在编程过程中，很多时候需要给对象设置变量，相当于给对象贴一个标签，这样更方便识别。

比如 a = 1，表示给对象 1 贴一个标签，在引用变量 a 进行代码编写时，就相当于在使用对象 1。

在命名变量时，需要注意如下规则：

- 变量名可以由字母、数字、下画线 `_` 组成，但不能以数字开头；
- 变量名不能是 Python 关键字，但可以包含关键字；
- 变量名不能包含空格；
- 变量名尽量取得有意义，容易让人识别。

5.8 Python 中的数字与字符串

Python 中万物皆对象，数字与字符串只是其中的两种对象。

为什么要先学这两种对象呢？因为这两种对象最常用，也比较容易学习。在后面的章节中将陆续讲解更多的对象。

5.8.1 数字

Python 中的数字有 3 种类型：**整数、浮点数(小数）、复数**。

有时需要对数字进行转换，可以使用对象的函数，转换为整数使用 `int` 函数，转换为小数使用 `float` 函数，转换为复数使用 `complex` 函数。

将字符串 '99' 赋值给 `num` 变量，看看不同函数对 `num` 变量处理的不同结果。

```
num = '99'
print(num)  # 返回 '99'
print(type(num)) # 返回<class  'str' >

print(int(num)) # 返回 99
print(type(int(num))) # 返回<class  'int' >

print(float(num)) # 返回 99.0
print(type(float(num))) # 返回<class  'float' >

print(complex(num)) # 返回 (99+0j)
print(type(complex(num))) # 返回<class  'complex' >
```

5.8.2 字符串

字符串就是一串字符，是一个及以上字符的集合。

Python 中的字符串必须被一对单引号('')或双引号("")包围起来。要将其他数据转换为字符串类型，可以使用 `str()` 函数。

比如，将数字转换为字符串类型。

```
num = 99
print(str(num)) # 返回 99
print(type(str(num))) # 返回<class  'str' >
# 在 Python 中，还有一些常用的特殊字符，比如换行符（\n）、制表符（\t）、回车符（\r）等。在遇到特殊字符，需要将其转换为普通字符时，在其前面加上“\”即可。
# 还有另一种转换方法是，在字符串的左外侧加上字母 r（大小写均可）。
print('我是谁! \n 我在哪儿! ') # \n 表示换行
print('我是谁! \\n 我在哪儿! ') # \\n 被识别为普通字符
print(r'我是谁! \n 我在哪儿! ') # \n 被识别为普通字符
```

5.9 算术运算符

Python 中的常用算术运算符有加(+)、减(-)、乘(*)、除(/)、取模(%)、幂(**)、取整数(//)。下面具体介绍这些运算符。

5.9.1 加(+)

加运算符是执行数字相加运算的符号，比如运行代码 `print(100+199)`，将返回数字 299。

同时，加运算符也可以进行字符串的连接运算，比如运行代码 `print('杨辉'+'99 分')`，返回文本 **杨辉 99 分**。

```
print(100 + 199) # 返回数字 299
print('杨辉' + '99 分') # 返回文本 杨辉 99 分
```

除此之外，加号还可以在 **列表、元组** 等对象中做连接。

5.9.2 减(-)

减运算符是执行数字相减运算的符号，比如运行代码 `print(100-99)`，返回结果 1 。

```
print(100-99) # 返回结果 1
print('100' - 99) # 不能正确计算
```

如果相减的两个值中有一个值不是标准数字，就不能正确进行计算。

运行代码 `print('100' - 99)` 后，提示“unsupported operand type(s) for - : 'str' and 'int' ”。

意思是一个值为字符串型，另一个值为整型，这两种数据类型不能在一起运算。

5.9.3 乘(*)

乘运算符是执行数字相乘运算的符号，比如运行代码 `print(100*99)`，返回 9900。乘运算符也有**重复** 的作用，可对字符串重复进行运算，比如运行代码 `print('python! '*3)`， 返回“python! python! python! ”。

```
print(100*99) # 返回 9900
print('python! '*3) # 返回 python! python! python!
```

除此之外，乘号还可以用于重复其他对象，比如 **列表、元组** 等。

5.9.4 除(/)

除运算符是执行数字相除运算的符号，比如运行代码 `print(63/8)`，返回 7.875。

```
print(63/8) # 返回数字 7.875
```

> 注意，相除的结果为 float 型，即使商是整数，但其类型也是浮点型(小数) 。

5.9.5 取模(%)

取模运算符是执行数字相除运算后取余数的符号，比如运行代码 `print(63%8)`，返回 7，这个值便是 63 除以 8 的余数。

```
print(63%8) # 返回余数 7
```

5.9.6 幂(**)

幂运算符是执行乘方运算的符号。`n**m` 是指 m 个 n 相乘，也叫 n 的 m 次方。比如运行代码 `print(4**8)`，表示 4 的 8 次方，返回 65536。

```
print(4**8) # 返回值为 65536
```

5.9.7 取整数(//)

取整运算符是执行数字相除运算后取商的整数的符号，比如运行代码 `print(63//8)`，直接相除的商为 7.875，只取商的整数部分，所以返回 7。

```
print(63//8) # 返回值为 7
```

5.10 比较运算符

比较运算符通常用于比较两个数值或两个表达式的大小，比较结果返回一个逻辑值(`True` 或 `False`)，条件成立返回 `True`，条件不成立返回 `False`。

Python 中的比较运算符有 **等于(==)、不等于(!=)、大于(>)、小于(<)、大于或等于(>=)、小于或等于(<=)**。

下面以数字为比较对象介绍比较运算符的使用方法。

5.10.1 等于(==)

等于运算符用来比较两个数值是否相等，比如运行代码 `print(9 == 9)`，返回逻辑值 True，又比如运行代码 `print(9==8)`，返回 False。

```
print(9==9) # 返回逻样值 True
print(9==8) # 返回逻辑值 False
```

5.10.2 不等于(!=)

不等于运算符用来比较两个数值是否不相等，比如运行代码 `print(9!=8)`，返回逻辑值 True，又比如运行代码 `print(9!=9)`，返回逻辑值 False。

```
print(9!=8) # 返回逻辑值 True
print(9!=9) # 返回逻辑值 False
```

5.10.3 大于(>)

大于运算符用来判断其左边的数值是否大于右边的数值，比如运行代码 `print(9>8)`，返回逻辑值 True；又比如运行代码 `print(9 > 9)`，返回逻辑值 False。

```
print(9>8) # 返回逻辑值 True
print(9>9) # 返回逻辑值 False
```

5.10.4 小于(<)

小于运算符用来判断其左边的数值是否小于右边的数值，比如运行代码 `print(8<9)`，返回逻辑值 True；又比如运行代码 `print(9<9)`，返回逻辑值 False。

```
print(8<9) # 返回逻辑值 True
print(9<9) # 返回逻辑值 False
```

5.10.5 大于或等于(>=)

大于或等于运算符用来判断其左边的数值是否大于或等于右边的数值，比如运行代码 `print(8>=9)`，返回逻辑值 False；又比如运行代码 `print(9>=9)`，返回逻辑值 True。

```
print(8>=9) # 返回逻辑值 False
print(9>=9) # 返回逻辑值 True
```

5.10.6 小于或等于(<=)

小于或等于运算符用来判断其左边的数值是否小于或等于右边的数值，比如运行代码 `print(8<=9)`，返回逻辑值 True；又比如运行代码 `print(9<=8)`，返回逻辑值 False。

```
print(8<=9) # 返回逻辑值 True
print(9<=8) # 返回逻辑值 False
```

5.11 赋值运算符

赋值运算符 (=) 表示将等号右侧的对象赋给等号左侧的变量。等号左、右两侧的关系。

5.11.1 赋值运算

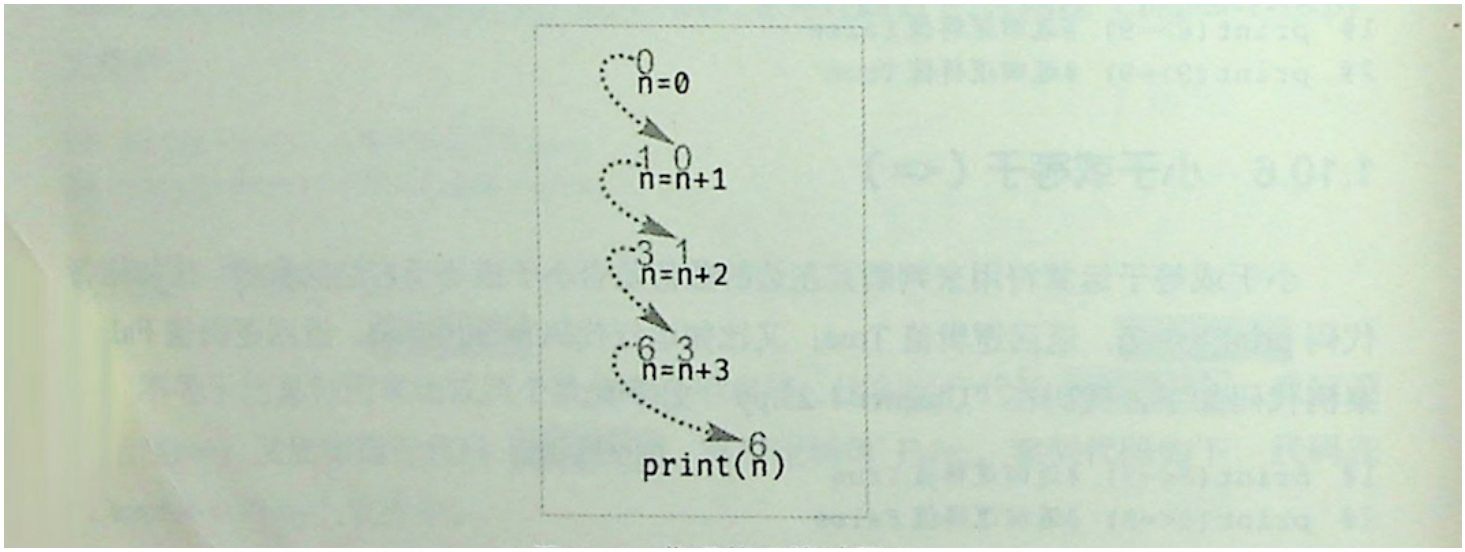
比如，`n=100` 表示变量 n 引用的对象是 100，`m=99` 表示变量 m 引用的对象是 99，代码 `print(n+m)` 表示将变量 n 引用的对象 100 与变量 m 引用的对象 99 相加，最后返回 199。

```
n = 100 # 将 100 赋值给变量 n
m = 99 # 将 99 赋值给变量 m
print(n+m) # 返回变量 n 与变量 m 加相的结果
```

5.11.2 累积式赋值运算

累积式赋值运算是编程中的一项重要技术。为了让读者更容易地理解累积式赋值运算的运算过程。

```
n = 0 # 变量 n 返回 0
n = n+1 # 变量 n 返回 1
n = n+2 # 变量 n 返回 3
n = n+3 # 变量 n 返回 6
print(n) # 在屏幕上输出变量 n 的值 6
```



除上面累积相加的累积式赋值运算外，还可以使用其他运算符做累积式赋值运算。

累积方式	写法 1	写法 2
加	n = n + 1	n+=1
减	n = n - 1	n-=1
乘	n = n * 100	n*=100
除	n = n / 100	n/=100
整除	n = n // 100	n//=100
取模	n = n % 100	n%=100
幂	n = n ** 100	n**=100

5.12 逻辑运算符

逻辑运算符一共有 3 个，分别是 **and、or 和 not**。

- and: 当 and 左右两边的条件都为真时，返回真(True）；否则，返回假(False）。
- or: 当 or 左右两边有一个条件为真时，返回真(True）；两个均为假，返回假(False）。
- not: 假的变成真的，真的变成假的，取反。

操作数 1	操作	操作数 2	结果
True	and	True	True
True	and	False	False
False	and	True	False
False	and	False	False

True	or	True	True
True	or	False	True
False	or	True	True
False	or	False	False
True	not		False
False	not		True

5.12.1 and（与）

当 and 运算符左右两边的条件都为真时，返回真(True）；当有一边的条件为假或两边的条件均为假时，返回假(False）。

```
print(True and True) # 左右两边均为 True，返回 True
print(100==100 and 10<11) # 对应案例
print(True and False) # 左边为 True，右边为 False，返回 False
print(100==100 and 10>11) # 对应案例
print(False and True) # 左边为 False，右边为 True，返回 False
print(100>100 and 10<11) # 对应案例
print(False and False) # 左右两边均为 False，返回 False
print(100>100 and 10>11) # 对应案例
```

5.12.2 or（或）

当 or 运算符左右两边任意一个条件为真时，返回真(True）；两个条件均为假，返回假(False）。

```
print(True or True) # 左右两边均为 True，则返回 True
print(100==100 or 10<11) # 对应案例
print(True or False) # 左边为 True，右边为 False，则返回 True
print(100==100 or 10>11) # 对应案例
print(False or True) # 左边为 False，右边为 True，则返回 True
print(100>100 or 10<11) # 对应案例
print(False or False) # 左右两边均为 False，则返回 False
print(100>100 or 10>11) # 对应案例
```

5.12.3 not（非）

如果对 True 取反，则返回 False；如果对 False 取反，则返回 True。

```
print(not True) # 如果对 True 取反，则返回 False
print(not 100==100) # 对应案例
print(not False) # 如果对 False 取反，则返回 True
```

5.13 成员运算符

除前面几个小节中讲解的算术运算符、比较运算符、赋值运算符、逻辑运算符外，Python 还支持使用成员运算符。

成员运算符用于测试字符串、列表等对象中是否包含指定的值。

成员运算符用 **in** 表示，返回值是逻辑值。

如果在指定的对象中找到了指定的值，则返回 True；否则返回 False。也可以使用 **not in** 来测试对象中没有指定的值。

```
print('杨辉' in  'IT 部杨辉 2000') # 返回 True
print('杨辉' not in  'IT 部杨辉 2000') # 返回 False
print('杨小辉' in  'IT 部杨辉 2000') # 返回 False
print('杨小辉'  not in  'IT 部杨辉 2000') # 返回 True
```

除可以在字符串中使用 in 运算符外，后面章节中将要学习的列表、集合、字典等对象也可以使用 in 运算符来做判断测试。

5.14 格式化字符串

在 Python 中，经常会对各种对象进行格式化处理。本节将使用 **format** 函数格式化指定的值，并将其插入字符串的占位符内。

5.14.1 使用位置和关键字格式化字符串

在使用 **format** 函数进行格式化时，使用花括号 **{}** 定义占位符，下面代码的返回值均为“恭喜杨辉获得 100 分。”

```
# 使用位置索引
# 按默认顺序获取 format 中的数据
print('恭喜 {} 获得 {} 分。' .format('杨辉' , 100))
# 按指定顺序获取 format 中的数据
print('恭喜 {0} 获得 {1} 分。' .format('杨辉' , 100))
# 使用关键字索引
# 按指定名称获取 format 中的数据
print('恭喜 {name} 获得 {score} 分。'.format(name='杨辉' , score=100))
```

也可以下面这么写：

```
name = '杨辉'
score = 100
print(f'恭喜 {name} 获得 {score} 分。')
```

```
name = '杨辉'
score = 100
print('恭喜 %s 获得 %d 分。' % (name, score))
```

5.14.2 数字格式设置

数字格式设置是常用设置，对数字格式化后返回的结果是字符串型数字。

```
print('{:.2f}'.format(3.1415926)) # 返回 3.14
print('{:.2%}'.format(0.1415926)) # 返回 14.16%
```

- : 表示要设置的值。
- .2 表示保留小数点后两位数。
- f 表示返回浮点数，也就是小数。
- % 表示设置成百分比格式。

5.14.3 对齐设置

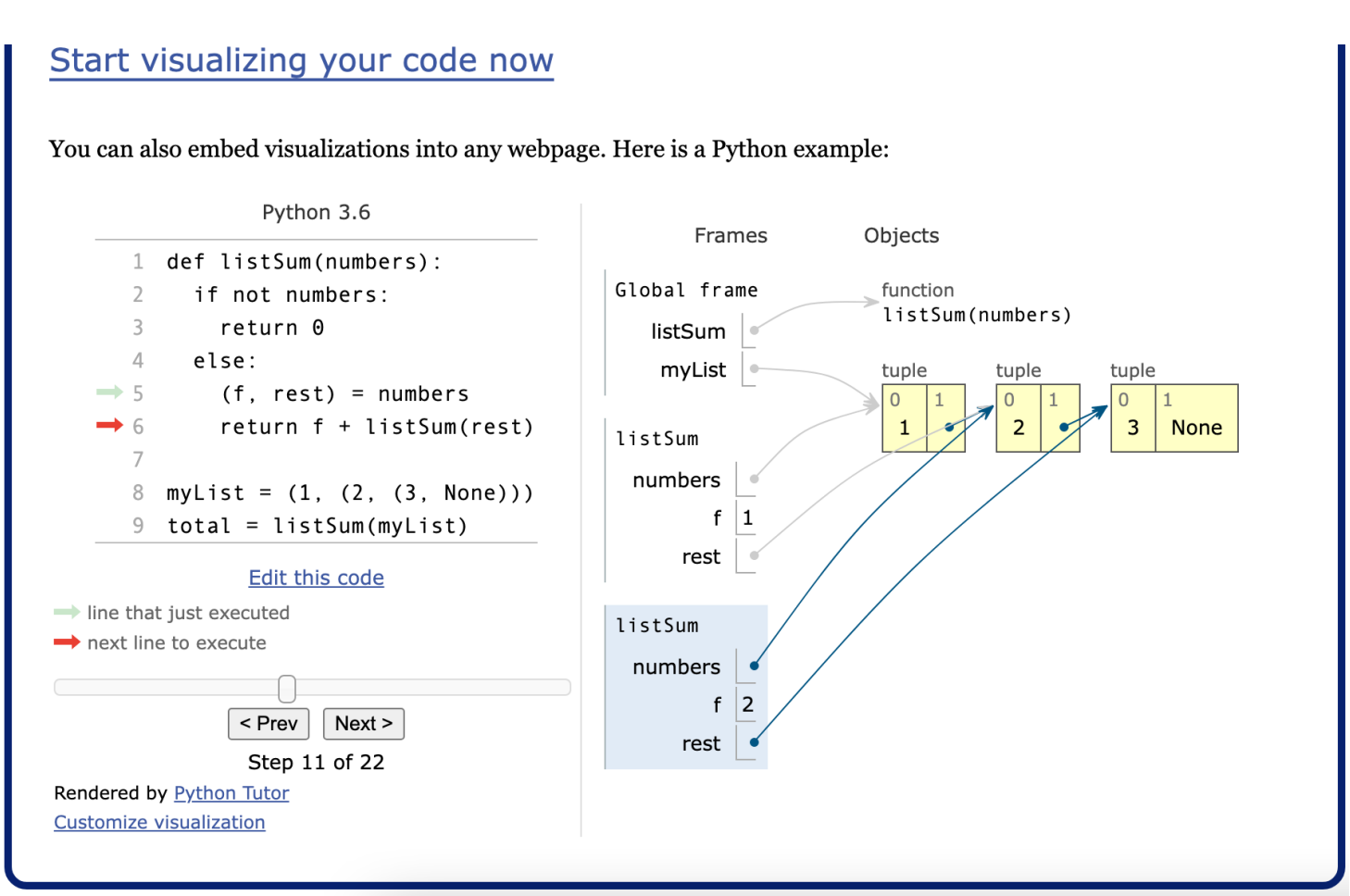
对齐设置是常用的格式化字符串的方式。

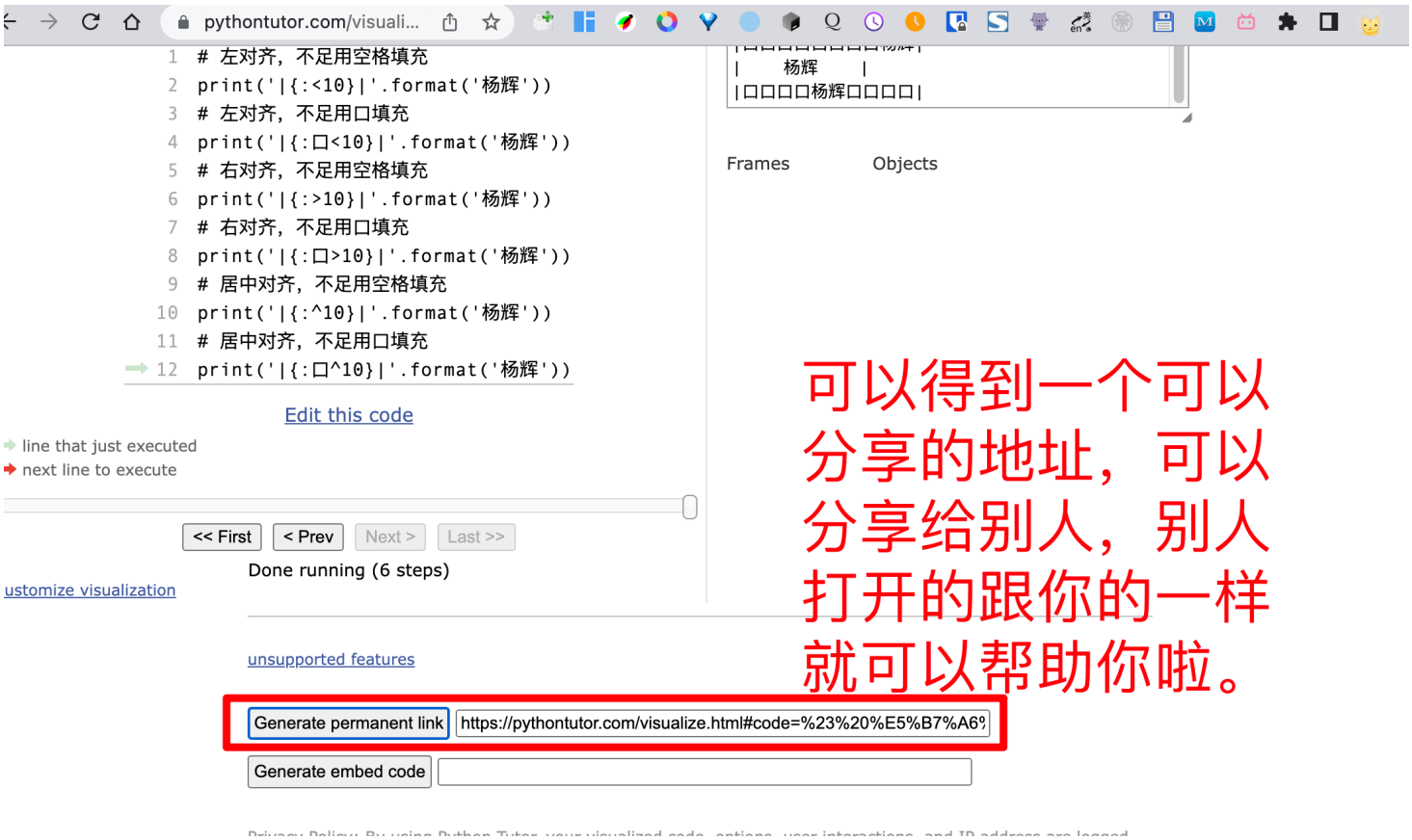
```
# 左对齐，不足用空格填充
print('|{:<10}|'.format('杨辉'))
# 左对齐，不足用口填充
print('|{: 口<10}|'.format('杨辉'))
# 右对齐，不足用空格填充
print('|{:>10}|'.format('杨辉'))
# 右对齐，不足用口填充
print('|{: 口>10}|'.format('杨辉'))
# 居中对齐，不足用空格填充
print('|{:^10}|'.format('杨辉'))
# 居中对齐，不足用口填充
print('|{: 口 ^10}|'.format('杨辉'))
```

- < 表示左对齐。
- > 表示右对齐。
- ^ 表示居中对齐。

5.15 pythontutor 使用

网址直达PythonTutor





6. Python 模块、包、库，三方库安装

6.1 什么是模块、包、库

- 模块(Module)
  - 模块是一个 Python 文件，扩展名为.py。
  - 在模块中能够组织 Python 代码段，把相关的代码放到一个模块里能让代码更好用、更易懂。
  - 在模块里能定义函数、类和变量，模块中也能包含可执行的代码。
- 包(Package)
  - 包是模块之上的概念，为了方便管理.py 模块文件，可以进行打包。
  - 包其实就是文件夹，只不过该文件夹下有名称为 `__init__.py` 的文件，否则就是普通的文件夹。
  - 包中可以有模块和子文件夹，假如子文件夹中也有 `__init__.py` 文件，那么它就是这个包的子包。
- 库(Library)
  - 在 Python 中，具有某些功能的模块和包都可以被称作库。

模块由诸多函数组成，包由诸多模块组成，库中可以包含模块、包和函数。

Python 中的库分为标准库和第三方库。

- 标准库就是安装 Python 时自带的库，可以直接使用。
- 第三方库是由第三方机构发布的，使用前需要安装。

6.2 Excel 读取库 xlrd

想用 Python **处理** Excel 数据，但是 Python 标准库中没有处理 Excel 文件的库，这时就需要安装第三方库。

> 如何使用 pip 在 Python 中进行库管理

`xlrd` 就是处理 Excel 文件的第三方库。下面介绍如何安装第三方库 `xlrd`。

> 不同的 IDE 和环境中有不同的安装方法，这个要根据环境的不同区别对待。

注意: 下面这条语句是为了演示一个错误的用法，你完全可以不运行下面的语句，而直接跳过下一句，到按照版本安装的方法。

```
1 pip install xlrd
```

```
import xlrd # 导入xlrd
wb = xlrd.open_workbook(r'./1-Scores.xlsx') # 读取工作簿对象。
```

```
---------------------------------------------------------------------------
XLRDError                                 Traceback (most recent call last)
<ipython-input-5-cd3c67ea92c8> in <module>
      1 import xlrd # 导入xlrd
----> 2 wb = xlrd.open_workbook(r'./1-Scores.xlsx') # 读取工作簿对象。

~/.local/lib/python3.6/site-packages/xlrd/__init__.py in open_workbook(filename, logfile, v
erbosity, use_mmap, file_contents, encoding_override, formatting_info, on_demand, ragged_ro
ws, ignore_workbook_corruption)
    168     # files that xlrd can parse don't start with the expected signature.
    169     if file_format and file_format != 'xls':
--> 170         raise XLRDError(FILE_FORMAT_DESCRIPTIONS[file_format]+'; not supported')
    171
    172     bk = open_workbook_xls(

XLRDError: Excel xlsx file; not supported
```

原因是最近 xlrd 更新到了 2.0.1 版本，只支持.xls 文件。

可以安装旧版 xlrd：

```
pip uninstall xlrd
```

```
pip install xlrd==1.2.0
```

6.2.1 xlrd 模块导入

安装好 **xlrd** 库后，要使用库中的功能，就需要导入 **xlrd**。

可以使用 **import** 语句来完成导入。导入的方法有以下两种。

- 方法 1: import 模块名1 [as 别名1] , 模块名2 [as 别名2]
  - 使用这种语法格式的 **import** 语句，可以导入库中的所有成员(变量、函数、类等）。
  - 如果模块名称太长，也可以用“**as 别名**”的方式来重命名。
  - 代码 `import xlrd` 将导入 **xlrd** 库的全部，这样就可以使用 **xlrd** 库中的所有功能了。
- 方法 2: from 模块名 import 成员名1 [as 别名1] , 成员名2 [as 别名2]
  - 使用这种语法格式的 **import** 语句，只导入库中指定的模块，而不是全部成员。
  - 代码 `from xlrd import open_workbook as owk` 则导入 xlrd 库中的 **open_workbook** 函数，别名为 **owk**，后面再次使用 **open_workbook** 函数时，可以使用 **owk** 来代替它。

```
import xlrd # 导入 xlrd
from xlrd import open_workbook as owk # 导入 xlrd 中的 open_workbook 函数
```

6.2.2 读取 Excel 工作表

Excel 的位置在不同的系统中有不同的表示：

- Mac 中：/Users/linken/Desktop/PythonExcelOperation
- Windows 中：C:\Desktop\
- 网络地址：https://github.com/WangLaoShi/Aliyun-Python/blob/main/Files/1-Scores.xlsx?raw=true

> **斜杠的位置不同**

```
import xlrd # 导入 xlrd
wb = xlrd.open_workbook(r'1-Scores.xlsx') # 读取工作簿对象。
```

上面的代码是读取加载 Excel 文件，除了这个还可以实现很多关于 Excel 文件的操作。

```
import xlrd # 导入 xlrd
xlrd.xlsx.ensure_elementtree_imported(False, None)
xlrd.xlsx.Element_has_iter = True
filePath = r'1-Scores.xlsx'

# 读取工作簿对象。
wb = xlrd.open_workbook(filePath)

# 读取工作簿下的所有工作表对象
all_ws1 = wb.sheets()

# 读取工作簿下的所有工作表对象的名称
all_ws2 = wb.sheet_names()

# 用索引值读取工作表 - 方法 1
ws1 = wb.sheet_by_index(0)

# 用索引值读取工作表 - 方法 2
ws2 = wb.sheets()[1]

# 用索引值读取工作表 - 方法 3
ws3 = wb.sheet_by_name('雪豹队')

# 直接通过工作簿读取工作表对象
ws4 = xlrd.open_workbook(filePath).sheet_by_name('飞龙队')
```

6.2.3 读取 Excel 行、列、单元格信息

```
import xlrd # 导入 xlrd

# 读取工作簿对象。
```

```
filePath = r'1-Scores.xlsx'
wb = xlrd.open_workbook(filePath)

# 读取工作表对象。
ws = wb.sheet_by_name('飞龙队')

# 返回工作表中已使用的行数。
row_count = ws.nrows

# 返回工作表中已使用的列数。
col_count = ws.ncols

# 返回工作表中指定行已使用的单元格对象。
row_obj = ws.row(1)

# 返回工作表中指定行已使用的单元格的值。
row_val = ws.row_values(1)

# 返回工作表中指定列已使用的单元格对象。
col_obj = ws.col(0)

# 返回工作表中指定列已使用的单元格的值。
col_val = ws.col_values(0)

# 返回工作表中指定行、列交叉单元格对象。
cell_obj = ws.cell(3,1)

# 返回工作表中指定行、列交叉单元格的值。
cell_val = ws.cell_value(3,1)
```

6.3 Excel 写入库 xlwt

前面介绍的 **xlrd** 库只能**读取** Excel 文件中的数据，如果要往 Excel 文件中**写入**数据，xlrd 库则**不**具备这个功能，需要安装 **xlwt** 库。

xlwt 库具有创建工作簿、工作表，以及将数据写入单元格的功能。

```
pip install xlwt
```

6.3.1 新建工作簿、新建工作表和写入数据

> Python 需要有写入文件权限，特别是 Linux 和 Mac 中

```
import xlwt # 导入 xlwt

# 新建工作簿
nwb = xlwt.Workbook('utf-8')

```

```
# 在新建工作簿中创建工作表
nws = nwb.add_sheet('工资表')

# 在工作表的指定单元格写入值
nws.write(0,0,'张三: 9000 元')

# 保存工作簿
nwb.save('工资表.xls')
```

6.4 Excel 修改库 xlutils

xlrd 库只能用于**读取**已经存在的 Excel 工作簿、工作表、单元格等相关信息，而 **xlwt** 库**只能新建**工作簿、新建工作表、将数据写入单元格等，没有办法对现有的工作簿进行**修改**。

要实现修改功能，就需要安装 **xlutils** 库，相当于在 **xlrd** 库和 **xlwt** 库之间建立起一座桥梁。但要注意，使用 xlutils 库一定要先安装 **xlrd** 库和 **xlwt** 库，否则安装 **xlutils** 库没有意义。

```
pip install xlutils
```

```
import xlrd # 导入 xlrd
from xlutils.copy import copy # 导入 xlutils 下 copy 模块下的 copy 函数

filePath = r'2-Salary.xls'

# 读取工作簿
wb = xlrd.open_workbook(filePath)

# 复制工作簿
nwb = copy(wb)

# 用索引号读取工作簿中的工作表
ws1 = nwb.get_sheet(0)

# 用名称读取工作簿中的工作表
ws2 = nwb.get_sheet('工资表')

# 在工作簿中新建工作表
ws3 = nwb.add_sheet('汇总表')

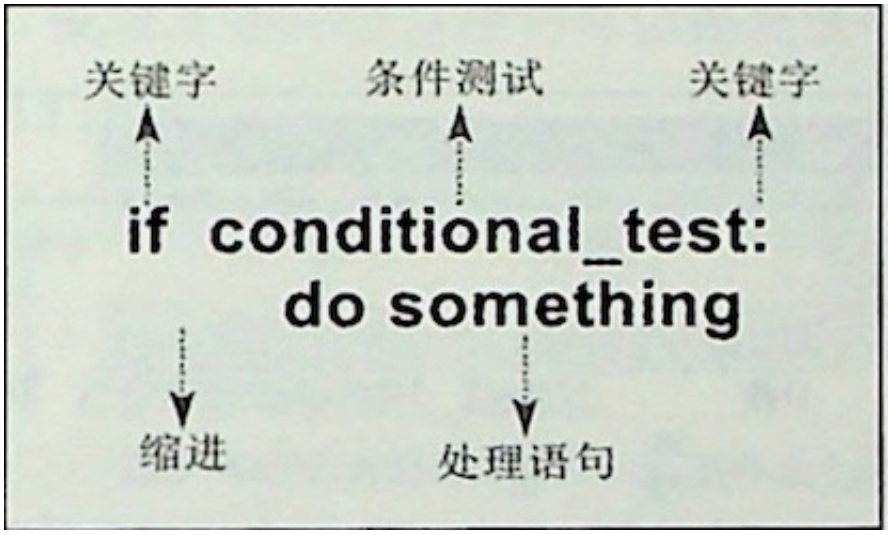
# 将数据写入单元格
ws3.write(0,0,'总计')

# 将数据写入单元格
ws3.write(0,1,12000)

# 保存工作簿
nwb.save('2-Salary-2.xls')
```

7. 条件语句

7.1 if 语句



- `if` 是关键字，固定写法，不能有任何变化。
- `conditional test` 是条件测试表达式，返回的结果是逻辑值(True 或 False）。
- `:` 是关键字，固定写法，不能有任何变化。
- `do something` 是条件测试表达式返回的值为 True 时执行的处理语句。

```
if expression:
    expr_true_suite
```

- if 语句的 `expr_true_suite` 代码块只有当条件表达式 `expression` 结果为真时才执行，否则将继续执行紧跟在该代码块后面的语句。
- 单个 if 语句中的 `expression` 条件表达式可以通过布尔操作符 `and`， `or` 和 `not` 实现多重条件判断。

7.1.1【例子】if 标准用法

```
if 2 > 1 and not 2 > 3:
    print('Correct Judgement!')

# Correct Judgement!
```

7.1.2【例子】if 标准用法

```
if 100>=90: # 条件成立
    print('优秀') # 执行处理语句
if 80>=90: # 条件不成立
    print('优秀') # 不执行处理语句
```

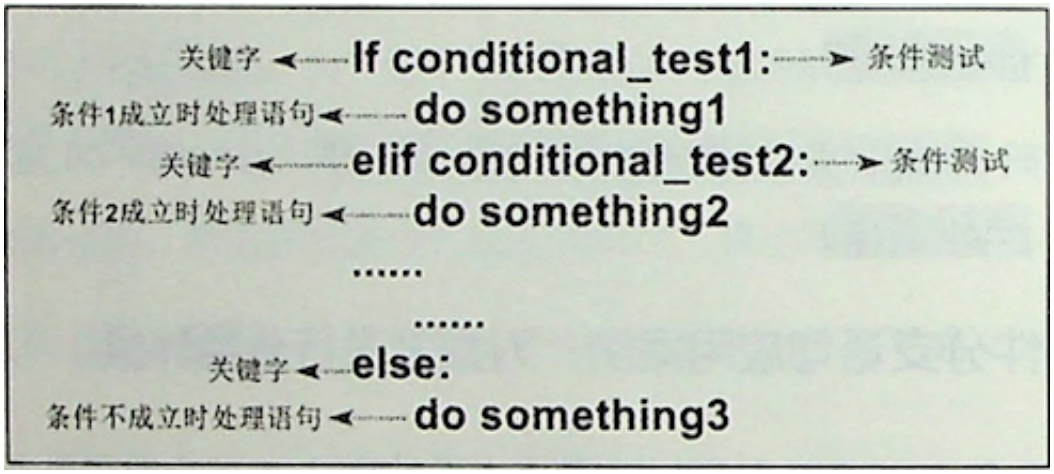
7.1.3【综合案例】根据分数判断等级

```
import xlrd # 导入 xlrd 库。
wb = xlrd.open_workbook('./3-Scores.xlsx') # 读取工作簿。
ws = wb.sheet_by_name('成绩表') # 读取工作表。
col_vals = ws.col_values(1) # 获取工作表指定列已用单元格区域的值。
for score in col_vals: # 循环列区域的每个值。
    if type(score)==float and score>=90: # 判断条件表达式是否成立。
        print(score,'优秀') # 条件成立，则执行。

```

- 第 1~4 行代码为数据的读取做准备工作，将工作表中分数列的所有数据读取到 **col_vals** 变量中。
- 第 5 行代码 `for score in col_vals`:，将 col_vals 变量中的每个分数循环读取给 score 变量。
- 第 6 行代码 `type(score) == float and score>=90:`，在 for 循环体中，首先判断 score 变量中的分数是否是 float 类型。为什么要做类型判断？B 列的第 1 个值“分数”是汉字，并不是数字，因此要做类型判断，是数字类型并且分数大于或等于 90，才能执行 `print(score,'优秀')` 语句。
- 第 7 行代码 `print(score, "优秀")`，如果第 6 行的条件成立，则执行该行代码。

7.2 if - else 分支语句




```
if expression:
    expr_true_suite
else:
    expr_false_suite
```

- Python 提供与 if 搭配使用的 else，如果 if 语句的条件表达式结果布尔值为假，那么程序将执行 else 语句后的代码。

7.2.1【例子】猜一猜小姐姐想的是哪个数字?

```
temp = input("猜一猜小姐姐想的是哪个数字？ ")
guess = int(temp) # input 函数将接收的任何数据类型都默认为 str。
if guess == 666:
    print("你太了解小姐姐的心思了！ ")
    print("哼，猜对也没有奖励！ ")
else:
    print("猜错了，小姐姐现在心里想的是 666！ ")
print("游戏结束，不玩儿啦！ ")
```

`if` 语句支持嵌套，即在一个 `if` 语句中嵌入另一个 `if` 语句，从而构成不同层次的选择结构。

7.2.2【例子】`else` 的缩进问题。

Python 使用缩进而不是大括号来标记代码块边界，因此要特别注意 `else` 的缩进问题。

```
hi = 6
if hi > 2:
    if hi > 7:
        print('好棒！ 好棒！')
else:
    print('切 ~')

# 无输出
```

7.2.3【例子】猜一猜小姐姐想的是哪个数字？

```
temp = input("猜一猜小姐姐想的是哪个数字？ ")
guess = int(temp)
if guess > 8:
    print("大了，大了")
else:
    if guess == 8:
        print("你太了解小姐姐的心思了！ ")
        print("哼，猜对也没有奖励！ ")
    else:
        print("小了，小了")
print("游戏结束，不玩儿啦！ ")
```

7.3 if - elif - else 分支语句

```
if expression1:
    expr1_true_suite
elif expression2:
    expr2_true_suite
    .
    .
```

```
elif expressionN:
    exprN_true_suite
else:
    expr_false_suite
```

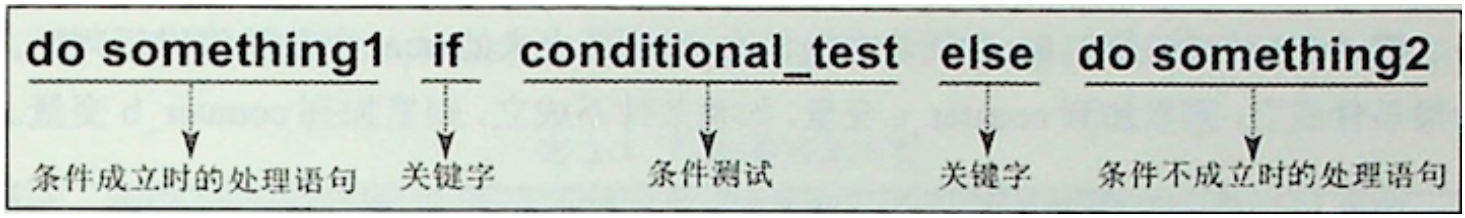
- elif 语句即为 else if，用来检查多个表达式是否为真，并在为真时执行特定代码块中的代码。

【例子】

```
temp = input('请输入成绩:')
source = int(temp)
if 100 >= source >= 90:
    print('A')
elif 90 > source >= 80:
    print('B')
elif 80 > source >= 60:
    print('C')
elif 60 > source >= 0:
    print('D')
else:
    print('输入错误! ')
```

7.4 if 条件分支语句单行写法

实际上，if···else···条件分支语句可以写在一行，也叫**三目运算**。如果判断后的处理语句不太复杂，则可以使用这种写法。




```
print('优秀') if 100>=90 else print('普通') #当前条件成立，在屏幕上打印 " 优 秀”
print('优秀') if 80>=90 else print(' 普通 ') #当前条件不成立，在屏幕上打印 " 普 通 "
```

- 第 1 行代码 print('优秀') if 100>=90 else print(' 普通 ')，因为 100>=90 是成立的，所以会执行代码 print(' 优秀')。
- 第 2 行代码 print('优秀') if 80>=90 else print(' 普通 ')，因为 80>=90 是不成立的，所以会执行代码 print(' 普通')。

7.5 assert 关键词

- `assert` 这个关键词我们称之为“断言”，当这个关键词后边的条件为 **False** 时，程序自动崩溃并抛出 `AssertionError` 的异常。

【例子】

```
my_list = ['lsgogroup']
my_list.pop(0)
assert len(my_list) > 0

# AssertionError
```

【例子】在进行单元测试时，可以用来在程序中置入检查点，只有条件为 True 才能让程序正常工作。

```
assert 3 > 7

# AssertionError
```

7.6 案例：对数字进行分类计数

统计［95，89，69，100，88，94，91］列表中数字大于或等于 90 的个数和数字小于 90 的个数。

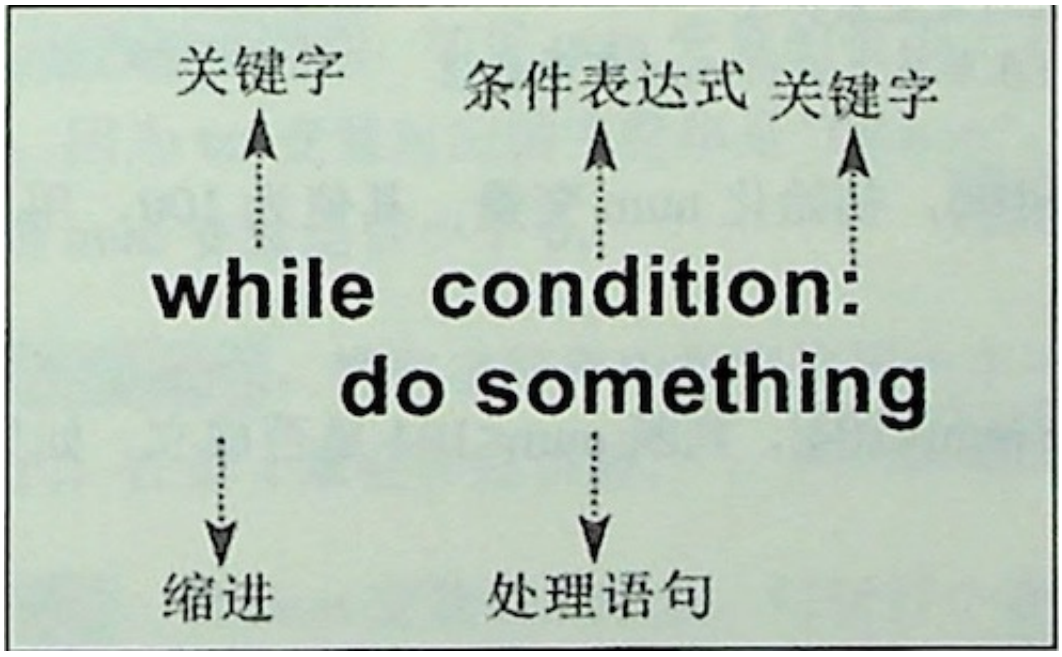
```
lst=[95,89,69,100,88,94,91] # 要被循环判断的数字列表。
counter_a,counter_b=0,0 # 分别初始化 counter_a 和 counter_b 变量值为 0。
for num in lst: # 循环 lst 变量中的数字。
    if num>=90: # 如果 num 大于等于 90。
        counter_a +=1 # 条件成立则将计数到 counter_a 变量中。
    else: # 如果 num 不大于等于 90。
        counter_b +=1# 条件不成立则将计数到 counter_b 变量中。
print('>=90 有 {} 个，<90 有 {} 个。'.format(counter_a,counter_b))# 统计>=90 与<90 的数字个数。
```

- 第 1 行代码 lst＝［95,89,69,100,88,94,91］，将列表赋值给 lst 变量。
- 第 2 行代码 counter_a，counter_b＝0,0，分别初始化 counter_a 和 counter_b，用于在后面做判断时存储条件成立的次数和条件不成立的次数。
- 第 3 行代码 for num in lst，循环读取 lst 列表中的数字，赋值给 num 变量。
- 第 4～7 行代码，是在 for 循环体中执行的，对循环出来的 num 变量的值进行判断，如果条件成立，则累加到 counter_a 变量，如果条件不成立，则累加到 counter_b 变量。
- 第 8 行代码 print('>=90 有{}个，<90 有 {} 个。'.format(counter_a，counter_b))，在屏幕上输出统计结果，最后返回的结果为“>=90 有 4 个，＜90 有 3 个。”

8. 循环语句

8.1 while 循环

`while` 语句最基本的形式包括一个位于顶部的布尔表达式，一个或多个属于 `while` 代码块的缩进语句。



- `while` 是关键字，固定写法，不能有任何变化。
- `condition` 是条件表达式，需要返回一个逻辑值 True 或 False。
- `:` 是关键字，可以理解为到此为止，后面不能再加任何代码，也是换行的标志。
- `do something` 是 while 循环的处理语句，具体如何处理、代码如何编写要根据特定的需求而定。这些处理语句都必须做缩进处理，不能与 while 关键字对齐。

```
while 布尔表达式:
    代码块
```

- `while` 循环的代码块会一直循环执行，直到布尔表达式的值为布尔假。
- 如果布尔表达式不带有 `<、>、==、！=、in、not in` 等运算符，仅仅给出数值之类的条件，也是可以的。
- 当 `while` 后写入一个非零整数时，视为真值，执行循环体；写入 `0` 时，视为假值，不执行循环体。
- 也可以写入 `str、list` 或任何序列，长度非零则视为真值，执行循环体；否则视为假值，不执行循环体。

```
num = 100 # 初始化 num 变量的值为 100
while num<104: # 当 num 小于 104 时循环
    num +=1 # 每次对变量累加 1
    print(num) # 在屏幕打印 num 变量的值
```

- 第 1 行代码 `num =100`，初始化 num 变量，其值为 100，用于在 while 循环体中做累加赋值运

算。

- 第 2 行代码 `while num<104:`，判断 num<104 是否成立，如果条件成立，则执行循环体中的语句。
- 第 3 行代码 `num +=1`，对 num 变量每次加 1，然后赋值给 num 变量。由于此行代码是在循环体内的，所以 while 循环体循环多少次就累加赋值多少次。
- 第 4 行代码 `print(num)`，在屏幕上输出 num 变量的值，该行代码是在循环体内的，所以打印输出多少次取决于该循环体循环多少次。

8.1.1 【例子】猜一猜小姐姐想的是哪个数字?

```
count = 0
while count < 3:
    temp = input("猜一猜小姐姐想的是哪个数字? ")
    guess = int(temp)
    if guess > 8:
        print("大了, 大了")
    else:
        if guess == 8:
            print("你太了解小姐姐的心思了! ")
            print("哼, 猜对也没有奖励! ")
            count = 3
        else:
            print("小了, 小了")
    count = count + 1
print("游戏结束, 不玩儿啦! ")
```

8.1.2【例子】循环字符串

```
txt =  'Python' # 要被循环的字符串。
num = 0 # 初始化 num 变量为 0。
while num<len(txt): # num 变量小于 txt 的字符长度时则开始循环。
    print(txt[num]) # 打印提取的每个字符。
    num += 1 # 对 num 变量进行累加。
```

8.1.3【例子】布尔表达式返回 0，循环终止。

```
string = 'abcd'
while string:
    print(string)
    string = string[1:]

# abcd
# bcd
# cd
# d
```

8.2 while - else 循环

```
while 布尔表达式:
    代码块
else:
    代码块
```

当 `while` 循环正常执行完的情况下，执行 `else` 输出，如果 `while` 循环中执行了跳出循环的语句，比如 `break`，将不执行 `else` 代码块的内容。

8.2.1【例子】比 5 大，还是比 5 小

```
count = 0
while count < 5:
    print("%d is  less than 5" % count)
    count = count + 1
else:
    print("%d is not less than 5" % count)

# 0 is  less than 5
# 1 is  less than 5
# 2 is  less than 5
# 3 is  less than 5
# 4 is  less than 5
# 5 is not less than 5
```

8.2.2【例子】增加一个 break 会怎么样

```
count = 0
while count < 5:
    print("%d is  less than 5" % count)
    count = 6
    break
else:
    print("%d is not less than 5" % count)

# 0 is  less than 5
```

8.3 【综合训练】使用 while 循环批量创建 Excel

```
import xlwt # 导入 xlwt 库。
wb = xlwt.Workbook('utf-8') # 新建工作簿。
year_num = 2010 # 初始化 year_num 变量为 2010。
while year_num < 2020: # 如果 year_num 变量小于 2020 则执行 while 循环体的语句。
```

```
    txt = '{}年业绩表'.format(year_num) # 使用变量 year_num 格式化为工作表名称。
    wb.add_sheet(txt) # 新建工作表。
    year_num += 1 # 累加 year_num 变量。
wb.save('./PerformanceWorksheet.xls') # 保存工作簿。
```

- 第 1 行代码 import xlwt，导入需要用到的 xlwt 库。
- 第 2 行代码 wb = xlwt. Workbook('utf-8')，新建一个工作簿并赋值给 wb 变量，此工作簿在循环新建工作表时使用。
- 第 3 行代码 year_num=2010，初始化 year_num 变量，其值为 2010，可以将此值看作年份。
- 第 4 行代码 while year_num<2020:，如果 year_num 变量的值小于 2020，则执行 while 循环体的语句。year_num 变量从 2010 累加到 2019，条件都成立。
- 第 5~7 行代码都属于 while 循环体中的语句。
  - 第 5 行代码 以 txt = '{}年业绩表'.format(year_num)，是将 year_num 变量格式化为工作表名称。
  - 第 6 行代码 wb.add_sheet(txt)，用来新建工作表，并将 txt 变量的值作为新建的工作表名称。
  - 第 7 行代码 year_num +=1，对 year_num 变量进行累加，每次加 1。
- 第 8 行代码 wb.save('./PerformanceWorksheet.xls')，当 while 循环结束后，执行工作簿的保存操作。注意，不要将此行代码放在 while 循环体中，否则循环一次就保存一次。

8.4 for 循环

`for` 循环是迭代循环，在 Python 中相当于一个通用的序列迭代器，可以遍历任何有序序列，如 `str、list、tuple` 等，也可以遍历任何可迭代对象，如 `dict` 。



- `for` 是关键字，固定写法，不能有任何变化。
- `item` 是元素，相当于变量名称，用户可自行命名，用于接收从 **iterable** 中循环出来的元素。
- `in` 是关键字，固定写法，不能有任何变化。

- `iterable` 是迭代器，类似于一个容器，表达形式由用户自行定义，从该迭代器中读取出的数据将传给 item 变量。
- `:` 也是关键字，可以理解为到此为止，后面不能再加任何代码，也是换行的标志。
- `do something` 是 for 循环的处理语句，具体如何处理、代码如何编写等要根据特定的需求而定。处理语句必须做缩进处理，不能与上面的 for 语句对齐。

```
for 迭代变量 in 可迭代对象:
    代码块
```

每次循环，迭代变量被设置为可迭代对象的当前元素，提供给代码块使用。

8.4.1 【例子】循环字符串

```
for i in 'ILoveLSGO':
    print(i, end='')  # end ='' 表示不换行输出

# I L o v e L S G O
```

8.4.2【例子】循环列表（直接法和通过长度）

```
member = ['林肯', '杨小辉', '谷爱凌', '苏翊鸣', '王菲']
for each in member:
    print(each)

for i in range(len(member)):
    print(member[i])
```

8.4.3【例子】循环访问字典（使用 items）

```
dic = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
for key, value in dic.items():
    print(key, value, sep=':', end=' ')
# a:1 b:2 c:3 d:4
```

8.4.4【例子】循环访问字典（使用 keys）

```
dic = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

for key in dic.keys():
    print(key, end=' ')

# a b c d
```

8.4.5【例子】循环访问字典（使用 values）

```
dic = {'a': 1, 'b': 2, 'c': 3, 'd': 4}

for value in dic.values():
    print(value, end=' ')

# 1 2 3 4
```

8.4.6【例子】循环序列数（range）

循环序列数是常见的一种循环方式，下面介绍循环指定范围内的序列数。

要指定序列数的范围，可以使用 Python 中的 range 函数。

- 函数语法：
  - `range(start,stop,step)`
- 参数说明：
  - start: 表示起始值，默认从 0 开始。例如，range(5）等价于 range(0,5）。
  - stop: 表示终止值，但不包括终止值。例如，range(0,5）的返回值是 0、1、2、3、4，不包括 5。
  - step: 步长，默认为 1。例如，range(0,5）等价于 range(0,5,1），返回值是 0、1、2、3、4。如果是 range(0,5,2），则返回值是 0、2、4。

8.4.7【例子】循环批量创建 Excel 文件

```
import xlwt # 导入 xlwt
for month_num in range(1,13): # 循环 1 到 13 之间的数字
    month_name = '{}月份.xls'.format(month_num) # 格式化数字为工作簿名
    nwb = xlwt.Workbook('utf-8') # 新建工作簿
    nwb.add_sheet(month_name) # 在工作簿中新建工作表
    nwb.save(month_name) # 保存工作簿
```



8.5 for 循环嵌套语句

for 循环语句在某些时候需要进行多层嵌套。下面以打印乘法表为例，来讲解 **for** 循环语句的嵌套用法。

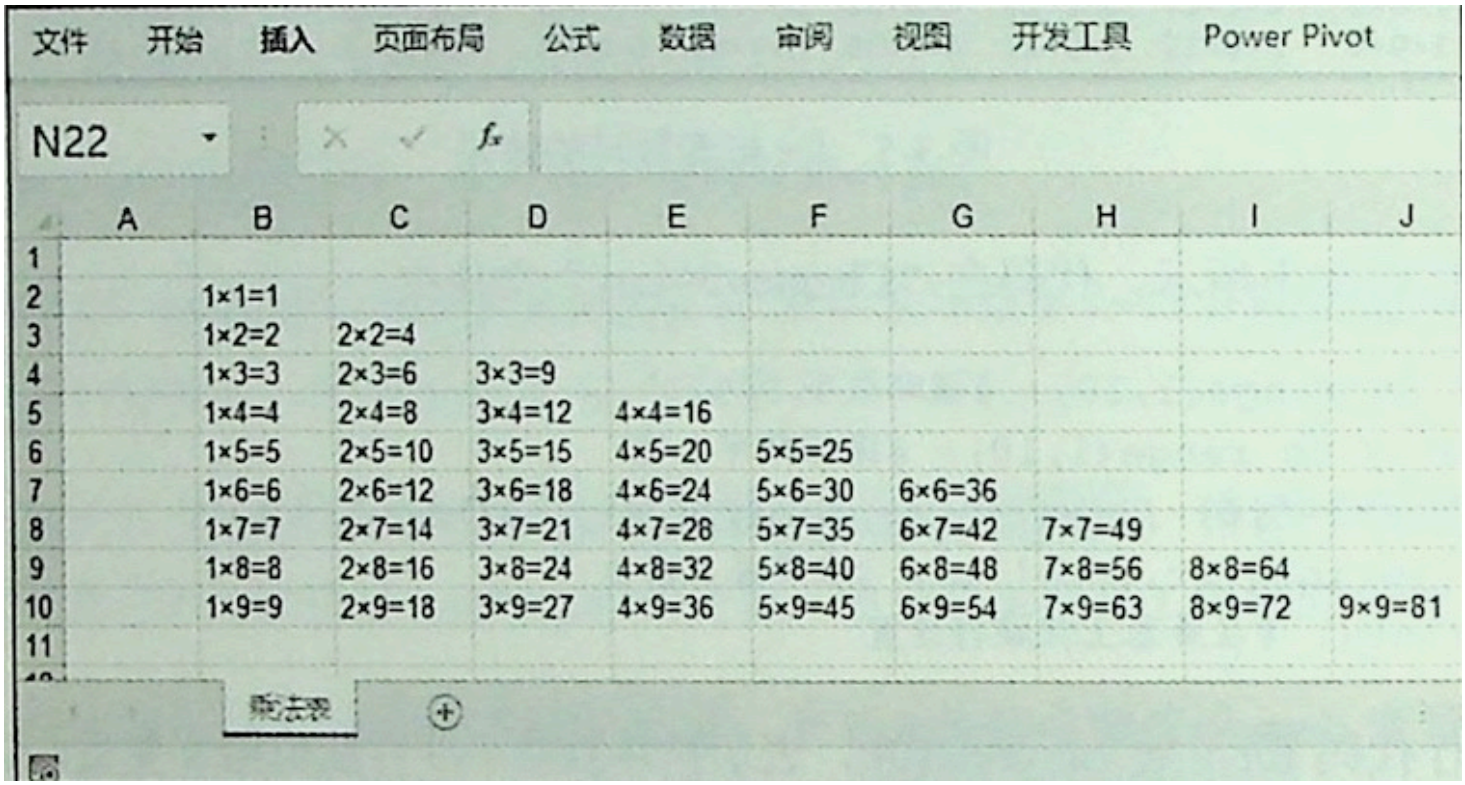
```
1 for x in range(1,10): # 循环数字 1~9
2     for y in range(1,10): # 循环数字 1~9
3         txt='{0}×{1}={2}'.format(y,x,x*y) # 格式化乘法表
4         print(txt,end='\t') # 在屏幕打印
5     print() # 在屏幕做换行设置
```

```
1×1=1   2×1=2   3×1=3   4×1=4   5×1=5   6×1=6   7×1=7   8×1=8   9×1=9
1×2=2   2×2=4   3×2=6   4×2=8   5×2=10  6×2=12  7×2=14  8×2=16  9×2=18
1×3=3   2×3=6   3×3=9   4×3=12  5×3=15  6×3=18  7×3=21  8×3=24  9×3=27
1×4=4   2×4=8   3×4=12  4×4=16  5×4=20  6×4=24  7×4=28  8×4=32  9×4=36
1×5=5   2×5=10  3×5=15  4×5=20  5×5=25  6×5=30  7×5=35  8×5=40  9×5=45
1×6=6   2×6=12  3×6=18  4×6=24  5×6=30  6×6=36  7×6=42  8×6=48  9×6=54
1×7=7   2×7=14  3×7=21  4×7=28  5×7=35  6×7=42  7×7=49  8×7=56  9×7=63
1×8=8   2×8=16  3×8=24  4×8=32  5×8=40  6×8=48  7×8=56  8×8=64  9×8=72
1×9=9   2×9=18  3×9=27  4×9=36  5×9=45  6×9=54  7×9=63  8×9=72  9×9=81
```

- 第 1 行代码 for x in range(1,10) ；，表示循环数字 1~9 并赋值给x变量，这是外层 for 循环。

- 第 2 行代码 for y in range(1,10），表示循环数字 1~9 并赋值给 y 变量，这是内层 for 循环。注意此循环是嵌套在外层循环中的，也就是说，外层 for 循环执行 1 次，内层 for 循环要执行 9 次。外层 for 循环总共需要执行 9 次，内层 for 循环要执行 81 次。属于内层 for 循环的第 3 行代码和第 4 行代码要执行 81 次。
- 第 3 行代码 txt = ('{0}×{1}={2}'.format(y,x,x*y)，使用 format 函数格式化两个乘数 x 和 y，以及乘积结果 x＊y，然后赋值给 txt 变量。
- 第 4 行代码 print(uxt,end=t），使用 print 函数将 tt 变量中的值输出到屏幕，end='\t' 表示以制表符结束，如果不特别指定，则默认以回车符结束。
- 第 5 行代码 print()，此行代码是与内层 for 循环语句同级的，属于外层 for 循环下的语句，它会执行 9 次，而不是执行 81 次。这行代码在每次内层 for 循环完成后，加一个回车符以起到换行的作用。前面说过 print 函数默认以回车符结束，所以没在 print 函数中做任何改动。

8.6 【综合训练】for 使用嵌套循环输出九九乘法表并保存在 Excel 中




```
import xlwt # 导入 xlwt 库
nwb=xlwt.Workbook('utf-8') # 新建工作簿
nws=nwb.add_sheet('乘法表') # 新建工作表
for x in range(1,10): # 循环数字 1 到 9
    for y in range(1,x+1): # 循环数字 1 到 x+1
        txt='{0}×{1}={2}'.format(y,x,x*y) # 格式化乘法公式
        nws.write(x,y,txt) # 将乘法公式写入单元格
nwb.save('./99Mul.xlsx') # 保存工作簿

```

- 第 1 行代码 import xlwt，导入需要用到的 xlwt 库。
- 第 2 行代码和第 3 行代码 nwb-xlwt. Workbook(utf-8）和 nws-nwb.add_ sheet(乘法表），

分别新建工作簿和在工作簿中新建工作表。

- 第 4 行代码 for x in range(1，10）:，这是外层 for 循环，循环数字 1~9，然后将循环出来的数字赋值给x变量。
- 第 5 行代码 for y in range(1，x+1）:，这是内层 for 循环，其中 x+1 表示终止循环的数字。如果外层 for 循环执行 1 次，则内层 for 循环为 range(1，1+1 次，即只能循环 1 次：如果外层 for 循环执行 2 次，则内层 for 循环为 range(1，2+1），即内层循环只能循环 2 次，以此类推。
- 第 6 行代码 txt ='{0} X {1}= {2}'.format(y，x，x＊y), 格式化循环出来的数字，组成乘法公式并赋值给 txt 变量。
- 第 7 行代码 nws.write(x，y，txt），将第 6 行代码中 txt 变量的值写入工作表中对应的单元格。
- 第 8 行代码 nwb.save('./99Mul.xlsx')，完成乘法表的写入后进行保存。注意，此行代码与外层 for 循环语句是同级的，因此该行代码只执行 1 次。如果将其放在外层 for 循环体中，就会执行 9 次：如果将其放在内层 for 循环体中，就会执行 45 次。虽然不影响结果，但更耗资源。

8.7 for - else 循环

```
for 迭代变量 in 可迭代对象:
    代码块
else:
    代码块
```

当 `for` 循环正常执行完的情况下，执行 `else` 输出，如果 `for` 循环中执行了跳出循环的语句，比如 `break`，将不执行 `else` 代码块的内容，与 `while – else` 语句一样。

【例子】

```
for num in range(10, 20):  # 迭代 10 到 20 之间的数字
    for i in range(2, num):  # 根据因子迭代
        if num % i == 0:  # 确定第一个因子
            j = num / i  # 计算第二个因子
            print('%d 等于 %d * %d' % (num, i, j))
            break  # 跳出当前循环
    else:  # 循环的 else 部分
        print(num, '是一个质数')

# 10 等于 2 * 5
# 11 是一个质数
# 12 等于 2 * 6
# 13 是一个质数
# 14 等于 2 * 7
# 15 等于 3 * 5
# 16 等于 2 * 8
# 17 是一个质数
# 18 等于 2 * 9
```

```
# 19 是一个质数
```

8.8 enumerate()函数

```
enumerate(sequence, [start=0])
```

- `sequence`：一个序列、迭代器或其他支持迭代对象。
- `start`：下标起始位置。
- 返回 enumerate(枚举) 对象

【例子】

```
seasons = ['Spring', 'Summer', 'Fall', 'Winter']
lst = list(enumerate(seasons))
print(lst)
# [(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
lst = list(enumerate(seasons, start=1))  # 下标从 1 开始
print(lst)
# [(1, 'Spring'), (2, 'Summer'), (3, 'Fall'), (4, 'Winter')]
```

`enumerate()` 与 for 循环的结合使用。

```
for i, a in enumerate(A)
    do something with a
```

用 `enumerate(A)` 不仅返回了 `A` 中的元素，还顺便给该元素一个索引值 (默认从 0 开始)。此外，用 `enumerate(A, j)` 还可以确定索引起始值为 `j` 。

【例子】

```
languages = ['Python', 'R', 'Matlab', 'C++']
for language in languages:
    print('I love', language)
print('Done!')
# I love Python
# I love R
# I love Matlab
# I love C++
# Done!


for i, language in enumerate(languages, 2):
    print(i, 'I love', language)
print('Done!')
# 2 I love Python
```

```
# 3 I love R
# 4 I love Matlab
# 5 I love C++
# Done!
```

8.9 break 语句

`break` 语句可以跳出当前所在层的循环。

【例子】

```
import random
secret = random.randint(1, 10) #[1,10]之间的随机数

while True:
    temp = input("猜一猜小姐姐想的是哪个数字? ")
    guess = int(temp)
    if guess > secret:
        print("大了, 大了")
    else:
        if guess == secret:
            print("你太了解小姐姐的心思了! ")
            print("哼, 猜对也没有奖励! ")
            break
        else:
            print("小了, 小了")
print("游戏结束, 不玩儿啦! ")
```

8.10 continue 语句

`continue` 终止本轮循环并开始下一轮循环。

【例子】

```
for i in range(10):
    if i % 2 != 0:
        print(i)
        continue
    i += 2
    print(i)

# 2
# 1
# 4
# 3
```

```
# 6
# 5
# 8
# 7
# 10
# 9
```

---

9. pass 语句

`pass` 语句的意思是“不做任何事”，如果你在需要有语句的地方不写任何语句，那么解释器会提示出错，而 `pass` 语句就是用来解决这些问题的。

【例子】

```
def a_func():

# SyntaxError: unexpected EOF while parsing
```

【例子】

```
def a_func():
    pass
```

`pass` 是空语句，不做任何操作，只起到占位的作用，其作用是为了保持程序结构的完整性。尽管 `pass` 语句不做任何操作，但如果暂时不确定要在一个位置放上什么样的代码，可以先放置一个 `pass` 语句，让代码可以正常运行。

---

10. 推导式

10.1 列表推导式

```
[expr for value in collection [if condition] ]
```

【例子】

```
x = [-4, -2, 0, 2, 4]
y = [a * 2 for a in x]
print(y)
# [-8, -4, 0, 4, 8]
```

【例子】

```
x = [i ** 2 for i in range(1, 10)]
print(x)
# [1, 4, 9, 16, 25, 36, 49, 64, 81]
```

【例子】

```
x = [(i, i ** 2) for i in range(6)]
print(x)

# [(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

【例子】

```
x = [i for i in range(100) if (i % 2) != 0 and (i % 3) == 0]
print(x)

# [3, 9, 15, 21, 27, 33, 39, 45, 51, 57, 63, 69, 75, 81, 87, 93, 99]
```

【例子】

```
a = [(i, j) for i in range(0, 3) for j in range(0, 3)]
print(a)

# [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

【例子】

```
x = [[i, j] for i in range(0, 3) for j in range(0, 3)]
print(x)
# [[0, 0], [0, 1], [0, 2], [1, 0], [1, 1], [1, 2], [2, 0], [2, 1], [2, 2]]

x[0][0] = 10
print(x)
# [[10, 0], [0, 1], [0, 2], [1, 0], [1, 1], [1, 2], [2, 0], [2, 1], [2, 2]]
```

【例子】

```
a = [(i, j) for i in range(0, 3) if i < 1 for j in range(0, 3) if j > 1]
print(a)

# [(0, 2)]
```

10.2 元组推导式

```
(expr for value in collection [if condition] )
```

【例子】

```
a = (x for x in range(10))
print(a)

# <generator object <genexpr> at 0x0000025BE511CC48>

print(tuple(a))

# (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
<generator object <genexpr> at 0x0000014CEC2E28B8>
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

10.3 字典推导式

```
{key_expr: value_expr for value in collection [if condition] }
```

【例子】

```
b = {i: i % 2 == 0 for i in range(10) if i % 3 == 0}
print(b)
# {0: True, 3: False, 6: True, 9: False}
```

10.4 集合推导式

```
{expr for value in collection [if condition] }
```

【例子】

```
c = {i for i in [1, 2, 3, 4, 5, 5, 6, 4, 3, 2, 1]}
print(c)
# {1, 2, 3, 4, 5, 6}
```

```
{1, 2, 3, 4, 5, 6}
```

其它

- `next(iterator[, default])` Return the next item from the iterator. If default is given and the iterator is exhausted, it is returned instead of raising StopIteration.

【例子】

```
e = (i for i in range(10))
print(e)
# <generator object <genexpr> at 0x0000007A0B8D01B0>

print(next(e))  # 0
print(next(e))  # 1

for each in e:
    print(each, end=' ')

# 2 3 4 5 6 7 8 9
```

【例子】

```
s = sum([i for i in range(101)])
print(s)  # 5050
s = sum((i for i in range(101)))
print(s)  # 5050
```

11. 异常处理

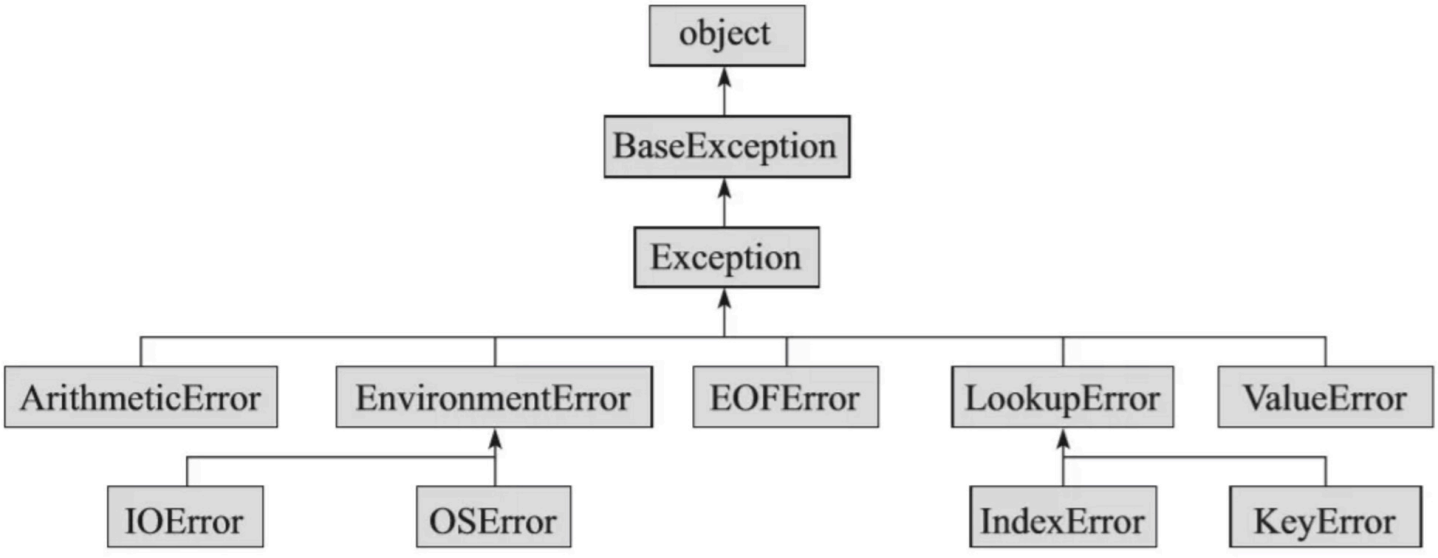
异常就是运行期检测到的错误。 计算机语言针对可能出现的错误定义了异常类型，某种错误引发对应的异常时，异常处理程序将被启动，从而恢复程序的正常运行。

11.1 Python 标准异常总结

- BaseException：所有异常的 ** 基类 **
- Exception：常规异常的 ** 基类 **
- StandardError：所有的内建标准异常的基类
- ArithmeticError：所有数值计算异常的基类
- FloatingPointError：浮点计算异常
- OverflowError：数值运算超出最大限制
- ZeroDivisionError：除数为零
- AssertionError：断言语句(assert) 失败
- AttributeError：尝试访问未知的对象属性
- EOFError：没有内建输入，到达 EOF 标记
- EnvironmentError：操作系统异常的基类
- IOError：输入 / 输出操作失败
- OSError：操作系统产生的异常(例如打开一个不存在的文件)

- WindowsError：系统调用失败
- ImportError：导入模块失败的时候
- KeyboardInterrupt：用户中断执行
- LookupError：无效数据查询的基类
- IndexError：索引超出序列的范围
- KeyError：字典中查找一个不存在的关键字
- MemoryError：内存溢出(可通过删除对象释放内存)
- NameError：尝试访问一个不存在的变量
- UnboundLocalError：访问未初始化的本地变量
- ReferenceError：弱引用试图访问已经垃圾回收了的对象
- RuntimeError：一般的运行时异常
- NotImplementedError：尚未实现的方法
- SyntaxError：语法错误导致的异常
- IndentationError：缩进错误导致的异常
- TabError：Tab 和空格混用
- SystemError：一般的解释器系统异常
- TypeError：不同类型间的无效操作
- ValueError：传入无效的参数
- UnicodeError：Unicode 相关的异常
- UnicodeDecodeError：Unicode 解码时的异常
- UnicodeEncodeError：Unicode 编码错误导致的异常
- UnicodeTranslateError：Unicode 转换错误导致的异常

异常体系内部有层次关系，Python 异常体系中的部分关系如下所示：



11.2 Python 标准警告总结

- Warning：警告的基类
- DeprecationWarning：关于被弃用的特征的警告
- FutureWarning：关于构造将来语义会有改变的警告
- UserWarning：用户代码生成的警告
- PendingDeprecationWarning：关于特性将会被废弃的警告
- RuntimeWarning：可疑的运行时行为 (runtime behavior) 的警告
- SyntaxWarning：可疑语法的警告
- ImportWarning：用于在导入模块过程中触发的警告
- UnicodeWarning：与 Unicode 相关的警告
- BytesWarning：与字节或字节码相关的警告
- ResourceWarning：与资源使用相关的警告

11.3 try - except 语句

```
try:
    检测范围
except Exception[as reason]:
    出现异常后的处理代码
```

try 语句按照如下方式工作：

- 首先，执行 `try` 子句(在关键字 `try` 和关键字 `except` 之间的语句)
- 如果没有异常发生，忽略 `except` 子句， `try` 子句执行后结束。
- 如果在执行 `try` 子句的过程中发生了异常，那么 `try` 子句余下的部分将被忽略。如果异常的类型和 `except` 之后的名称相符，那么对应的 `except` 子句将被执行。最后执行 `try – except` 语句之后的代码。
- 如果一个异常没有与任何的 `except` 匹配，那么这个异常将会传递给上层的 `try` 中。

【例子】

```
try:
    f = open('test.txt')
    print(f.read())
    f.close()
except OSError:
    print('打开文件出错')

# 打开文件出错
```

【例子】

```
try:
    f = open('test.txt')
    print(f.read())
    f.close()
except OSError as error:
    print('打开文件出错 \n 原因是: ' + str(error))

# 打开文件出错
# 原因是: [Errno 2] No such file or directory: 'test.txt'
```

一个 `try` 语句可能包含多个 `except` 子句，分别来处理不同的特定的异常。最多只有一个分支会被执行。

【例子】

```
try:
    int("abc")
    s = 1 + '1'
    f = open('test.txt')
    print(f.read())
    f.close()
except OSError as error:
    print('打开文件出错 \n 原因是: ' + str(error))
except TypeError as error:
    print('类型出错 \n 原因是: ' + str(error))
except ValueError as error:
    print('数值出错 \n 原因是: ' + str(error))

# 数值出错
# 原因是: invalid literal for int() with base 10: 'abc'
```

【例子】

```
dict1 = {'a': 1, 'b': 2, 'v': 22}
try:
    x = dict1['y']
except LookupError:
    print('查询错误')
except KeyError:
    print('键错误')
else:
    print(x)

# 查询错误
```

`try-except-else` 语句尝试查询不在 `dict` 中的键值对，从而引发了异常。这一异常准确地说应属于 `KeyError`，但由于 `KeyError` 是 `LookupError` 的子类，且将 `LookupError` 置于

`KeyError` 之前，因此程序优先执行该 `except` 代码块。所以，使用多个 `except` 代码块时，必须坚持对其规范排序，要从最具针对性的异常到最通用的异常。

【例子】

```
dict1 = {'a': 1, 'b': 2, 'v': 22}
try:
    x = dict1['y']
except KeyError:
    print('键错误')
except LookupError:
    print('查询错误')
else:
    print(x)

# 键错误
```

【例子】一个 `except` 子句可以同时处理多个异常，这些异常将被放在一个括号里成为一个元组。

```
try:
    s = 1 + '1'
    int("abc")
    f = open('test.txt')
    print(f.read())
    f.close()except (OSError, TypeError, ValueError) as error:
    print('出错了! \n 原因是: ' + str(error))

# 出错了!
# 原因是: unsupported operand type(s) for +: 'int' and 'str'
```

11.4 try - except - finally 语句

```
try:
    检测范围
except Exception[as reason]:
    出现异常后的处理代码
finally:
    无论如何都会被执行的代码
```

不管 `try` 子句里面有没有发生异常， `finally` 子句都会执行。

【例子】如果一个异常在 `try` 子句里被抛出，而又没有任何的 `except` 把它截住，那么这个异常会在 `finally` 子句执行后被抛出。

```
def divide(x, y):
    try:
```

```
        result = x / y
        print("result is", result)
    except ZeroDivisionError:
        print("division by zero!")
    finally:
        print("executing finally clause")


divide(2, 1)
# result is 2.0
# executing finally clause
divide(2, 0)
# division by zero!
# executing finally clause
divide("2", "1")
# executing finally clause
# TypeError: unsupported operand type(s) for /: 'str' and 'str'

```

11.5 try - except - else 语句

如果在 `try` 子句执行时没有发生异常，Python 将执行 `else` 语句后的语句。

```
try:
    检测范围
except:
    出现异常后的处理代码
else:
    如果没有异常执行这块代码
```

使用 `except` 而不带任何异常类型，这不是一个很好的方式，我们不能通过该程序识别出具体的异常信息，因为它捕获所有的异常。

```
try:
    检测范围
except(Exception1[, Exception2[,...ExceptionN]]):
   发生以上多个异常中的一个，执行这块代码
else:
    如果没有异常执行这块代码
```

【例子】

```
try:
    fh = open("testfile.txt", "w")
    fh.write("这是一个测试文件，用于测试异常!!")
except IOError:
    print("Error: 没有找到文件或读取文件失败")
else:
```

```
    print("内容写入文件成功")
    fh.close()

# 内容写入文件成功
```

注意：`else` 语句的存在必须以 `except` 语句的存在为前提，在没有 `except` 语句的 `try` 语句中使用 `else` 语句，会引发语法错误。

11.6 raise 语句

Python 使用 `raise` 语句抛出一个指定的异常。

【例子】

```
try:
    raise NameError('HiThere')
except NameError:
    print('An exception flew by!')

# An exception flew by!
```

阿里云天池 Python 训练营 - Part2

12. 字符串处理

12.1 字符串切片

字符串是 Python 中常见的一种数据类型。字符串切片就是截取字符串。在 Python 中，可以利用字符串的切片特性进行提取、拆分、合并等操作，但不能对字符串进行修改。

12.1.1 单字符切片

单字符切片是对字符串中指定位置的单个字符进行截取。

语法结构为：**字符串[索引位置]**, 索引位置的序号是从 0 开始的。

关于索引位置，既能以开头为基准进行切片，也能以结尾为基准进行切片。

```
txt='Python与Excel' #要做切片处理的字符串。
print(txt[0],txt[1],txt[2]) #索引位置以开头为基准切片，返回值 'P y t'。
```

```
print(txt[-1],txt[-2],txt[-3]) #索引位置以结尾为基准切片，返回值 'l e c'。
```

12.1.2 多字符切片

如果截取的不是单个字符，而是多个字符，则切片的语法结构为：**字符串[开始索引:结束索引:步长]**。

多字符截取有几种常见的切片方式。

```
txt='Python与Excel' # 要做切片处理的字符串。
print(txt[2:9]) # 以开头为基准切片，返回值为 'thon 与 Ex'。
print(txt[-10:-3])   # 以结尾为基准切片，返回值为 'thon 与 Ex'。
print(txt[:7])  # 以开头为基准切片，返回值为 'Python 与'。
print(txt[:-5]) # 以结尾为基准切片，返回值为 'Python 与'。
print(txt[7:])  # 以开头为基准切片，返回值为 'Excel'。
print(txt[-5:]) # 以开头为基准切片，返回值为 'Excel'。
print(txt[2:-5]) # 开始索引以开头为基准，结束索引以结尾为基准，返回 'thon 与'。
print(txt[-10:7]) # 开始索引以结尾为基准，结束索引以开头为基准，返回 'thon 与'。
print(txt[:]) # 截取整个字符串切片，返回值为 'Python 与 Excel'。
print(txt[::2]) # 截取整个字符串切片，步长为 2，返回值为 'Pto 与 xe'。
print(txt[::-1]) # 截取整个字符串切片，步长为 -1，返回值为 'lecxE 与 nohtyP'。
print(txt[::-2]) # 截取整个字符串切片，步长为 -2，返回值为 'lcEnhy'。
```

12.1.3 字符串切片案例：根据身份证号判断性别

本案例对员工信息表中 B 列的身份证号进行性别判断。在 18 位身份证号中判断第 17 位数字，在 15 位身份证号中判断第 15 位数字。如果数字是奇数，则性别为男；如果数字是偶数，则性别为女。

本案例的编程思路是，使用字符串切片从身份证号的第 15 位开始截取到第 17 位，再截取最后一位数字。这样不管身份证号是 15 位，还是 18 位，最终都截取到了要判断性别的数字，之后的数据处理就比较容易了。

```
import xlrd,xlwt # 导入所需库。
wb = xlrd.open_workbook('./6-ID-1.xls') # 读取工作簿。
ws = wb.sheet_by_name('员工信息表') # 读取工作表。
nwb = xlwt.Workbook('utf-8') # 新建工作簿。
nws = nwb.add_sheet('员工信息表-1') # 新建工作表。
nws.write(0,0,'姓名') # 在表头写入 '姓名'。
nws.write(0,1,'身份证号') # 在表头写入 '身份证号'。
nws.write(0,2,'性别') # 在表头写入 '性别'。
row_num = 0 # 初始化 row_num 变量为 0。
while row_num < ws.nrows-1: # 当 row_num 小于 '员工信息表' 已用行数时，开始循环。
    row_num += 1 # 累加 row_num 变量。
    card = ws.cell_value(row_num,1) # 获取单元格的身份证号信息。
    sex_num = int(card[14:17][-1]) # 截取判断性别的数字。
    sex = '男' if sex_num % 2 == 1 else '女' # 根据数字判断性别。
    name = ws.cell_value(row_num,0) # 获取姓名。
    nws.write(row_num,0,name) # 将姓名写入新工作表中的 A 列单元格。
    nws.write(row_num,1,card) # 将身份证号写入新工作表中的 B 列单元格。
```

```
    nws.write(row_num,2,sex) # 将性别写入新工作表中的 C 列单元格。
nwb.save('./6-ID-2.xls') # 保存新建的工作簿。
```

- 第 1～9 行代码都是在为数据的读取和写入做准备。
- 第 13 行代码 sex_num=int(card[14:16][-1])是性别判断的关键语句。首先截取第 15～17 位数字，然后截取最右侧的数字。
  - 比如，对 18 位身份证号进行截取，'230102********7789[14:16]的截取结果为“778”，然后执行语句'778'[-1], 截取结果为“8”。
  - 再比如，对 15 位身份证号进行截取，'632123******051'[14:17]的截取结果为“1”，然后执行语句'1'[-1], 截取结果为“1”。这样对 18 位身份证号和 15 位身份证号都能截取到判断性别的数字，再使用 int 函数将其转换为整数。
- 第 14 行代码 sex='男' if sex_num%2==1 else '女', 对存储性别数字的 sex_num 变量除以 2 求奇偶，然后根据奇偶做性别判断。
- 第 16～18 行代码是将信息写入新工作簿中的新工作表。
- 第 19 行代码是保存新工作簿。

12.2 字符串统计 len/count

有时需要统计字符串的一些信息，比如统计字符串长度、统计指定子字符串在父字符串中出现的次数等。

12.2.1 统计字符串长度 len

统计字符串、列表、元组等对象的长度或项目个数，可以使用 **len** 函数。

函数语法：**len(s)**

参数说明：

s: 参数可以是序列，例如字符串、元组、列表、字典、集合等。

例如，统计字符串 'Python' 的长度。

```
print('Python',len('Python')) # 返回 'Python 6'。
```

12.2.2 按条件统计字符串 count

统计指定子字符串在父字符串中出现的次数，可以使用 **count** 函数。

函数语法：**count(sub[,start[,end]])**

参数说明：

- sub: 必选参数，搜索的子字符串。

- start: 可选参数，字符串开始搜索的位置，默认从第 0 个字符开始搜索。
- end: 可选参数，字符串结束搜索的位置，默认搜索到字符串最后。

例如，在 '张三19, 李四, 张三9, 林林6, 张三12, 李四8' 字符串中搜索关键词“张三”。

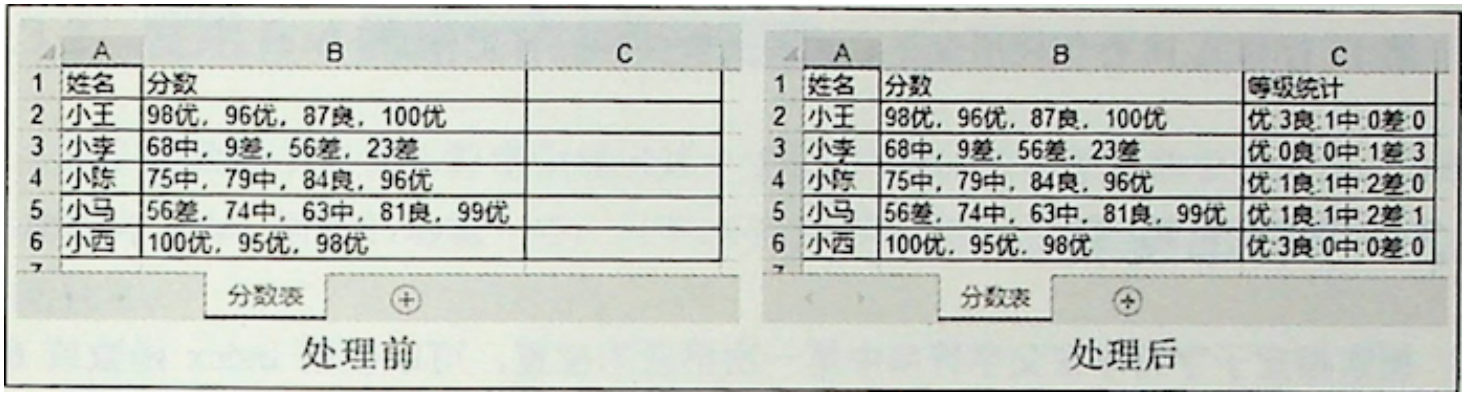
```
txt='张三19, 李四, 张三9, 林林6, 张三12, 李四8' # 被查找的字符串。
print(txt.count('张三')) # 返回值为 3。
print(txt.count('张三',4)) # 返回值为 2。
print(txt.count('张三',4,13)) # 返回值为。
```

- 第 2 行代码 print(txt.count('张三')), 是在整个字符串中搜索“张三”，该字符串中有 3 个“张三”，所以返回值为 3。
- 第 3 行代码 print(txt.count('张三',4)), 是从字符串的第 4 个位置开始搜索到最后，也就是在字符串，'李四, 张三 9, 林林 6, 张三 12, 李四 8' 中搜索“张三”，该字符串中有 2 个“张三”，所以返回值为 2。
- 第 4 行代码 print(txt.count('张三',4,13)), 是从字符串的第 4 个位置开始搜索到第 13 个位置，也就是在字符串，'李四, 张三9, 林' 中搜索“张三”，该字符串中只有 1 个“张三”，所以返回值为 1。

12.2.3 字符串统计应用案例：统计各等级出现的次数

本案例统计分数表中每个人获得**优、良、中、差**的次数，将结果写入 C 列单元格，如图所示。

本案例的编程思路是将 **4** 个等级作为搜索关键字在每个单元格中进行计数，然后将每个等级的计数结果写入 C 列。



处理前

	A	B	C
1	姓名	分数	
2	小王	98优，96优，87良，100优	
3	小李	68中，9差，56差，23差	
4	小陈	75中，79中，84良，96优	
5	小马	56差，74中，63中，81良，99优	
6	小西	100优，95优，98优	

分数表

处理后

	A	B	C
1	姓名	分数	等级统计
2	小王	98优，96优，87良，100优	优:3良:1中:0差:0
3	小李	68中，9差，56差，23差	优:0良:0中:1差:3
4	小陈	75中，79中，84良，96优	优:1良:1中:2差:0
5	小马	56差，74中，63中，81良，99优	优:1良:1中:2差:1
6	小西	100优，95优，98优	优:3良:0中:0差:0

分数表

```
import xlrd # 导入读取 xls 文件的库。
from xlutils.copy import copy # 导入复制工作簿的函数。
wb = xlrd.open_workbook('./7-ScoreAnalists-1.xls') # 读取工作簿。
ws = wb.sheet_by_name('分数表') # 读取工作表。
nwb = copy(wb) # 复制工作簿产生一个副本。
nws = nwb.get_sheet('分数表') # 读取副本工作簿中的工作表。
row_num = 0 # 初始化 row_num 变量为 0。
txt = '' # 初始化 txt 变量为空。
while row_num<ws.nrows-1: # 当 row_num 变量小于已使用单元格区域行数时。
    row_num += 1 # 则对 row_num 变量累加 1。
```

```
    score = ws.cell_value(row_num,1) # 获取 B 列单元格的值。
    for level in '优良中差': # 循环 '优良中差'4 个等级。
        lev_sco = '{}:{}\t'.format(level,score.count(level)) # 统计每个级别的
个数，并进行格式化。
        txt += lev_sco # 累积连接各等级数量。
    nws.write(row_num,2,txt) # 将统计结果写入 C 列单元格。
    txt = '' # 重新初始化 txt 变量，便于存储下个单元格各等级的统计结果。
nws.write(0,2,'等级统计') # 在 C 列写入表头。
nwb.save('./7-ScoreAnalists-1.xls') # 保存副本工作簿。
```

- 第 1～8 行代码为读取工作表数据和写入工作表做准备。
- 第 10 行和第 11 行代码用于获取单元格的值。
- 第 12～14 行代码循环判断每个等级在单元格中出现的次数，关键语句是 score.count(level), 并且使用 txt+=lev_sco 来累计各等级出现的次数。
- 第 15 行和第 16 行代码用来写入统计结果与重新初始化。nws.write(row_num,2,txt)是将 txt 变量中的统计结果写入副本工作簿中工作表的 C 列。txt='' 用于重新初始化 txt 变量。
- 第 17 行和第 18 行代码用来在 C 列写入表头和保存工作簿。

12.3 字符串搜索 index/find

搜索指定子字符串在父字符串中第一次出现的位置，可以使用 **index** 函数或 **find** 函数。

12.3.1 使用 index 函数搜索字符串位置

index 函数用于从字符串中找出子字符串第一个匹配项的索引位置，**如果查找的字符串不存在，则返回错误提示**。

函数语法：**index(sub[,start[,end]])**

参数说明：

- sub: 必选参数，搜索的子字符串。
- start: 可选参数，字符串开始搜索的位置，默认从第 0 个字符开始搜索。
- end: 可选参数，字符串结束搜索的位置，默认搜索到字符串最后。

例如，在字符串 '张三 19, 李四, 张三 9, 林林 6, 张三 12, 张三 8' 中搜索关键词“张三”。

```
txt='张三 19, 李四, 张三 9, 林林 6, 张三 12, 张三 8' # 被查找的字符串。
print(txt.index('张三')) # 返回值为 0。
print(txt.index('张三',18)) # 返回值为 21。
print(txt.index('张三',6,16)) # 返回值为 8。
```

- 第 2 行代码 print(txt.index(' 张三)), 是在整个字符串中搜索“张三”第 1 次出现的位置，返回值为 3。
- 第 3 行代码 print(txt.index('张三',18)), 是从字符串的第 18 个位置开始搜索“张三”第 1 次出现的

位置，返回值为 21。

- 第 4 行代码 print(txt.count('张三',6,16)), 是在字符串的第 6 个位置到第 16 个位置中搜索“张三”第 1 次出现的位置，返回值为 8。

12.3.2 使用 find 函数搜索字符串位置

find 函数用于从父字符串中找出某个子字符串第一个匹配项的索引位置，该函数的功能与 index 函数的功能一样，**只不过子字符串不在父字符串中时不会报异常，而是返回 -1**。

函数语法：**find(sub[,start[,end]])**

参数说明：

- sub: 必选参数，搜索的子字符串。
- start: 可选参数，字符串开始搜索的位置，默认从第 0 个字符开始搜索。
- end: 可选参数，字符串结束搜索的位置，默认搜索到字符串最后。

例如，在字符串 'Python' 中搜索 'Excel'，分别使用 find 函数和 index 函数，代码如下所示。

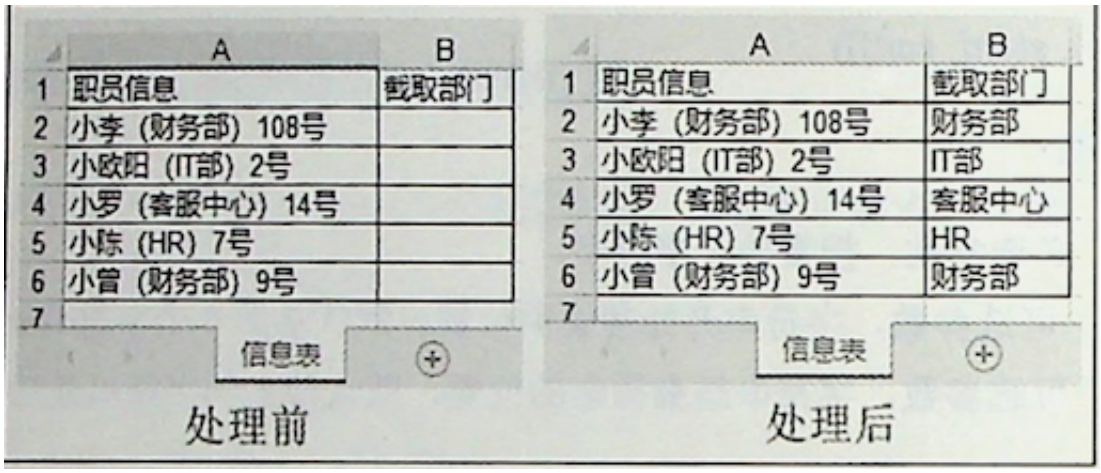
```
1 print('Python'.find('Excel')) # 查找不到返回 -1。
2 print('Python'.index('Excel')) # 查找不到返回错误提示。
```

- 第 1 行代码 print('Python'.find('Excel')), 当使用 find 函数搜索不到时，返回 -1。
- 第 2 行代码 print('Python'.index('Excel')), 当使用 index 函数搜索不到时，返回错误提示。提示错误为“**ValueError: substring not found**”，表示未找到子字符串

12.3.3 字符串搜索应用案例：提取指定位置的信息

本案例截取信息表中 A 列的部门信息，将截取结果写入 B 列对应的单元格中，如图所示。

本案例的编程思路是首先获取“(”和“）”的位置，然后使用字符串切片的方法截取两个位置之间的字符串。



```
import xlrd # 导入读取 xls 文件的库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb = xlrd.open_workbook('./8-Departments-1.xls') # 读取工作簿。
ws = wb.sheet_by_name('信息表') # 读取工作表。
nwb = copy(wb) # 复制工作簿产生一个副本。
nws = nwb.get_sheet('信息表') # 读取副本中的工作表。
row_num = 0 # 初始化 row_num 变量为 0。
while True: # 条件为 True，表示会一直循环，在循环中做终止循环处理。
    row_num += 1 # 对 row_num 变量累加 1。
    if row_num > ws.nrows-1: # 当 row_num 变量大于已使用单元格区域行数时。
        break # 则终止循环。
    info = ws.cell_value(row_num, 0) # 获取 A 列单元格的值。
    strat = info.find(' (')+1 # 搜索 ' (' 的位置，应该从 ' (' 之后，所以最后要加
1。
    end = info.find(') ') # 搜索 ') ' 的位置。
    dept = info[strat:end] # 截取 A 列单元格 ' (' 和 ') ' 之间的部门信息。
    nws.write(row_num,1,dept) # 将截取到的部门信息写入 B 列单元格。
nwb.save('./8-Departments-1.xls') #保存工作簿。
```

- 第 1～7 行代码为读取工作表数据和写入工作表做准备。
- 第 8 行代码开始循环读取单元格。
- 第 9～16 行代码是循环体中的处理语句。
- 第 10 行和第 11 行代码用来终止循环。
- 第 12～16 行代码首先获取单元格的值，再获取“（”和“）”的索引位置，然后截取部门信息，最后将部门信息写入副本工作簿中工作表 B 列对应的单元格。
- 第 17 行代码保存工作簿副本。

12.4 字符串替换 replace

字符串替换的本质就是有条件地对字符串进行修改。**前面不是说过字符串是只读属性，不能修改吗?**

实际上，替换后的字符串内存地址已经不是替换前的字符串内存地址，也就是说，并没有修改替换前的字符串，替换后生成了一个新的字符串。

12.4.1 字符串替换方法 replace

replace 函数用于把字符串中指定的旧字符串替换成指定的新字符串，默认全部替换。

函数语法：**replace(old,new[,count])**

参数说明：

- old: 必选参数，被替换的旧字符串。
- new: 必选参数，新字符串，用于替换旧字符串。
- count: 可选参数，替换的次数，默认替换所有出现的旧字符串。

例如，将字符串 'A组-优秀；B组-良好；C组-优秀；D组-优秀' 中的“优秀”替换为“晋级”。

```
txt='A组-优秀；B组-良好；C组-优秀；D组-优秀' # 被替换的字符串。
print(txt.replace('优秀','晋级')) # 将所有 '优秀' 替换为 '晋级'。
print(txt.replace('优秀','晋级',1)) # 将前 1 个 '优秀' 替换为 '晋级'。
print(txt.replace('优秀','晋级',2)) # 将前 2 个 '优秀' 替换为 '晋级'。
```

- 第 2 行代码 print(txt.replace('优秀','晋级')), 如果不指定第 3 个参数，则默认将所有的“优秀”替换为“晋级”，替换结果为 'A组-晋级；B组-良好；C组-晋级；D组-晋级'。
- 第 3 行代码 print(txt.replace('优秀','晋级',1)), 如果指定第 3 个参数为 1，则表示将第 1 个“优秀”替换为“晋级”，替换结果为 'A组 - 晋级；B组-良好；C组-优秀；D组-优秀。
- 第 4 行代码 print(txt.replace('优秀','晋级',2)), 如果指定第 3 个参数为 2，则表示将前两个“优秀”替换为“晋级”，替换结果为 'A组-晋级；B组-良好；C组-晋级；D组-优秀。

12.5 字符串拆分与合并 split/join

字符串的拆分与合并可以使用字符串切片的方法来完成，但字符太多就不方便了，表达也不够简洁、灵活。

本小节讲解使用 **split** 函数和 **join** 函数来完成字符串的拆分与合并。

12.5.1 拆分字符串为列表 split

split 函数用于拆分字符串，可以指定分隔符对字符串进行切片，并返回拆分后的字符串列表。

函数语法：**split([sep] [,maxsplit])**

参数说明：

- sep: 可选参数，表示分隔符，
  - 默认为空格(''), 但是不能为空('') 。
  - 分隔符可以是单个字符，也可以是多个字符。
  - 如果是多个字符，则被看作一个整体。
- maxsplit: 可选参数。
  - 表示要执行的最大拆分数。
  - -1(默认值)表示无限制。

例如，对字符串 '10 20 50' 和 '78|98|100' 进行拆分。

```
print('10 20 50'.split()) # 默认以 ' ' 进行拆分。
print('10 20 50'.split('|')) # 指定的拆分符号在字符串中不存在。
print('78|98|100'.split('|')) # 指定的拆分符号在字符串中存在。
print('78|98|100'.split('|',1)) # 指定拆分个数。
```

- 第 1 行代码 print('10 20 50'.split()), 对字符串'10 20 50'进行拆分，由于 split 函数中没有指定任何参数，所以默认以空格对整个字符串进行拆分，返回值为['10','20','50']。
- 第 2 行代码 print('10 20 50'.split('|')), 对字符串 '10 20 50' 按分隔符“|”进行拆分，由于字符串中并不存在“|”，因此返回的结果不是字符串，而是列表['102050']，列表中只有一个元素 '10 20 50'。
- 第 3 行代码 print('78|98|100'.split('|')), 对字符串 '78|98|100' 按分隔符“|”进行拆分，返回值为['78','98','100']。
- 第 4 行代码 print('78|98|100'.split('|',1)), 对字符串 '78|98|100' 按分隔符“|”进行拆，split 函数的第 2 个参数 **1** 表示只拆分到第 1 次出现的“|”，因此返回值为['78','98|100']。

12.5.2 合并列表为字符串 join

前面通过 **split** 函数将字符串拆分成列表，列表中存储的是拆分出来的子字符串。

现在要反向操作，将列表中的子字符串合并成一个大的字符串，可以使用 **join** 函数来完成。

函数语法：**join(iterable)**

参数说明：
iterable: 必选参数，可以是列表、元组等可迭代对象，但其中的值只能为字符串，不能是其他数据类型。

例如，以“-”为分隔符对列表['张三','18',' 财务部']进行合并。

```
print('-'.join(['张三','18','财务部'])) # 返回 '张三-18-财务部'。
```

13. Python 有序对象--列表处理技术

13.1 列表的创建与删除

列表用中括号 `[]` 表示，列表里的元素用逗号分隔。

下面介绍列表的创建和删除。

```
lst1 = [];print(lst1) # 创建空列表方法 1。
lst2 = list();print(lst2) # 创建空列表方法 2。
lst3 = [1,2,3];print(lst3) # 创建多个元素的列表。
lst3.clear();print(lst3) # 清空列表中的所有元素。
del lst3 # 删除 lst3 列表。
```

- 第 1 行代码 lst1=[]，使用一对空的中括号创建空列表。运行 print(lst1)后，在屏幕上输出结果[]。

- 第 2 行代码 lst2=list()，使用 list 类创建一个空列表，运行 print(lst2)后，在屏幕上输出结果[]。
- 第 3 行代码 lst3=[1,2,3], 创建有多个元素的列表，运行 print(lst3)后，在屏幕上输出结果[1,2,3]。
- 第 4 行代码 lst3.clear()，清空 lst3 列表中的所有元素，运行 print(lst3)后，在屏幕上输出结果[]。
- 第 5 行代码 del lst3，使用 del 语句删除指定的列表。删除后，lst3 列表就不存在了。

13.2 列表切片

列表、字符串和元组对象都支持以索引序号的方式进行切片。

13.2.1 单元素切片

列表的切片方法与字符串的切片方法一样，语法结构为**列表[索引位置]**, 索引位置的序号是从 0 开始的。

```
lst=['张三',19,[80,89,97]] # 要进行切片的列表。
print(lst[0],lst[1],lst[2]) # 以开头为基准切片，返回值：张三 19 [80, 89, 97]。
print(lst[-1],lst[-2],lst[-3]) # 以结尾为基准切片，返回值：[80, 89, 97] 19 张三。
```

13.2.2 多元素切片

在列表中可以截取一部分元素，语法结构为**列表[开始索引: 结束索引: 步长]**。注意，切片的结果中不包含结束索引位置的元素。

```
lst=[7,3,12,54,6,9,88,2,47,33,55] # 要做切片处理的列表。
print(lst[2:5]) # 以开头为基准切片，返回[12,54,6]。
print(lst[-9:-6]) # 以结尾为基准切片，返回[12,54,6]。
print(lst[:4]) # 以开头为基准切片，返回[7,3,12,54]。
print(lst[:-7]) # 以结尾为基准切片，返回[7,3,12,54]。
print(lst[6:]) # 以开头为基准切片，返回[88,2,47,33,55]。
print(lst[-5:]) # 以开头为基准切片，返回[88,2,47,33,55]。
print(lst[5:-2]) # 开始索引以开头为基准，结束索引以结尾为基准，返回[9,88,2,47]。
print(lst[-6:9]) # 开始索引以结尾为基准，结束索引以开头为基准，返回[9,88,2,47]。
print(lst[:]) # 截取整个列表切片，返回[7,3,12,54,6,9,88,2,47,33,55]。
print(lst[::2]) # 截取整个列表切片，步长为 2，返回[7,12,6,88,47,55]。
print(lst[::-1]) # 截取整个列表切片，步长为 -1，返回
[55,33,47,2,88,9,6,54,12,3,7]。
print(lst[::-2]) # 截取整个列表切片，步长为 -2，返回[55,47,88,6,12,7]。
```

切片要求	以开头为基准	以结尾为基准	返回值
从指定位置开始，截取到指定结束位置	print(lst[2:5])	print(lst[-9:-6])	[12,54,6]

从最左侧开始，截取到指定结束位置	print(lst[:4])	print(lst[:-7])	[7,3,12,54]
从指定位置开始，截取到最右侧	print(lst[6:])	print(lst[-5:])	[88,2,47,33,55]
开始索引以开头为基准，结束索引以结尾为基准	print(lst[5:-2])	print(lst[-6:9])	[9,88,2,47]
----------	----------	----------	----------
截取整个列表切片	print(lst[:])		[7,3,12,54,6,9,88,2,47,33,55]
截取整个列表切片，步长为 2	print(lst[::2])		[7,12,6,88,47,55]
截取整个列表切片，步长为 -1	print(lst[::-1])		[55,33,47,2,88,9,6,54,12,3,7]
截取整个列表切片，步长为 -2	print(lst[::-2])		[55,47,88,6,12,7]

13.3 列表元素的增加、删除和修改

列表在 Python 中的操作非常灵活，除可以对列表做切片外，还可以对列表进行增加、删除和修改元素等操作。

13.3.1 列表元素的修改

对列表中的元素进行修改，语法结构为列表 `[索引位置] = 修改的值` 。案例代码如下

```
lst = ['张三',18,[100,90]] # 被修改的列表
lst[0] = '小明' # 修改列表中的第 0 个元素。
lst[1] = '18岁' # 修改列表中的第 1 个元素。
lst[2] = 190 # 修改列表中的第 2 个元素。
print(lst) # 在屏幕打印修改后的 lst 列表。
```

13.3.3 列表元素的增加

在列表中增加元素可以使用加运算符 **(+)**、**append 函数**、**extend 函数**、**insert 函数**来完成，下表列出了不同的实现方法。

名称	语法结构	注释

+	list += list	使用加运算符的累积功能
append	append(object)	在列表末端增加一个元素
extend	extend(iterable)	在列表末端增加多个元素
insert	insert(index,object)	在列表指定位置增加一个元素

```
lst = ['张三'];print(lst) # 原始列表。
lst += ['6年级'];print(lst) # 使用积累方式增加。
lst.append('9班');print(lst) # 使用 append 方法增加单个元素。
lst.extend([85,96]);print(lst) # 使用 extend 方法增加多个元素。
lst.insert(3,'12岁');print(lst) # 使用 insert 在列表指定位置插入。
```

上面代码的运行结果：
['张三']
['张三','6年级']
['张三','6年级','9班']
['张三','6年级','9班',85,96]
['张三','6年级','9班','12岁',85,96]

13.3.3 列表元素的删除

在列表中删除元素，可以使用 **remove 函数**、**del 函数**、**pop 函数**来完成，它们的语法结构及注释如表所示。

删除列表元素的几种方法的语法结构及注释

名称	语法结构	注释
remove	remove(object)	从列表中删除指定的元素，不是指定元素的位置
pop	pop()	默认删除列表中的最后一个元素
pop	pop(index)	删除列表中指定位置的元素
del	del	删除指定列表范围的元素

下面看看使用不同方法的小案例，代码如下所示。

```
lst = ['张三', '6年级', '9班', '12岁', 85, 96];print(lst) #原始列表。
lst.remove('12岁');print(lst) #使用 remove 方法删除列表元素。
lst.pop();print(lst) #使用 pop 方法删除最列表最后一个元素。
lst.pop(2);print(lst) #使用 pop 方法删除指定位置元素。
del lst[1:];print(lst) #使用 del 语句删除指定列表区域元素。
```

上面的代码的运行效果：

['张三','6 年级','9 班','12 岁',85,96]
['张三','6 年级','9 班',85,96]
['张三','6 年级','9 班',85]
['张三','6 年级',85]
['张三']

13.4 列表操作符

列表与列表之间可以进行连接、比较等操作，也可以对列表进行重复操作，还可以判断指定元素是否在列表中等，这些操作可能只需要使用一个符号就可以完成。

13.4.1 列表操作符基础

列表操作符有 **+、*、in 和比较运算符**。下面列举几个小案例，代码如下所示。

```
print([1,2,3]+[4,5]) # 列表连接使用 + 运算符。
print([1,2,3]*3) # 重复列表使用 * 运算符。
print(2 in [1,2,3]) # 判断某个值是否在列表中存在。
print([1,2,3]==[1,2,3]) # 列表比较运算。
print([1,2,3]<[1,3,2]) # 列表比较运算
```

13.5 列表推导式

列表推导式在逻辑上相当于一个 for 循环语句，只是形式上更加简洁。列表推导式执行完成后会创建新的列表。无论列表推导式的写法如何变化，最后都会返回列表对象。**如果循环的目的是将数据写入指定单元格，那么最好用标准的循环语句，而不要用列表推导式。**

13.5.1 标准列表推导式

列表推导式的语法结构：**[表达式 for 变量 in 列表]**

例如，将 **['89','96','100','72']** 中的文本型数字转换为标准整数，可以使用列表推导式或 for 循环语句，代码如下所示。

```
# 原始列表。
lst = ['89','96','100','72']
# 使用列表推导。
lst1 = [int(n) for n in lst]
print(lst1)
# 使用循环方式。
lst2 = []
for n in lst:
    lst2.append(int(n))
print(lst2)
```

13.5.2 列表推导式变异

如果推导列表中的元素不是单值，而是列表或其他可循环序列，则在使用列表推导式或 for 循环时，可以将要循环的元素拆分。比如列表 **[[1,2,5],[10,5,6],[8,5,3]]**, 此列表中的元素也是列表，假如求每个子列表的值的乘积，可以使用两种列表推导式和两种 for 循环语句完成，代码如下所示。

```
1  # 原始列表
2  lst = [[1,2,5],[10,5,6],[8,5,3]]
3  # 列表推导式 1
4  lst1 = [l[0]*l[1]*l[2] for l in lst]
5  print(lst1)
6  # 列表推导式 2
7  lst2 = [x*y*z for x,y,z in lst]
8  print(lst2)
9  # 循环方式 1
10 lst3 = []
11 for l in lst:
12     lst3 += [l[0]*l[1]*l[2]]
13 print(lst3)
14 # 循环方式 2
15 lst4 = []
16 for x,y,z in lst:
17     lst4 += [x*y*z]
18 print(lst4)
```

- 第 1 种列表推导式 (第 4 行和第 5 行代码)：
  - 首先看 **lst1=[l[0]*l[1]*l[2] for l in lst]** 部分，
  - l 表示循环出来的每个子列表，**l[0]*l[1]*l[2]** 表示将子列表的第 0、1、2 个元素相乘。
  - 运行 print(lst1)，在屏幕上输出的结果为[10,300,120]。
- 第 2 种列表推导式(第 7 行和第 8 行代码)：
  - 首先看 **lst2=[x*y*z for x,y,z in lst]** 部分，x、y、z 分别表示子列表的第 0、1、2 个元素，
  - x*y*z 表示将它们相乘。运行 print(lst2)，
  - 在屏幕上输出的结果为[10,300,120]。
- 第 1 种 for 循环方式(第 10～13 行代码)与第 1 种列表推导式思路相同。
- 第 2 种 for 循环方式(第 15～18 行代码)与第 2 种列表推导式思路相同。

> 注意，每个子列表的元素个数必须相同，否则会出错。例如 [[1,2],[3,4]] 符合要求，而 [[1,2],[3]] 不符合要求。

13.5.3 嵌套列表推导式

嵌套列表推导式的语法结构：**[表达式 for 变量1 in 列表1 for 变量2 in 变量1 for 变量3 in 变量2...]**。

可以多层嵌套，注意放在 in 后面的对象必须是可迭代对象。

例如，在列表 **[[1,2],[3,4,5],[6,7]]** 中有 3 个元素，每个元素也是列表，下面将这些列表元素合并放在同一个列表中，并且每个数字还要乘以 10，结果为 **[10,20,30,40,50,60,70]**。

可以使用嵌套列表推导式或嵌套 for 循环语句实现，代码如下所示。

```
1  # 原始列表。
2  lst = [[1,2],[3,4,5],[6,7]]
3  # 使用嵌套列表推导。
4  lst1 = [v*10 for l in lst for v in l]
5  print(lst1)
6  # 使用嵌套循环方式。
7  lst2 = []
8  for l in lst:
9      for v in l:
10         lst2.append(v*10)
11 print(lst2)
```

- 在第 4 行和第 5 行代码中，
  - 代码 **lst1 =[v*10 for l in lst for v in l]** 在执行时首先运行 **for l in lst**，
  - l 分别循环出 **[1,2]**、**[3,4,5]**、**[6,7]** 3 个元素，
  - 再运行 for v in l，v 分别循环出 **[1,2]** 中的 1、2，**[3,4,5]** 中的 3、4、5，**[6,7]** 中的 6、7，并将这些循环出来的数字乘以 10。
- 第 5 行代码运行 **print(lst1)**，在屏幕上输出的结果为 **[10,20,30,40,50,60,70]**。
- 如果使用标准的 **for** 循环语句，则可以使用第 7～11 行代码来完成，最后 **lst2** 变量存储的就是转换后的数字。运行第 11 行代码 **print(lst2)**，在屏幕上输出的结果为 **[10,20,30,40,50,60,70]**。

13.5.4 条件列表推导式

条件列表推导式语法结构：**[表达式 for 变量 in 列表 if 条件判断]**

例如，对列表 **[85,68,98,74,95,82,93,88,74]** 进行筛选，筛选出大于或等于 **90** 的值，生成新的列表。

下面分别使用条件列表推导式和 for 循环语句来实现，代码如下所示。

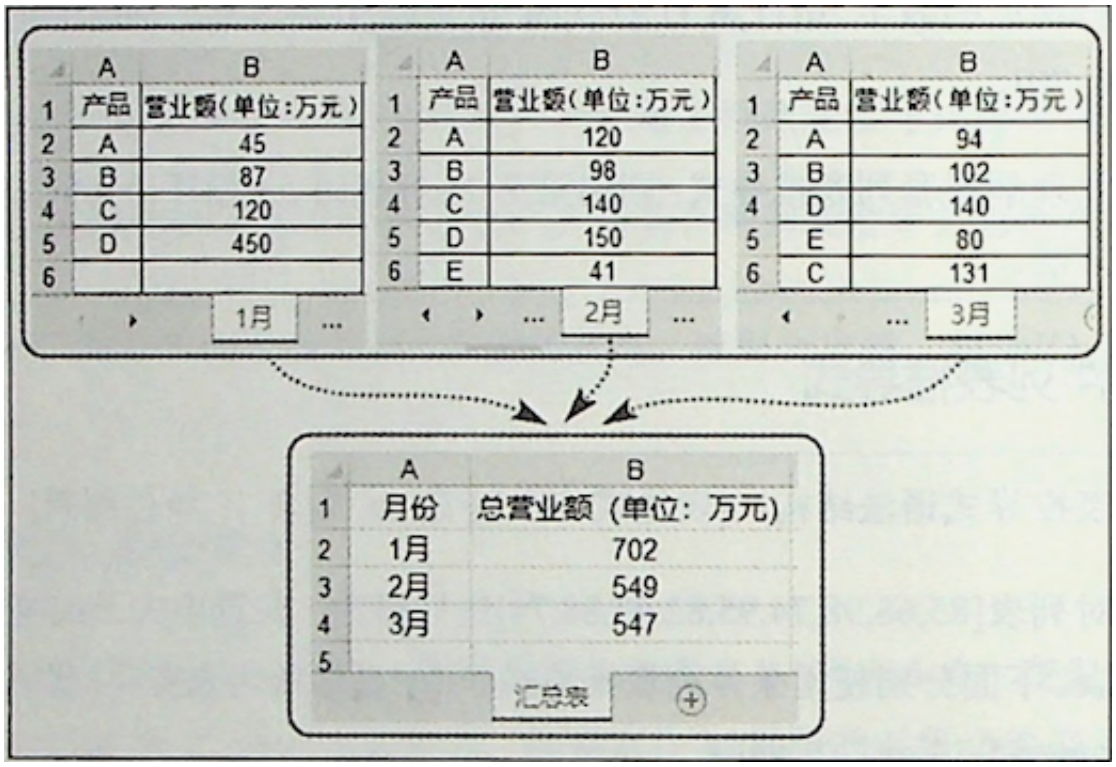
```
1  # 原始列表。
2  lst = [85,68,98,74,95,82,93,88,74]
3  # 使用条件列表推导。
4  lst1 = [n for n in lst if n>=90]
5  print(lst1)
6  # 使用条件循环方式。
7  lst2 = []
8  for n in lst:
9      if n>= 90:
10         lst2.append(n)
```

```
print(lst2)
```

- 第 4 行代码 **lst1=[n for n in lst if n>=90]**, **for n in lst** 后面的 **if n>=90** 表示当条件成立时，n 变量保留。
- 运行第 5 行代码 **print(lstl)**，屏幕上的输出结果为 **[98,95,93]**。
- 第 7～10 行代码，是在 for 循环体中使用 if 语句完成判断。
- 运行第 11 行代码 **print(lst2)**，屏幕上的输出结果为 **[98,95,93]**。

13.5.5 列表推导式应用案例 1: 自动汇总多工作表数据

本案例对工作簿中“1 月”，“2 月”，“3 月”工作表的 B 列数据进行求和，然后写入新工作簿的新工作表，如图所示。




```
import xlwt,xlrd # 导入读取与写入 xls 文件的库。
wb = xlrd.open_workbook('./14-Turnover123-1.xls') # 读取工作簿。
nwb = xlwt.Workbook('uft-8');nws=nwb.add_sheet('汇总表') # 新建工作簿与工作表。
lst = [[ws.name,sum(ws.col_values(1)[1:])] for ws in wb.sheets()] # 求和工作表中 B 列的金额。
row_num = 0 # 初始化 row_num 变量为 0。
for rows in [['月份','总营业额']] + lst: # 将表头连接到 lst 列表前面，并开始循环。
    nws.write(row_num,0,rows[0]) # 将月份写入 A 列。
    nws.write(row_num,1,rows[1]) # 将每个月的总营业额写入 B 列。
    row_num += 1 # 累加 row_num 变量，并做为写入数据时的行号。
nwb.save('./14-Turnover123-2.xls') # 保存工作簿。
```

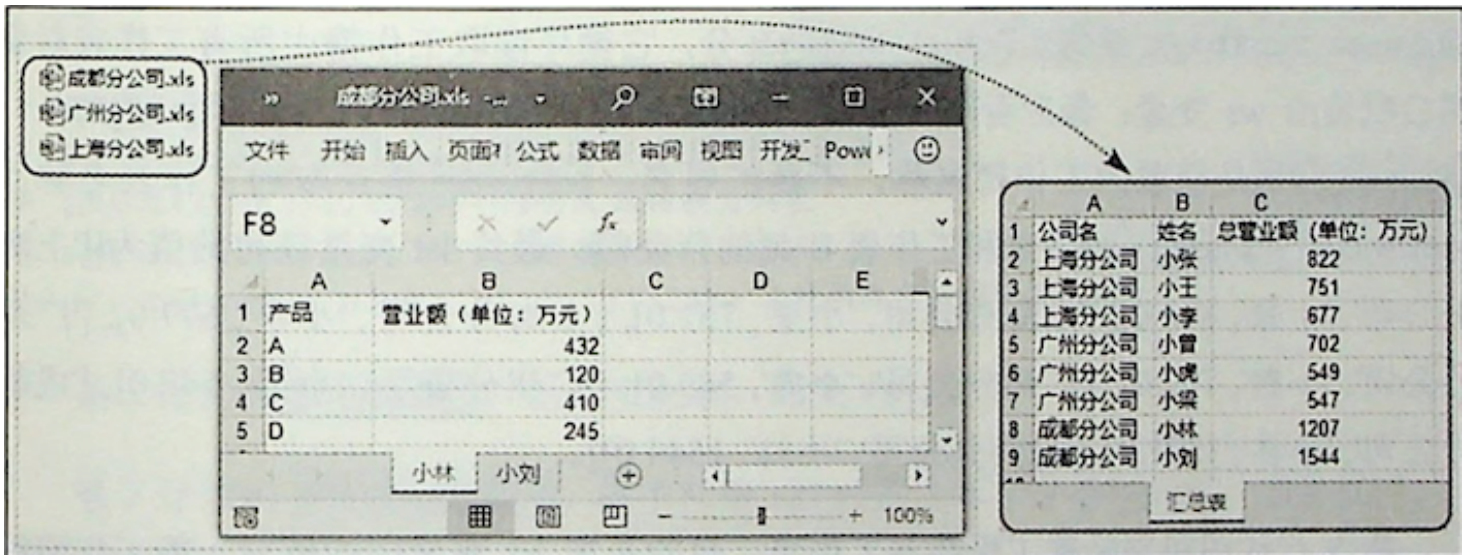
- 第 1～3 行代码为读取和写入数据做准备。
- 第 4 行代码 **lst = [[ws.name,sum(ws.col_values(1)[1:])] for ws in wb.sheets()]**
  - 其中 **for ws in wb.sheets()]** 用来循环读取工作簿中的每个工作表，再赋值给 **ws** 变量。

- ws.name 用来获取工作表名称，
- sum(ws.col_values(1)[1:]) 用来获取工作表 B 列的数据并进行求和。

- 整个列表推导的处理要求是将工作表名称与总金额组成列表，整行代码运行的结果是将 [['1月',702.0],['2月',549.0],['3月',547.0]] 赋值给 lst 变量。
- 第 5～9 行代码是将获取的 lst 列表中的值写入新工作表。
- 第 10 行代码用来保存新建的工作簿。

13.5.6 列表推导式应用案例 2: 汇总多工作簿数据

汇总指定文件夹中所有工作簿下所有工作表 B 列的数据，然后写入新工作簿的工作表，如图所示。



本案例的编程思路是循环读取每个工作簿中每个工作表 B 列的数据，然后进行求和。

```
import os,xlwt,xlrd #导入操作系统接口模块，xls 读取与写入库。
files = os.listdir('销售表') #获取销售表文件夹下的所有工作簿名称。
lst = [[file.split('.')[0],ws.name,sum(ws.col_values(1)[1:])] for file in
files for ws in xlrd.open_workbook('销售表 /'+file).sheets()] #对每个工作簿下每
个工作表 B 列的数字求和。
print(lst)
lst = [['公司名','姓名','总营业额']]+lst #将表头连接到 lst 列表前面。
nwb = xlwt.Workbook('utf-8');nws = nwb.add_sheet('汇总表') #新建工作簿与工作表。
row_num = 0 #初始化 row_num 为 0。
for l in lst: #循环 lst 列表中的每个元素。
    nws.write(row_num,0,l[0]) #将公司名写入 A 列。
    nws.write(row_num,1,l[1]) #将工作表名写入 B 列。
    nws.write(row_num,2,l[2]) #将每个人的业绩写入 C 列。
    row_num += 1 #累加 row_num 变量，并做为写入数据时的行号。
nwb.save('./15-SalesReport.xls') #保存工作簿。
```

- 第 1 行代码 import os,xlwt,xlrd，其中 os 表示导入操作系统接口模块，此模块是内置的，无须安装，主要是为了使用其中的 listdir 函数。

- 第 2 行代码 files = os.listdir('销售表')，表示获取“销售表”文件夹中的所有文件名。当前“销售

表”文件夹中只有工作簿，所以获取了所有工作簿名称。将工作簿名称赋值给 files 变量，files 变量中的值为 **['上海分公司.xls','广州分公司.xls','成都分公司.xls']** 。

- 第 3 行代码 **lst=[[file.split('.')[0],ws.name,sum(ws.col_values(1)[1:])] for file in files for ws in xlrd.open_workbook('销售表 /'+file).sheets()]**，这行代码是嵌套列表推导式结构。
  - 先看 **for file in files** 部分，它循环获得工作簿名称并赋值给 file 变量；
  - 再看 **for ws in xlrd.open_workbook('销售表/'+file).sheets()** 部分，它循环读取工作簿中所有工作表对象，然后赋值给 **ws** 变量；
  - 最后看 **[file.split('.')[0],ws.name,sum(ws.col_values(1)[1:])]** 部分，**file.split('.')[0]** 是获取的工作簿名称，不要扩展名，**ws.name** 是获取的工作表名称，**sum(ws.col_values(1)[1:])** 是求和工作表 B 列的营业额。
  - 最终 lst 变量获得的值为 **['上海分公司','小张',822.0],['上海分公司','小王',751.0],['上海分公司','小李',677.0],['广州分公司','小曾,702.0],[' 广州分公司 ',' 小虎 ',549.0],[广州分公司','小梁',547.0],['成都分公司','小林,1207.0],[' 成都分公司 ',' 小刘 ',1544.0]]**。
- 第 5 行代码用来新建工作簿与工作表，目的是把 lst 变量中的值写入新工作簿的新工作表。
- 从第 6 行代码开始都是将 lst 变量中的数据写入工作表的操作，这里不再赘述。

13.6 列表的转换

前面学习了列表的切片，以及列表元素的添加、删除、修改等操作，如果想将其他对象转换为列表，或者想对列表的位置、顺序等进行调整，又该如何操作呢?

13.6.1 类对象转换 list

在处理数据时有时需要将其他对象转换为列表，比如将元组、集合、字典转换为列表，任何可迭代对象均可直接或间接地转换为列表。

要完成这些转换可以使用 **list** 类，对类进行实例化可以创建对象，因此可以通过 list 类来创建列表对象。

类语法：**list([iterable])**

参数说明：
iterable: 可选参数，可迭代对象。

下面是其他常见对象转换为列表的例子，代码如下所示。

```
print(list()) # 创建空列表。
print(list('123')) # 将字符串转换为单个字符的列表。
print(list((1,2,3))) # 将元组转换为列表。
print(list({1,2,3})) # 将集合转换为列表。
print(list({'a':1,'b':2,'c':3})) # 将字典中的键转换为列表。
```

- 第 1 行代码 print(list0)，创建空列表，结果为 [] 。
- 第 2 行代码 print(list(123))，将字符串 '123' 转换为单个字符的列表，结果为 [1,2,3].
- 第 3 行代码 print(list((1,2,3)))，将元组 (1,2,3) 转换为列表 [1,2,3] 。
- 第 4 行代码 print(list((1,2,3)))，将集合(1,2,3} 转换为列表[1,2,3]。
- 第 5 行代码 print(list({'a':1,'b':2,'c':3}))，将字典 {'a':1,'b':2,'c':13} 中的键转换为列表 ['a','b','c']。

其中，元组、集合、字典这几种对象暂时还没有介绍到，它们也是Python中的重要对象，在这里暂时知道可以将它们转换为列表对象就可以了。

13.6.2 反转列表reverse

要将列表中的元素反转，可以使用 reverse 函数。

函数语法：**reverse()**

参数说明：

该函数没有参数，可以对列表中的元素进行反向排序。

例如，要将[1,2,3]转换为[3,2,1]，可以使用 `reverse` 函数，代码如下所示。

```
lst = [1,2,3,4] # 提供的列表。
lst.reverse() # 反转 lst 列表。
print(lst) # 在屏幕上打印 lst 列表。
```

第 2 行代码 lst.reverse() 直接对lst列表使用 reverse 函数。注意，此函数没有任何参数，直接 reverse() 即可。

13.6.3 列表复制 copy

列表的复制分为**浅复制**和**深复制**。

- 浅复制只引用对象的内存地址
- 深复制是重新开辟一个新的内存空间，得到完全独立的新对象。

浅复制使用的是copy函数。

函数语法：**copy()**

参数说明：

该函数没有参数。

copy 函数的案例代码如下所示。

```
# 列表中元素是单值。
lst1 = [1,2,3,4] # 被复制的列表。
lst2 = lst1.copy() # 浅复制lst1列表，并赋值给lst2变量。
lst1[3] = 100 # 修改lst1中的元素。也可以修改lst2中的元素。
print(lst1,lst2) # 对浅复制前、后两个列表的数据。
# 列表中元素是容器型元素。
lst3 = [1,[2,3],4] # 被复制的列表，注意列表的第1个元素[2,3]也是列表。
lst4 = lst3.copy() # 浅复制lst3列表，并赋值给lst4变量。
lst3[1][0] = 100 # 修改lst3中的元素。也可以修改lst4中的元素。
print(lst3,lst4) # 对浅复制前、后两个列表的数据。
```

先看看当列表中的元素是单值时，在复制列表时的变化。

- 第 2 行代码 **lst1 = [1,2,3,4]** 是准备要复制的列表。
- 第 3 行代码 **lst2 = lst1.copy()**，表示对 **lst1** 列表进行复制，然后赋值给变量 **lst2**。
- 第 4 行代码 **lst1[3] = 100**，对 lst1 列表中的第 3 个元素进行修改，将原来的 **4** 修改为 **100**。
- 第 5 行代码 **print(lst1,lst2)** ，在屏幕上输出 lst1 和 lst2 两个列表，返回值为 **[1,2,3,100] [1,2,3,4]**。
  - 对比一下结果，**lst1** 的第 **3** 个元素与 **lst2** 的第 **3** 个元素不一致，原因是原来 **lst1[3]** 中对象4的内存地址换成了对象100的内存地址，
  - 而 lst2[3] 还是引用原来对象4的内存地址，引用的内存地址没有跟着变化成对象100的内存地址。

再看看当列表中的元素是容器型元素时，在复制列表时的变化。

- 第 7 行代码 **lst3 = [1,[2,3],4]** 是准备要复制的列表。
- 第 8 行代码 **lst4 = lst3.copy()**，对 **lst3** 列表进行复制，然后赋值给变量 **lst4**。
- 第 9 行代码 **lst3[1][0] = 100**，对 **lst3** 列表中第 **1** 个元素的第 **0** 个元素进行修改，将原来的 **2** 修改为 **100**。
- 第 10 行代码 **print(lst3,lst4)**，在屏幕上输出 **lst3** 和 **lst4** 两个列表，返回值为 **[1,[100,3],4] [1,[100,3],4]**。对比一下结果，发现 **lst3** 和 **lst4** 的结果完全一样，原因是复制列表时引用的是列表元素的内存地址，比如 **lst3** 中的第 1 个元素 [2,3]，此元素也是一个列表，程序会在内存中给该列表分配一个内存地址。虽然使用 **lst3[1][0] = 100** 将其中的 **2** 修改为 **100**，但外层列表分配的内存地址没有变，所以最后返回的结果还是一样的。

如果希望复制出的新列表与原来的列表没有任何关联，则可以使用深复制。深复制要先导入**copy**标准模块，然后使用中**copy**模块中的**deepcopy**函数。

函数语法： **deepcopy(x)**

参数说明：

x:必选参数，被深复制的对象。

deepcopy 函数的案例代码如下所示。

```
# 列表中元素是单值。
import copy # 导入复制模块。
lst1 = [1,2,3,4] # 准备要被复制的列表。
lst2 = copy.deepcopy(lst1) # 使用复制模块下的深复制方法来复制lst1，并赋值给lst2。
lst1[3] = 100 # 修改lst1中的元素。也可以修改lst2中的元素。
print(lst1,lst2) # 对深复制前、后两个列表的数据。
# 列表中元素是容器型元素
lst3 = [1,[2,3],4] # 准备要被复制的列表。
lst4 = copy.deepcopy(lst3) # 使用复制模块下的深复制方法来复制lst3，并赋值给lst4。
lst3[1][0]=100 # 修改lst3中的元素。也可以修改lst4中的元素。
print(lst3,lst4) # 对深复制前、后两个列表的数据。
```

以上代码与上段中的代码基本相同，只是在第 4 行和第 9 行代码中，将原来的浅复制函数修改成了深复制函数。复制后的新列表与原来的列表没有任何关联，因为深复制为新列表开辟了新的内存地址，而不是引用原来列表元素的内存地址。因此，无论是修改原列表中的元素，还是修改新列表中的元素，彼此都不会受影响。

13.6.4 列表组合 zip

zip 函数是 Python 中的一个内建函数，它接收一系列可迭代的对象作为参数，将对象中对应的元素打包成一个个tuple(元组)，然后返回由这些元组组成的一个可迭代对象。

函数语法：**zip(*iterables)**

参数说明：

iterables:至少1个可迭代对象。

下面是 zip 函数的使用案例，代码如下所示。

```
l = [['a','b','c'],[1,2,3]] # 提供要重新组合的列表。
lst1 = list(zip(l[0],l[1])) # 将要转换的值分别放到zip的不同参数位置。
print(lst1) # 在屏幕打印重新组合后lst1的结果。
lst2 = list(zip(*l)) # 直接将整个l列表放到zip中。
print(lst2) # 在屏幕打印重新组合后lst2的结果。
lst3 = list(zip(*lst2)) # 再组合回原来的结构。
print(lst3) # 在屏幕打印重新组合后lst3的结果。
print(list(zip(iter([1,2,3]))))
```

- 第 2 行代码 **lst1=list(zip(l[0],l[1]))**,将l[0]和l[1]分别放入 zip 函数的第1个和第2个参数中,如果后面还有数据,可以继续放入第 3 个、第 4 个参数中。zip(l[0],l[1]) 转换出的是一个可迭代对象＜zip object at 0x00000247618012C8＞,这种可迭代对象的优势是不占用内存空间,在需要时才获取其中的数据,比如使用循环语句获取其中的元素,或者使用 list 类将其转换为列表,具体处理方式依据具体情况而定。
- 第 3 行代码 print(lst1),在屏幕上输出的结果为 **[('a',1),('b',2),('c',3)]**。

- 第 4 行代码 **lst2=list(zip(*l))** ,直接将 **l** 变量中的列表放到 zip 参数中,这种表达方式需要在列表前面加 *。最后 lst2 返回的结果与 lst1 返回的结果是一样的。
- 第 5 行代码 **print(lst2)**,在屏幕上输出的结果为 **[('a',1),('b',2),('c',3)]**。
- 第 6 行代码 **lst3=list(zip(*lst2))**,用同样的表达方式将 **lst1** 列表或 **lst3** 列表转换回去。返回的结果与l变量中的列表结构相同。
- 第 7 行代码 **print(lst3)**,在屏幕上输出的结果为 **[('a','b','c),(1,2,3)]** ,与1变量中的[['a','b','c],[1,2,3]]结构相同,只不过列表中的元素由原来的列表类型变成了元组类型。

13.7 列表常用统计方式

用户经常会对列表进行各种汇总统计，本节就来讲解列表的求和、求平均值、计数等统计函数，以及按条件对列表元素进行计数、位置查找等。

13.7.1 常用统计函数 len/sum/max/min

可以对列表进行一些常见的统计操作，比如计数、求和、求最大值、求最小值、求平均值。这些统计函数都比较简单，下面看看它们的应用，代码如下所示。

```
lst = [100,99,81,86] # 被处理的列表。
print(len(lst)) # 计数处理。
print(sum(lst)) # 求和处理。
print(max(lst)) # 求最大值处理。
print(min(lst)) # 求最小值处理。
print(sum(lst)/len(lst)) # 求平均值处理。
print(len(list(zip(*[[1],[2]]))))
print(len(list(iter(range(5)))))
```

- 第 2 行代码 print(len(lst)) 是对 ls t列表计数。在统计字符串长度中讲解过 len函数，只不过这里对象为列表。
- 第 3 行代码 print(sum(lst)) 是对 lst 列表求和，sum函数比较简单，在前面章节的一些案例中已经使用过了。
- 第 4 行代码 print(max(lst) 是对 lst 列表求最大值。
- 第 5 行代码 print(min(lst)) 是对 lst 列表求最小值。
- 第 6 行代码 print(sum(lst)/len(lst) 是对 lst 列表求平均值。Python 没有内置的求平均值函数，只能用求和结果除以元素个数来获取平均值。

实际上，**len、sum、max、min 函数**可以对所有可迭代的对象进行统计，比如后面将学习的元组、集合等都可以使用这些函数。

13.7.2 常用统计函数 count/index

在列表中，要统计指定元素在列表中出现的次数，可以使用count函数；要统计指定元素在列表中出现的位置，可以使用index函数。这两个函数的案例代码如下所示。

```
lst = ['a','b','c','b','b'] # 被处理的列表。
print(lst.count('b')) # 统计'b'在列表中出现的次数。
print(lst.index('b')) # 统计'b'在列表中第1次出现的位置。
```

- 第 2 行代码 print(lst.count('b'))，统计 'b' 在lst列表中出现的次数，返回值为 3。
- 第 3 行代码 print(lst.index('b'))，统计 'b' 在列表中第 1 次出现的位置，返回值为 1。index函数的第2个和第3个参数可以用来指定起止位置。

14. Python 有序对象--元组处理技术

- 什么是元组?

  Python 中的元组与列表类似，同属序列类型，都可以按照特定顺序存放一组数据，数据类型不受限制，切片方式也相同。

- 元组与列表的区别在于：

  元组存储的数据不能被修改，比如不能对元组的元素进行添加、删除。可以将元组看作是只读属性的列表。

- 为什么要使用元组?

  虽然元组在操作上没有列表灵活，但元组占用的内存空间更小，存取速度更快。所以，某些内置函数的返回值是元组类型。比如，前面学习的zip函数，迭代出来的每个元素就是元组。

14.1 元组的创建与删除

元组用小括号 `()` 表示，元组里的元素用逗号分隔。

下面介绍元组的创建和删除方法，代码如下所示。

```
tup1 = ();print(tup1) # 创建空元组方法1。
tup2 = tuple();print(tup2) # 创建空元组方法2。
tup3 = (1,2,3);print(tup3) # 创建多个元素的元组。
tup4 = (100,);print(tup4) # 创建单个元素的元组。
del tup3 # 删除元组。
```

- 第 1 行代码 tupl = (),使用一对空小括号。运行 print(tup1) 后,屏幕上的输出结果为()。
- 第 2 行代码 tup2 = tuple(),使用tuple类创建元组。运行 print(tup2) 后,屏幕上的输出结果为()。
- 第 3 行代码 tup3 = (1,2,3),在小括号中输入元组的元素。运行 print(tup3) 后,屏幕上的输出结果

为(1,2,3)。

- 第 4 行代码 tup4 = (100,),在小括号中输入元组的元素。**注意,如果元组中的元素只有一个,则需要在这个元素的后面添加逗号,否则程序不能正确识别**。运行 print(tup4) 后,屏幕上的输出结果为(100,)。
- 第 5 行代码 del tup3,使用 del 语句删除指定的元组。删除后,tup3 元组就不存在了。

14.2 元组的基本操作

元组虽然没有列表灵活,但一些基本的操作还是可以实现的,比如切片、合并、循环、推导、转换等。

14.2.1 元组的合并

元组的合并非常简单。比如 **tup =(1,2,3)+(4,5,6)** ,输出结果为 **(1,2,3,4,5,6)**。这种合并方式是比较好理解的。而对于累积式合并,很多读者会有疑惑,下面举一个例子, 代码如下所示。

```
1 tup = (1,2,3) # 元组。
2 print(id(tup),tup) # 合并前在屏幕打印元组内存地址和组。
3 tup += (4,5,6) # 将tup元组与(4,5,6)合并。
4 print(id(tup),tup) # 合并前在屏幕打印元组内存地址和组。
```

- 第 1 行代码 tup=(1,2,3)，这是最开始的元组。
- 第 2 行代码 print(id(tup),tup)，输出 tup 元组的内存地址和tup元组的值，返回结 果为“2382958184472(1,2,3)”。
- 第 3 行代码 tup+=(4,5,6)，将 tup 元组与(4,5,6) 合并，再赋值给tup。
- 第 4 行代码 print(id(tup),tup)，再次输出 tup 元组的内存地址和 tup 元组的值，返回结果为“2382958104296(1,2,3,4,5,6)”。

对比合并前后的输出结果，合并前 tup 的内存地址是 2382958184472，合并后 tup 的内存地址是 2382958104296。这两个内存地址不同，说明 tup 变量标识符已经从 2382958184472 转换绑定到2382958104296。也就是说，用户看到的 tup 变量没有变化，但它绑定的内存地址已经变了。如果不明白其中的道理，就会觉得元组也是可以修改的。

> 注意，代码中的 id 用于获取对象的内存地址，而且内存地址会动态变化。如果用户在计算机上测试上面的代码，则 id 可能与书上的 id 不同。这不重要，只要合并前后两个 id 的值不同就可以了。

14.2.2 元组的复制

元组也可以进行深复制和浅复制，只不过浅复制只能使用copy模块中的浅复制。下面看看元组复制的一些特性，代码如下所示。

```
1 import copy # 导入复制模块。
```

```
2  tup1 = (1,2,3) # 准备要复制的元组。
3  tup2 = copy.copy(tup1) # 将浅复制tup1元组，并赋值给tup2变量。
4  tup3 = copy.deepcopy(tup1)# 将深复制tup1元组，并赋值给tup3变量。
5  print(id(tup1),id(tup2),id(tup3)) # 在屏幕打印tup1、tup2、tup3元组的内存地址。
6
7  tup4 = (1,[2],3) # 准备要复制的元组。
8  tup5 = copy.copy(tup4) # 将浅复制tup4元组，并赋值给tup5变量。
9  tup6 = copy.deepcopy(tup4) # 将深复制tup4元组，并赋值给tup6变量。
10 print(id(tup4),id(tup5),id(tup6)) # 在屏幕打印tup4、tup5、tup6元组的内存地址。
```

先看元组的元素为不可变类型的对象时的情况。

- 第 2 行代码 tupl = (1,2,3)，元组中的每个元素都是不可变类型的对象，此元组是准备被复制的对象。
- 第 3 行代码 tup2 = copy.copy(tup1)，此代码使用了 copy 模块中的浅复制 copy 函数，并且将复制结果赋值给 tup2 变量。
- 第 4 行代码 tup3 = copy.deepcopy(tup1)，此代码使用了 copy 模块中的深复制 deepcopy 函数，并且将复制结果赋值给 tup3 变量。
- 第 5 行代码 print(id(tupl),id(tup2),id(tup3))，在屏幕上输出 tupl、tup2、tup3的内存地址，结果为“2328759322728 2328759322728 2328759322728”。观察返回结果，可以发现3个元组的内存址相同。

再看元组的元素为可变类型的对象时的情况。

- 第 7 行代码 tup4 = (1,[2],3)，其中的 [2] 是可变类型的对象，此元组是准备被复制的对象。
- 第 8 行代码 tup5 = copy.copy(tup4)，此代码使用了 copy 模块中的浅复制 copy 函数，并且将复制结果赋值给 tup5 变量。
- 第 9 行代码 tup5 = copy.copy(tup4)，此代码使用了 copy 模块中的深复制 deepcopy 函数，并且将复制结果赋值给 tup6 变量。
- 第 10 行代码 print(id(tup4),id(tup5),id(tup6))，在屏幕上输出tup4、tup5、tup6的内存地址，结果为“2328759323128 2328759323128 2328760171720。”观察返回结果，可以发现 tup4 和 tup5 的内存地址相同，而 tup6 的内存地址与其他两个内存地址不同。

因此，当元组中有不可变类型的对象时，执行深复制和浅复制都不会再开辟内存空间，用的是同一个内存地址；当元组中有可变类型的对象时，执行深复制会重新开辟一块内存空间。

14.2.3 元组的循环

元组也可以像列表一样做元组推导式和 for 循环，比如将元组(1，2，3)的每个元素乘 10，再返回一个新元组，可以分别使用元组推导式和 for 循环语句完成，代码如下所示。

```
tup = (1,2,3) # 被循环的元组。
tup1 = (t*10 for t in tup) # 元组推导式。
print(tup1) # 在屏幕打印tup1元组。
print(tuple(tup1)) # 将tup1迭代器转换为元组。

tup2 = () # 创建空元组。
for t in tup: # 循环tup元组的元素。
    tup2 += (t*10,) # 将tup中的元素乘以10，再积累合并到tup2变量。
print(tup2) # 在屏幕打印tup2元组。
```

第 1 行代码 tup = (1,2,3) 是被循环的元组，也可以是其他可迭代对象。

先看看使用元组推导式进行循环处理的情况。

- 第 2 行代码 tup1=(t＊10 for t in tup)，用元组推导式循环出每个元素，再乘 10，然后将推导结果赋值给 tupl 变量。
- 第 3 行代码 print(tup1)，在屏幕上输出 tup1 元组，结果为“＜generator object＜genexpr＞at 0x000001F7BE98D5C8＞”。也就是说，元组推导式的结果不是元组，而是生成器，生成器也是可迭代对象。
- 第4行代码 print(tuple(tupl))，使用 tuple 类对象将 tupl 转换为元组，返回结果为(10,20,30)。

再看看使用 for 循环语句进行处理的情况。

- 第 6 行代码 tup2=()，新建一个空元组。
- 第 7 行代码 for t in tup:，循环 tup 元组中的每个元素。
- 第 8 行代码 tup2 += (t*10，)，将循环出来的元素乘 10，再积累并合并到 tup2 变量。
- 第 9 行代码print(tup2)，在屏幕上输出tup2，返回结果为(10,20,30)。

14.2.4 类对象转换

在 Python 中，可以使用 tuple 类对象创建或转换一个元组对象，比如创建空元组以及将字符串、列表、集合、字典这些可迭代对象转换为元组。

类语法：**tuple([iterable])**

参数说明：

iterable:可选参数，要转换为元组的可迭代序列。

下面看几个转换的例子，代码如下所示。

```
print(tuple('123')) # 将字符串转换为单个字符的元组。
print(tuple([1,2,3])) # 将列表转换为元组。
```

```
print(tuple({1,2,3})) # 将集合转换为元组。
print(tuple({'a':1,'b':2,'c':3})) # 将字典中的键转换为元组。
```

14.3 元组常用统计函数

元组的统计函数与列表的统计函数相同，这里做一下简单介绍，案例代码如下所示。

```
tup = (50,60,74,63,50,95,74,80,50) # 被统计的元组。
print(len(tup)) # 计数
print(max(tup)) # 最大值
print(min(tup)) # 最小值
print(sum(tup)) # 求和
print(tup.count(50)) # 条件计数
print(tup.index(80)) # 条件定位
```

15. Python 无序对象--字典处理技术

Python中的字典是放在花括号中的，字和说明之间用冒号：分隔。

Python 中字典的标准表示方法为 {key:value，……} 。

- key(键) 在字典中必须具有唯一性，且必须是不可变对象，如字符串、数字或元组。
- value(值) 可以重复，也可以是任何数据类型，如字符串、元组、列表、集合等。
- 字典是无序的，只能通过键来存取对应的值，而不能像列表那样通过索引位置来存取对应的值。

15.1 字典的基础操作

本节将讲解创建字典及对字典进行存取操作的方法。通过一些相关的小案例，介绍在处理 Excel 数据时如何应用字典。

15.1.1 字典的创建与删除

下面介绍字典的创建与删除，案例代码如下所示。

```
dic1=dict();print(dic1) # 使用dict类创建空字典。
dic2=dict(王五=22,麻子=24);print(dic2) # 使用dict类创建字典。
dic3={};print(dic3) # 直接使用{}创建空字典。
dic4={'张三':18,'李四':20};print(dic4) # 直接使用{}创建字典。
del dic4 # 删除指定的字典。
```

15.1.2 字典中键值的获取

想获取字典中每个键对应的值，该怎么办？想获取字典中所有的键或所有的值，又该如何操作？案例

代码如下所示。

```
dic={'张三': 18, '李四': 20} # 准备的字典。
print(dic['李四']) # 获取指定键对应的值。
print(dic.keys()) # 获取字典的所有键。
print(dic.values() )# 获取字典的所有值。
print(dic.items()) # 获取字典的所有键值。
```

15.2 字典键值的修改、增加和删除

字典键值的修改、增加、删除与前面讲解的列表、元组的相关操作类似，但是也有不同之处，下面分别讲解。

15.2.1 字典键值的增加

向字典中增加更多的键值，一般使用update函数，也可以用修改键值的方式操作。

看看下面的小例子，代码如下所示。

```
dic={} # 空字典
dic.update(李四=88);print(dic) # 使用update函数向dic字典添加键值 方法1。
dic.update({'麻子':96});print(dic) # 使用update函数dic字典添加键值 方法2。
dic['张三']=99;print(dic) # 修改方法向dic字典添加键值。
```

15.2.2 字典键值的删除

删除字典键值可以使用pop函数、clear函数和del语句。下面是删除字典键值的小例子，代码如下所示。

```
dic={'张三':84,'李四':88,'王二':79,'麻子':99} # 准备的字典。
print(dic.pop('张三'));print(dic)  # 删除指定键值。
del dic['李四'];print(dic) # 删除指定键值。
dic.clear();print(dic) # 清空字典。
```

15.2.3 字典键值的修改

修改字典中键对应的值，表示方法为：字典名[键]=修改的值。如果是修改键呢?

实际上没有直接修改键的方法，可以使用间接的方式，表示方法为：字典名[新键] = 字典名.pop(旧键)。案例代码如下所示。

```
dic={'张三':20,'李四':18,'麻子':35} # 准备的字典。
dic['张三']=100;print(dic) # 修改键值中的值。
dic['王五']=dic.pop('李四');print(dic) # 修改键值中的键。
```

15.3 字典的转换

字典的转换操作是指将列表、元组等可迭代对象的元素转换为对应的字典，本节使用 dict 类和 dict.fromkeys函数完成转换。

15.3.1 类对象转换dict

在上节中使用 dict 类创建了字典对象，实际上，dict 类有三种不同的写法都可以生成字典对象。

本节将详细介绍 dict 类的相关语法。

dict 类语法：
dict(**kwargs)
dict(mapping,**kwargs)
cdict(iterable,**kwargs)

参数说明：

**kwargs:关键字，采用“键=值”的方式创建字典。
mapping:元素的容器，采用映射函数的方式创建字典。
iterable:可迭代对象，采用可迭代对象的方式创建字典。

下面演示使用dict类创建字典的三种方式，代码如下所示。

```
1 dic1=dict(a=1,b=2);print(dic1)
2 dic2=dict(zip(('a','b'),(1,2)));print(dic2)
3 dic3=dict([('a',1),('b',2)]);print(dic3)
```

- 第1行代码 dicl=dict(a=1,b=2)，使用 **kwargs 关键字的方式，也就是“键=值”的方式创建字典，可以用这种方式添加任意多个键值对。执行 print(dic1)后，返回结果为 {'a:1,'b:2} 。
- 第2行代码 dic2=dict(zip(('a','b'),(1,2)))，使用元素的容器方式，一般使用 zip 函数，比如 zip(('a',b'),(1,2))，表示将键放在一组，将值放在另一组。执行 print(dic2)后，返回结果为 {'a':1,'b:2} 。
- 第3行代码 dic3=dict([('a',1),('b',2)])，使用可迭代对象的方式，比如[('a',1),('b',2)]，将键与值放在同一个容器中，组织形式不定。元组、列表、字符串等可迭代对象只要结构正确均可以转换为字典。执行 print(dic3)后，返回结果为 {'a':1,'b:2} 。

15.3.2 dict.fromkeys 转换法

在Python中，除使用 dict 类创建字典外，还可以用 dict.fromkeys 函数创建一个新字典，下面介绍该函数的语法。

函数语法：

dict.fromkeys(seq[,value])

参数说明：
seq: 必选参数，字典的键，可以是元组、列表、字符串等可迭代对象。
value: 可选参数，设置键对应的值，默认值为None。

下面是几个小例子，代码如下所示。

```
dic1=dict.fromkeys(('a','b'),1);print(dic1)
dic2=dict.fromkeys(['a','b'],1);print(dic2)
dic3=dict.fromkeys('abc',1);print(dic3)
dic4=dict.fromkeys(['a','a','b']);print(dic4)
dic5=dict.fromkeys([('a',1),('a',1)]);print(dic5)
```

- 第 1 行代码 dicl=dict.fromkeys(('a','b'),1)，将元组('a','b')作为键，键的值统一为1。运行print(dic1)后，返回字典｛'a:1,'b:1｝。
- 第 2 行代码 dic2=dict.fromkeys(['a','b'],1)，将列表['a','b]作为键，键的值统一为1。运行print(dic2)后，返回字典｛'a:1,'b':1｝。
- 第 3 行代码 dict.fromkeys('abc',1)，将字符串'abc'的每个字符作为键，键的值统一为1。运行print(dic3)后，返回字典｛'a':1,'b':1,'c':1｝。
- 第 4 行代码 dic4=dict.fromkeys(['a','a','b'])，将列表['a','a','b']作为键，列表中有相同的元素，只保留一个。所以运行 print(dic4)后，返回字典｛'a':None, 'b':None｝。
- 第 5 行代码 dic5=dict.fromkeys([('a',1),('a',1)])，将列表[('a',1),('a',1)]作为键，字典的键是元组，而且两个元组相同，这时会去掉重复的，只保留一个。运行 print(dic5)后，返回字典｛('a',1):None｝。

再次提醒读者，字典的键可以为数字、字符串、元组等不可变类型的数据，不能为列表、集合等可变类型的数据。

16. Python 无序对象 -- 集合处理技术

在Python中，集合是一个无序的不重复元素序列，元素被放在 `{}` 中，是可迭代的，不支持任何索引或切片操作。与列表相比，集合的主要优点是具有高度优化的方法，可以检查集合中是否包含特定元素，也可以进行并集、交集、差集、比较等操作。

16.1 集合的创建与删除

集合与列表、元组、字典一样，可以创建和删除。集合创建与删除的案例代码如下所示。

```
set1 = set();print(set1) # 创建空集合。
set2 = {1,2,3};print(set2) # 创建有元素的集合。
set3 = frozenset(set2);print(set3) # 转换为不可变集合。
del set1 # 删除指定集合。
```

- 第 1 行代码 set1 = set()，使用 set 类创建空集合，运行 print(set1) 后，返回结果为 set()。创建空集合必须使用 set()，而不能用 `{}`，因为 `{}` 是创建空字典。
- 第 2 行代码 set2 = {1，2，3}，将元素放在花括号中，运行 print(set2) 后，返回结果为 {1，2，3}，此种集合是可变集合，也就是可变对象，不能作为字典的键。
- 第 3 行代码 set3 = frozenset(set2)，在 set2 集合外层套上 frozenset 类，运行 print(set3) 后，返回结果为frozenset({1，2，3})。frozenset 函数的参数不一定是可变集合，可以是任何可迭代对象。不可变集合类似字符串、元组，可以作为字典的键，缺点是一旦创建便不能更改，除内容不能更改外，其他功能及操作与可变集合set一样。
- 第 4 行代码 del set1，删除指定的集合，删除后集合将不存在。

16.2 集合元素的添加与删除

我们知道了集合可以分为可变集合和不可变集合，本节讲解集合元素的添加与删除，当然是针对可变集合而言的。

16.2.1 集合元素的添加

向集合中添加单个元素或多个元素，可以使用 add 函数或 update 函数，来看看它们的区别是什么。案例代码如下所示。

```
set1 = {1,2,3} # 原集合
set1.add(4);print(set1) # 向集合添加单个元素。
set1.update({5,6,7});print(set1) # 向集合添加多个元素。
```

- 第 2 行代码 set1.add(4),表示向 set1 集合中添加元素 4,运行 print(set1) 后,返回的结果为 {1,2,3,4}。
- 第 3 行代码 set1.update({5,6,7}),表示向 set1 集合中添加另一个集合 {5,6,7},运行 print(set1) 后,返回的结果为 {1,2,3,4,5,6,7}。实际上,update 函数的参数可以是任何可迭代对象,比如字符串、列表、元组等。

> 注意，如果集合中的所有元素都是数字，则这些数字会按照从小到大的顺序排列，否则排列是混乱的。为了让读者看到演示效果，这里将数字作为集合的元素。

16.2.2 集合元素的删除

删除集合中的元素可以用 `remove、discard、pop、clear` 函数，根据不同的要求用不同的函数。

案例代码如下所示。

```
set1 = {'a','b','c','d'} # 原集合
```

```
1 set1.remove('a');print(set1) # 删除集合中元素'a'。
2 set1.discard('b');print(set1) # 删除集合中元素'b'。
3 set1.pop();print(set1) # 随机删除集合中的一个元素。
4 set1.clear();print(set1) # 清空集合中所有元素
5
```

- 第 2 行代码 set1.remove('a')，表示删除集合中的指定元素'a'，如果元素不存在，则返回错误。运行print(set1) 后，返回的结果为 {'c'，'d'，'b'}。
- 第 3 行代码 set1.discard('b')，表示删除集合中的指定元素b'，如果元素不存在，则忽略，不会返回错误。运行 print(set1) 后，返回的结果为 {'d'，'c'}。
- 第 4 行代码 set1.pop()，表示随机删除集合中的一个元素。运行 print(set1) 后，如果删除的是 'd'，则返回{'c'}；如果删除的是 'c'，则返回 {'d'}。
- 第 5 行代码 set1.clear()，表示清空集合中的所有元素。运行 print(set1) 后，返回的结果为 set()。

> 注意，由于集合中的元素不是数字，所以每次运行第 2、3、4 行代码返回的结果的顺序会有所不同。

16.3 集合之间的大小比较

集合之间的大小比较就是判断某个集合是否完全包含另一个集合，可以使用大于（>）、小于（<）、大于或等于（>=）、小于或等于（<=）、等于（==）、不等于（!=）这些比较运算符来做判断。

16.3.1 集合的比较运算

下面用几个小例子讲解集合之间是如何进行比较的，代码如下所示。

```
1 print(2 in {1,2,3}) # 2是否包含在集合中。
2 print({1,2,3}=={3,2,1}) # 两个集合是否相同。
3 print({2,3}>{1,2,3}) # 集合大于比较。
4 print({2,3}<{1,2,3}) # 集合小于比较。
```

- 第1行代码 print(2 in{1,2,3}),判断集合 {1,2,3} 中是否包含 2,返回的结果为 True。
- 第2行代码 print({1,2,3}={3,2,1}),判断 {1,2,3} 与 {3,2,1} 是否相等,集合是无序的,只要两个集合中的数据相同就相等,返回的结果是 True。
- 第3行代码 print({2,3}>{1,2,3}),其本质是判断 {1,2,3} 中的元素是否被 {2,3} 包含,返回的结果为 False。
- 第4行代码 print({2,3}<{1,2,3}),其本质是判断 {2,3} 中的元素是否被 {1,2,3} 包含,返回的结果为 True。

16.3.2 集合比较运算应用案例：判断指定的多个等级是否存在

在“评级表”工作表中，1～4 季度每个人有不同的等级，判断包含“优”和“良”两个等级的有哪些人，将结果写在F列，如图所示。



处理前

	A	B	C	D	E	F
1	月份	1季度	2季度	3季度	4季度	判断
2	张三	差	良	中	优	
3	李四	优	差	良	差	
4	王二	中	差	良	差	
5	麻子	优	良	良	优	
6	小明	差	中	优	差	
7	小林	良	优	中	差	

评级表

处理后

	A	B	C	D	E	F
1	月份	1季度	2季度	3季度	4季度	判断
2	张三	差	良	中	优	√
3	李四	优	差	良	差	√
4	王二	中	差	良	差	×
5	麻子	优	良	良	优	√
6	小明	差	中	优	差	×
7	小林	良	优	中	差	√

评级表

本案例的编程思路是将每个人4个季度的等级转换成集合，然后与集合｛'优，良｝进行比较判断，如果包含｛'优，良｝，则条件成立。

```
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导入函数
# 读取工作簿和工作表。
wb = xlrd.open_workbook('./25-Scores-1.xls');ws=wb.sheet_by_name('评级表')
nwb = copy(wb);nws=nwb.get_sheet('评级表') # 复制工作簿及读取副本工作簿下的工作表。
row_num = 0 # 初始化row_num变量为0。
for row in tuple(ws.get_rows())[1:]: # 循环读取每行记录。
    row_num += 1 # 对row_num变量累加1。
    if {'优','良'}<={v.value for v in row[1:-1]}: # 判断每行的等级是否包
含'优'和'良'。
        nws.write(row_num,5,'√') # 如果包含则写入'√'。
    else: #否则
        nws.write(row_num,5,'×') # 如果不包含则写入'×'。
nwb.save('./25-Scores-2.xls') # 保存副本工作簿。
```

- 第 1～5 行代码为数据的读取和写入做准备。
- 第 6 行代码 for row in tuple(ws.get_rows()[1:]:，将每行数据循环赋值给 row 变量。
- 第 8 行代码 if {'优','良'}< =｛v.value for v in row[l:－1]}:， 其中，{v.value for v in row[1:－1]} 将 row 元组中的元素使用集合推导式转换为集合，集合推导式也具有去重功能。然后判断集合是否大于或等于{'优','良'}，也就是{'优','良'}这个集合是否被包含或刚好与集合相等。
- 第 9 行代码 nws.write(row_num,5,'✓')，如果第 8 行代码的条件成立，则将 “✓” 写入F列。
- 第 11 行代码 nws.write(row_num,5,'x')，如果第 8 行代码的条件不成立，则将 “x” 写入F列。
- 第 12 行代码 nwb.save('Chapter-8-6-1.xls')，保存副本工作簿。

16.4 集合的转换

将可迭代对象转换为集合，除使用集合推导式外，还可以使用set类，这种方式更直接、简单。

16.4.1 类对象转换 set

前面学习过使用 set 类创建空集合，实际上也可以使用set类将字符串、列表、元组等可迭代对象转换为集合。

类语法：**set([iterable])**

参数说明：

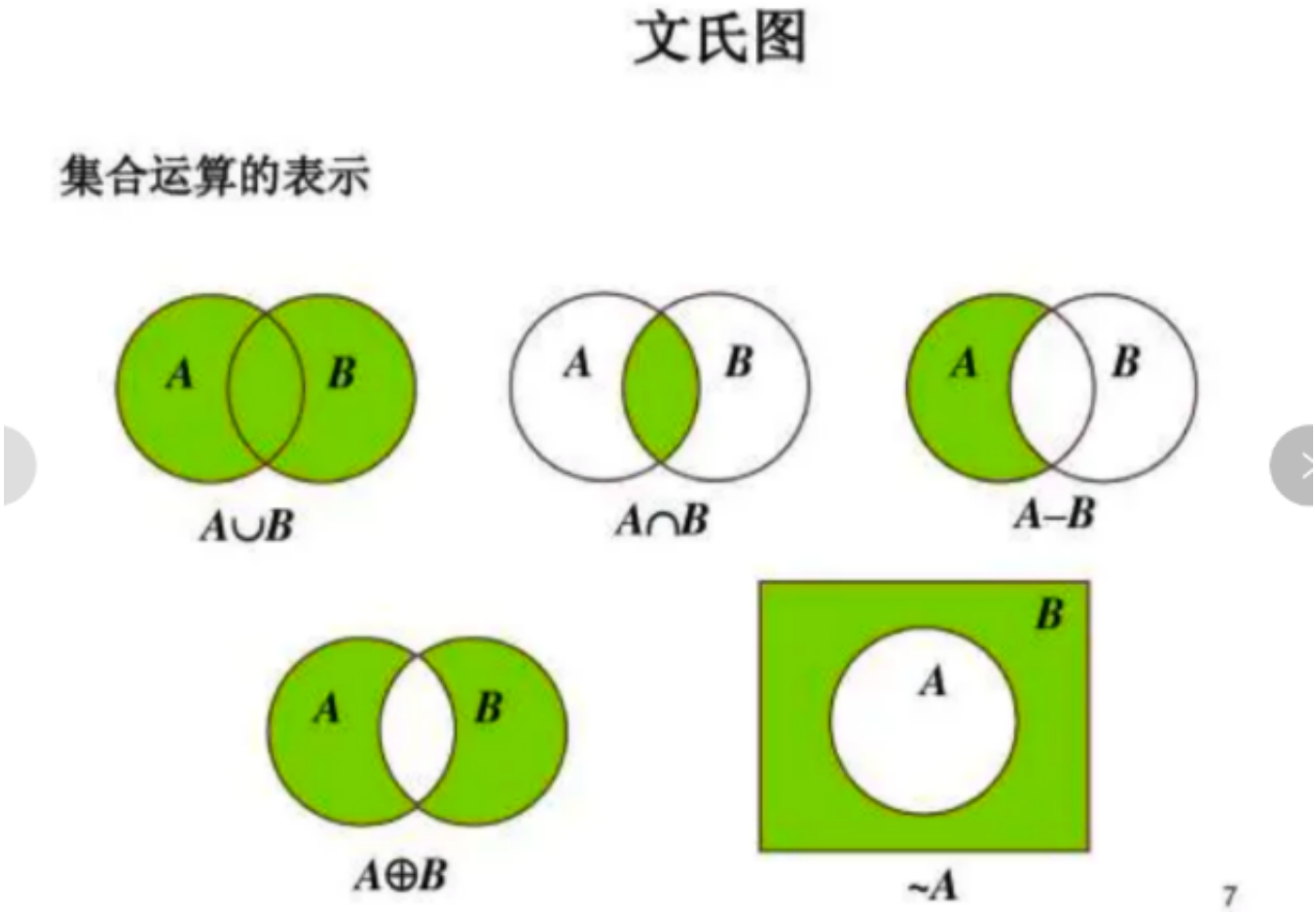
iterable:可选参数，要转换为集合的可迭代序列。

下面是使用set类进行转换的例子，代码如下所示。

```
set1 = set('123');print(set1) # 将字符串转换为集合。
set2 = set([1,2,3]);print(set2) # 将列表转换为集合。
set3 = set((1,2,3));print(set3) # 将元组转换为集合。
set4 = set({'a':1,'b':2,'c':3});print(set4) # 将字典转换为集合。
set5 = set(range(1,4));print(set5) # 将可迭代对象转换为集合。
print(set('aadad'))
```

- 第 1 行代码 set1＝set('123'),将字符串 '123' 转换为集合,运行 print(set1) 后,返回的结果为 {'1','3','2'}。
- 第 2 行代码 set2＝set([1,2,3]),将列表 [1,2,3] 转换为集合,运行 print(set2) 后,返回的结果为 {1,2,3}。
- 第 3 行代码 set3＝set(1,2,3)),将元组 (1,2,3) 转换为集合,运行 print(set3) 后,返回的结果为 {1,2,3}。
- 第 4 行代码 set4＝set({'a'：1,b'：2,'c'：3}),将字典 {'a':1,'b':2,'c':3} 的键转换为集合,运行 print(set4) 后,返回的结果为 {'b','c','a'}。
- 第 5 行代码 set5＝set(range(1,4)),将可迭代对象 range(1,4) 换为集合,运行 print(set5) 后,返回的结果为 {1,2,3}。

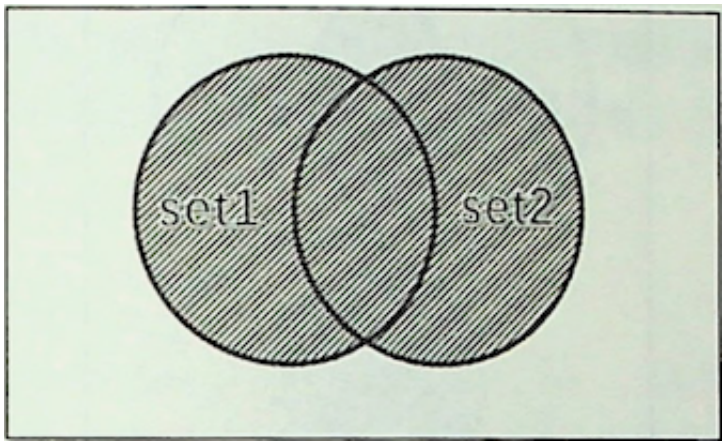
16.5 集合的运算



在 Python 中，集合之间可以做关系运算，例如做并集、交集、差集及对称差集运算。本节将逐个讲解集合的运算，每种集合运算都有配图，图中用斜线填充的部分就是集合运算后得到的结果。

16.5.1 并集运算

并集运算指两个或更多集合合并，结果中包含了所有集合的元素，重复的元素只会出现一次，可以参考图所示的效果。下面以 set1 和 set2 两个集合为例讲解并集运算的原理，代码如下所示。



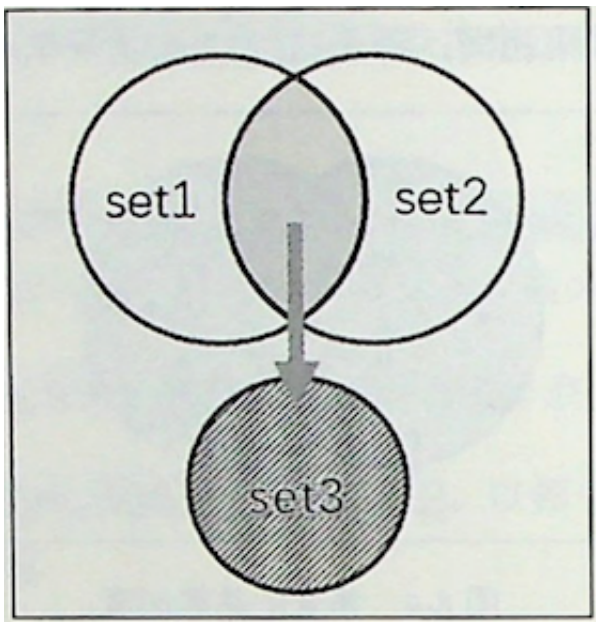

```
set1 = {1,2,3,4,5,6} # 集合1。
set2 = {4,5,6,7,8,9} # 集合2。
print(set1.union(set2)) # 集合1与集合2做并集。
print(set2.union(set1)) # 集合2与集合1做并集。
```

```
print(set1|set2) # 集合1与集合2做并集。
print(set2|set1) # 集合2与集合1做并集。
```

- 第 3 行和第 4 行代码将 set1 与 set2 两个集合进行合并,使用 union 函数完成,无论写法是 set1.union(set2),还是 set2.union(set1),结果都相同,返回的结果都是 {1,2,3,4,5,6,7,8,9}。
- 第 5 行和第 6 行代码使用 "|" 符号将 set1 和 set2 两个集合进行合并,set1|set2 与 set2|set1 两种写法的运算结果相同,都是{1,2,3,4,5,6,7,8,9}。

16.5.2 交集运算

交集运算求两个或更多集合中都包含的元素。下面以 set1 和 set2 两个集合为例讲解交集运算的原理，代码如下所示。




```
set1 = {1,2,3,4,5,6 } # 集合1。
set2 = {4,5,6,7,8,9} # 集合2。
set3 = set1.intersection(set2);print(set3) # 集合1与集合2交集，结果可以存储在新集合3。
set3 = set2.intersection(set1);print(set3) # 集合2与集合1交集，结果可以存储在新集合3。
set3 = set1&set2;print(set3) # 集合1与集合2交集，结果可以存储在新集合3。
set3 = set2&set1;print(set3) # 集合2与集合1交集，结果可以存储在新集合3。
set1.intersection_update(set2);print(set1) # 集合1与集合2交集，交集结果存储在集合1。
set2.intersection_update(set1);print(set2) # 集合2与集合1交集，交集结果存储在集合2。
```

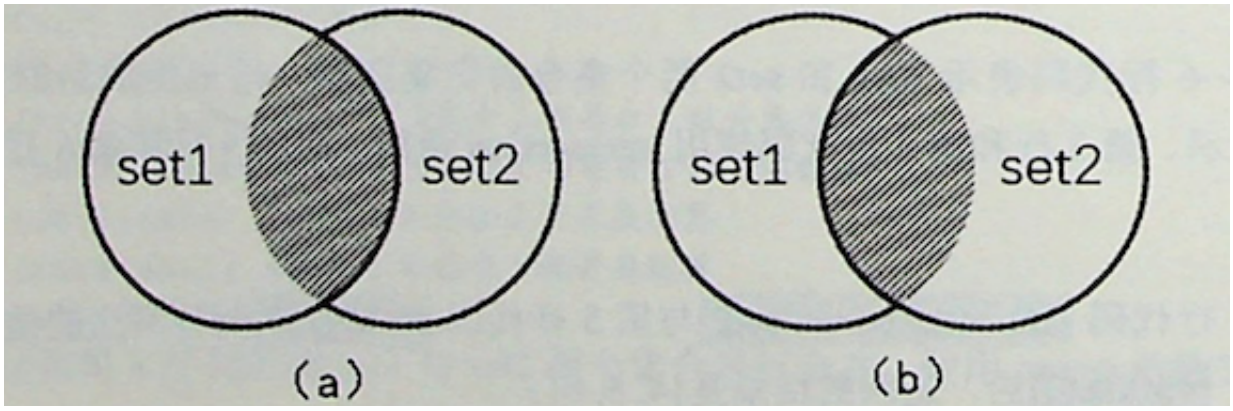
- 第 3～6 行代码表示 set1 和 set2 两个集合做交集运算，结果生成新的集合，如图所示。第 3 行和第 4 行代码使用 intersection 函数，而第 5 行和第 6 行代码使用 & 符号。
    - 第 3 行代码 set1.intersection(set2) 与第 5 行代码 set1& set2 两种写法的结果是等价的。运行 print(set3) 后，返回的结果是 {4，5，6}。

- 第 4 行代码 set2.intersection(set1)与第 6 行代码 set2&set1 两种写法的结果是等价的。运行 print(set3) 后，返回的结果是 {4，5，6}。

- 第 7 行和第 8 行代码虽然也是 set1 和 set2 两个集合做交集运算，但返回结果的存储方式有所不同。

  - 第 7 行代码 set1.intersection_update(set2)，以 set1 为存储集合与 set2 集合做交集运算，也就是交集结果存储在 set1 中，如图(a) 所示。运行 print(set1)后，返回的结果为 {4，5，6}。

  - 第8行代码 set2.intersection_update(set1)，以 set2 为存储集合与 set1 集合做交集运算，也就是交集结果存储在 set2 中，如图(b) 所示。此行代码在运行时，set2 集合中的 {4，5，6，7，8，9} 与 set1 集合中的 {4，5，6} 做交集运算，为什么 set1 集合变成了 {4，5，6}？原因是第 7 行代码在运行时，set1 中存储了交集的结果。运行 print(set2) 后，返回的结果虽然也是 {4，5，6}，但要明白其中的变化。



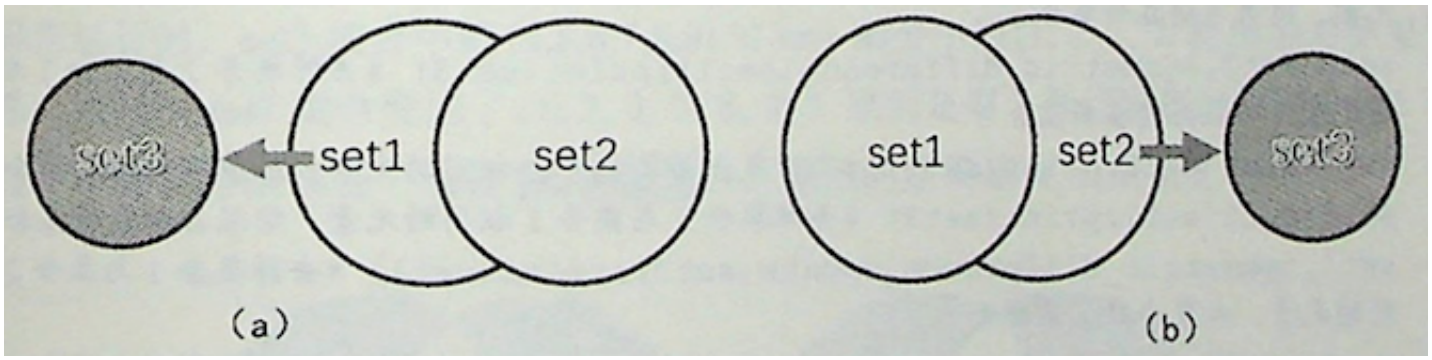
16.5.3 差集运算

差集运算就是两个集合相减，用一个集合中的元素减去另一个集合中的元素。下面以 set1 和 set2 两个集合为例讲解差集运算的原理，代码如下所示。

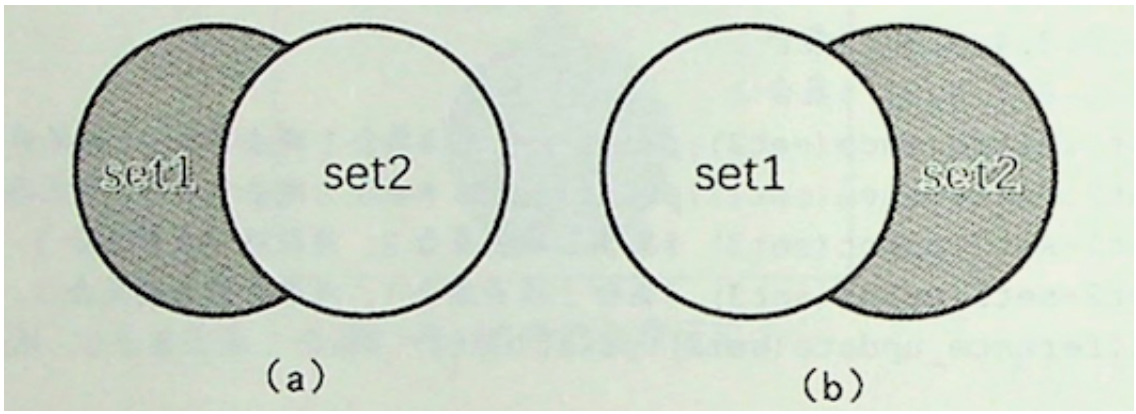
```
1 set1 = {1,2,3,4,5,6} # 集合1。
2 set2 = {4,5,6,7,8,9} # 集合2。
3 set3 = set1.difference(set2);print(set3) # 集合1减去集合2，结果可以存储在新集合
4 3。
5 set3 = set2.difference(set1);print(set3) # 集合2减去集合1，结果可以存储在新集合
6 3。
7 set3 = set1-set2;print(set3) # 集合1减去集合2，结果可以存储在新集合3。
8 set3 = set2-set1;print(set3) # 集合2减去集合1，结果可以存储在新集合3。
  set1.difference_update(set2);print(set1) # 集合1减去集合2，差集结果存储在集合1
  中。
  set2.difference_update(set1);print(set2) # 集合2减去集合1，差集结果存储在集合2
  中。
```

- 第 3～6 行代码是 set1 和 set2 两个集合做差集运算，结果生成新的集合。第 3 行和第 4 行代码使用difference 函数，而第 5 行和第 6 行代码使用减号(－)。

- 第 3 行代码 set1.difference(set2)) 与第 5 行代码 set1-set2 两种写法的结果是等价的，如图(a)所示。运行 print(set3) 后，返回的结果是{1，2，3}。
- 第 4 行代码 set2.difference(set1) 与第 6 行代码 set2-set1 两种写法的结果是等价的，如图(b)所示。运行 print(set3)后，返回的结果是{1，2，3}。



- 第 7 行代码 set1.difference_update(set2)，以 set1 为存储集合与 set2 集合做差集运算，也就是差集结果存储在 set1中，如图(a)所示。运行 print(set1) 后，返回的结果为 {1，2，3}。
- 第 8 行代码 set2.difference_update(set1)，以 set2 为存储集合与 set1集合做差集运算，也就是差集结果存储在 set2 中，如图(b)所示。此行代码在运行时，set2 集合中的 {4，5，6，7，8，9} 与 set1 集合中的 {1，2，3} 做差集运算，为什么 set1 集合变成了 {1，2，3}? 原因是第 7 行代码在运行时，set1 中存储了差集的结果。运行 print(set2) 后，返回的结 果是 {4，5，6，7，8，9}。



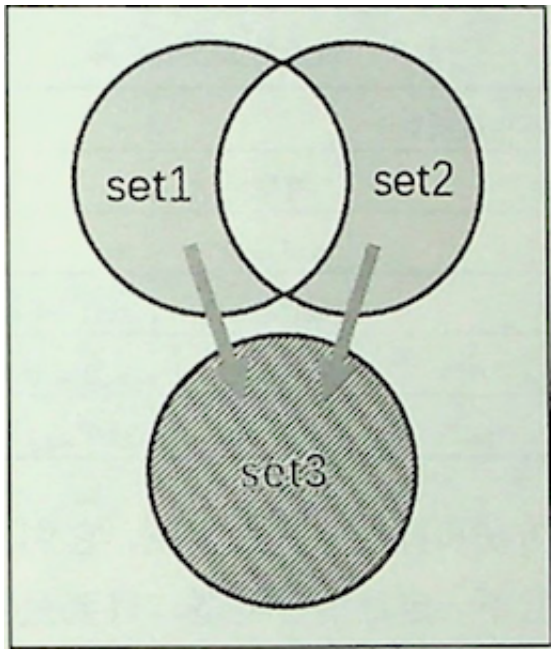
16.5.4对称差集运算

对称差集运算返回两个集合中不重复的元素集合，即移除两个集合中都存在的元素。下面以 set1 和 set2 两个集合为例讲解对称差集运算的原理，代码如下所示。

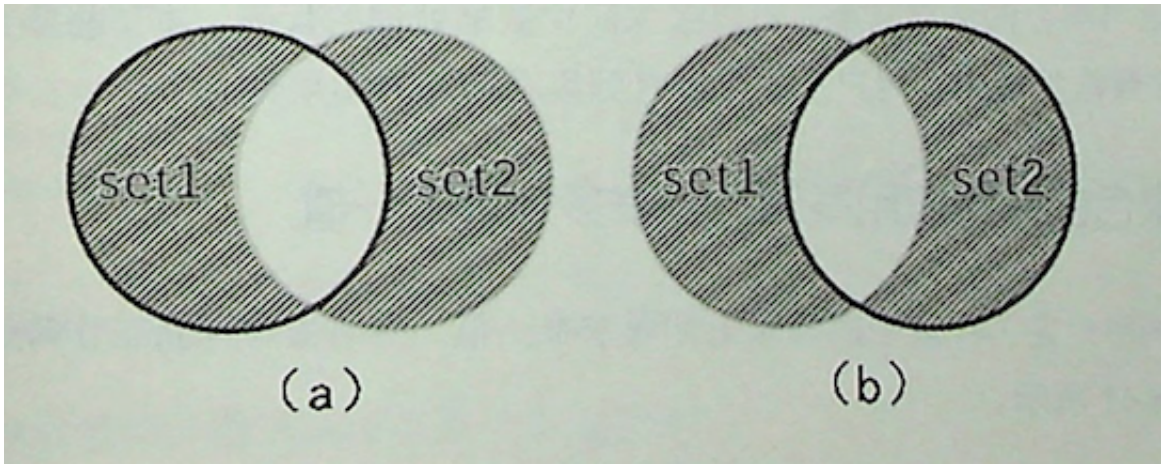
```
set1 = {1,2,3,4,5,6} # 集合1。
set2 = {4,5,6,7,8,9} # 集合2。
set3 = set1.symmetric_difference(set2);print(set3) # 去掉集合1与集合2相同元素，
结果存储在新集合3。
set3 = set2.symmetric_difference(set1);print(set3) # 去掉集合2与集合1相同元素，
结果存储在新集合3。
set3 = set1^set2;print(set3) # 去掉集合1与集合2相同元素，结果存储在新集合3。
set3 = set2^set1;print(set3) # 去掉集合2与集合1相同元素，结果存储在新集合3。
```

```
set1.symmetric_difference_update(set2);print(set1) # 去掉集合1与集合2相同元素，结果可以存储在集合1。
set2.symmetric_difference_update(set1);print(set2) # 去掉集合2与集合1相同元素，结果可以存储在集合2。
```

- 第 3～6 行代码是 set1 和 set2 两个集合做对称差集运算，结果生成新的集合。第 3 行和第 4 行代码使用 symmetric_difference 函数，第 5 行和第 6 行代码使用 ^ 符号。对称差集运算示意图如图所示。
    - 第3行代码 set1.symmetric_difference(set2) 与第 5 行代码 set1^set2 两种写法的结果是等价的。运行 print(set3) 后，返回的结果是 {1，2，3，7，8，9} 。
    - 第4行代码 set2.symmetric_difference(set1) 与第 6 行代码 set2 ^ set1 两种写法的结果是等价的。运行 print(set3) 后，返回的结果是 {1，2，3，7，8，9} 。



- 第 7 行代码 set1.symmetric_difference_update(set2)，以 set1 为存储集合与 set2 集合做对称差集运算，也就是对称差集结果存储在 set1 中，如图(a)所示。运行print(set1)后，返回的结果为 {1，2，3，7，8，9} 。
- 第 8 行代码 set2.symmetric_difference_update(set1)，以 set2 为存储集合与 set1 集合做对称差集运算，也就是对称差集结果存储在 set2 中，如图(b)所示。此行代码在运行时，set2 集合中的 {4，5，6，7，8，9} 与 set1 集合中的 {1，2，3，7，8，9} 做对称差集运算，为什么 set1 集合变成了 {1，2，3，7，8，9} ？原因是第 7 行代码在运行时，set1 中存储了对称交集的结果。运行 print(set2) 后，返回的结果是 {1，2，3，4，5，6} 。



16.5.5 集合运算小结

假如现在有 set1 和 set2 两个集合，可以将集合之间的运算用表格做一下总结，如表所示。

集合之间的运算

集合运算方式	运算结果生成新集合		运算结果存储在原集合
	函数方法	符号方法	函数方法
并集	set1.union(set2)	set1|set2	—
交集	set1.intersection(set2)	set1&set2	set1.intersection_update(set2)
差集	set1.difference(set2)	set1-set2	set1.difference_update(set2)
对称差集	set1.symmetric_difference(set2)	set1^set2	set1.symmetric_difference_update(set2)

集合的运算可以用函数方法和符号方法来实现，它们之间的区别是什么呢?

- 如果使用函数方法，则函数的参数不一定是集合类型，只要是可迭代对象即可。
- 如果使用符号方法，则符号两侧必须是集合类型，否则会出错。

在通过表格的方式总结集合之间的运算时，会发现一个问题，为什么并集中没有将运算结果存储在原集合中的方法？也就是说，为什么没有 `set1.union_update(set2)` 这种方法？虽然没有 `union_update` 函数来实现，但有另一种方式可以实现，方法为 `set1.update(set2)`，没错，就是利用集合的 `update` 函数来实现的。虽然 update 函数是向指定集合中添加元素，但从另一个角度看，就是两个集合的并集，而且并集结果也没有生成新集合，是存储在原集合中的。

17. 总结

17.1 Python 数据结构总结

以下是 Python 中常用的数据结构（列表、元组、字典、集合）的特性总结及对比表：

特性	列表 (List)	元组 (Tuple)	字典 (Dictionary)	集合 (Set)

有序性	有序	有序	无序	无序
是否可重复	可以重复	可以重复	键不可重复，值可重复	不可重复
是否可遍历	可以通过迭代遍历	可以通过迭代遍历	可以通过 `keys()`、`values()` 和 `items()` 遍历	可以通过迭代遍历
是否可变	可以改变	不可以改变	可以改变	可以改变
拷贝后的内存地址	`copy` 和 `deepcopy` 内存地址都不同	- 若包含可变对象：`copy` 地址相同，`deepcopy` 地址不同 - 若不包含可变对象：3者地址相同	`copy` 和 `deepcopy` 地址都不同	`copy` 和 `deepcopy` 地址都不同
常用方法	`append()`，`extend()`，`pop()` 等	无（只读操作，如 `count()`）	`get()`，`keys()`，`values()` 等	`add()`，`remove()`，`union()` 等
典型应用场景	存储有序数据，可修改内容	存储有序、不可修改的数据	键值对映射，用于快速查找	去重操作、数学集合运算

17.1.1 详细解释

1. **列表（List）**：
   - 有序结构，数据可以重复。
   - 支持索引和切片操作，可以动态修改内容。
   - 拷贝时无论是 `copy` 还是 `deepcopy`，新对象的内存地址都会变化。
2. **元组（Tuple）**：
   - 有序结构，数据可以重复。
   - 不可变对象，不能动态修改内容。
   - 如果元组内部包含可变对象（如列表），`copy` 后的内存地址与原元组相同，而 `deepcopy` 会改变内存地址。
3. **字典（Dictionary）**：

- 无序结构，键必须唯一，但值可以重复。
- 支持通过键快速查找值，常用于存储映射关系。
- 拷贝时，无论是 `copy` 还是 `deepcopy`，新字典的内存地址都会变化。

4. **集合（Set）**：
   - 无序结构，数据不能重复。
   - 常用于去重或进行集合间的数学运算（如交集、并集等）。
   - 拷贝后，`copy` 和 `deepcopy` 都会生成新的内存地址。

17.1.2 使用建议

- **列表**：适合需要有序、可变的数据存储。
- **元组**：适合存储固定、不变的数据（如函数返回多个值）。
- **字典**：适合键值对存储、快速查找的场景。
- **集合**：适合去重或集合运算（如判断两个集合是否有交集）。

阿里云天池 Python 训练营 - Part3

18. Python 自定义函数

函数是Python编程中的核心内容之一。

函数的最大优点是可以增强代码的重用性和可读性。

Python 中有很多内置函数，但内置函数并不能满足用户的需求，这时用户可以创建自定义函数。

本章将介绍如何自定义函数。

18.1 自定义函数编写规范

Python中的自定义函数需要遵循一定的规范，比如定义时的关键字是什么、返回值关键字是什么、参数的写法，以及如何调用自定义函数等。下面来介绍一下相关的知识点。

18.1.1 函数的定义

在自定义函数时，有固定的语法格式。语法格式如下：

```
def 函数名（参数）：
    函数体
```

```
    return 返回值
```

- def：函数的关键字，不能没有，也不能改，有此关键字就表示是自定义函数。
  - 函数名：函数的名称，根据函数名调用函数。
  - 函数名是包含字母、数字、下划线的任意组合，不能以数字开头。
  - 函数名可以任意命名，建议尽量定义成可以表示函数功能的名称。
- 参数：为函数体提供数据。
  - 参数是被放在小括号中的，
  - 可以设置多个参数，
  - 参数之间用逗号分隔。
  - 在小括号的后面加冒号。
- return 返回值：
  - 整体作为自定义函数的可选参数，用于设置自定义函数的返回值。
  - 也就是说，自定义函数可能有返回值，也可能没有返回值，视情况而定。

> 注意，函数体和返回值语句要做缩进处理，也就是从冒号开始换行的语句都要做缩进处理。

18.1.2 自定义函数的创建与调用

了解了自定义函数的语法格式后，下面学习自定义函数的创建与调用，比如创建一个单价与数量相乘的函数并调用。

> 因为代码是自上而下运行的，所以一定要先定义函数，再调用函数。

自定义函数案例的代码如下所示。

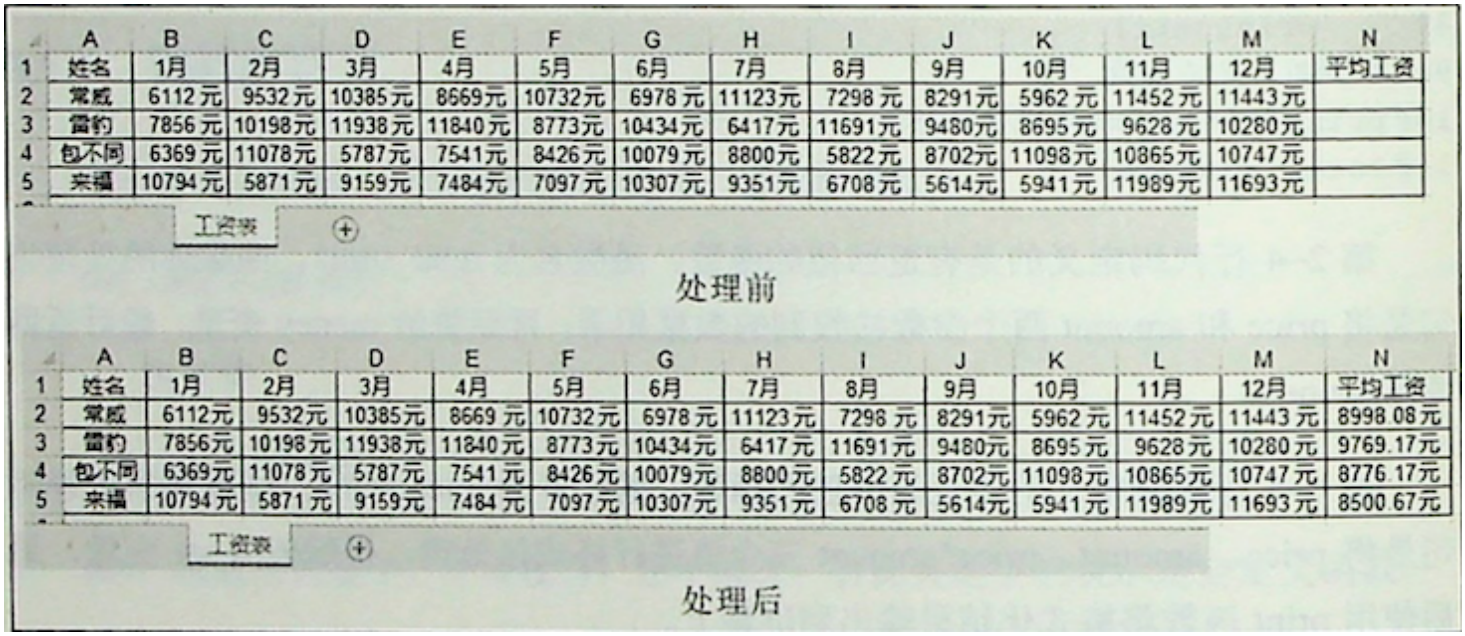
```
# 有返回值的自定义函数
def total_sum1(price,amount):
    money=price*amount
    return money

# 无返回值的自定义函数
def total_sum2(price,amount):
    str='单价:{} 数量:{} 金额:{}'.format(price,amount,price*amount)
    print(str)

# 调用自定义函数
print(total_sum1(10,20))
total_sum2(10,20)
```

18.1.3 自定义函数应用案例：平均函数的定义及应用

本案例将每个人1～12月的工资进行平均，然后写入N列对应的单元格，如图所示。



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	姓名	1月	2月	3月	4月	5月	6月	7月	8月	9月	10月	11月	12月	平均工资
2	常威	6112元	9532元	10385元	8669元	10732元	6978元	11123元	7298元	8291元	5962元	11452元	11443元	
3	雷豹	7856元	10198元	11938元	11840元	8773元	10434元	6417元	11691元	9480元	8695元	9628元	10280元	
4	包不同	6369元	11078元	5787元	7541元	8426元	10079元	8800元	5822元	8702元	11098元	10865元	10747元	
5	宋福	10794元	5871元	9159元	7484元	7097元	10307元	9351元	6708元	5614元	5941元	11989元	11693元	

工资表

处理前

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	姓名	1月	2月	3月	4月	5月	6月	7月	8月	9月	10月	11月	12月	平均工资
2	常威	6112元	9532元	10385元	8669元	10732元	6978元	11123元	7298元	8291元	5962元	11452元	11443元	8998.08元
3	雷豹	7856元	10198元	11938元	11840元	8773元	10434元	6417元	11691元	9480元	8695元	9628元	10280元	9769.17元
4	包不同	6369元	11078元	5787元	7541元	8426元	10079元	8800元	5822元	8702元	11098元	10865元	10747元	8776.17元
5	宋福	10794元	5871元	9159元	7484元	7097元	10307元	9351元	6708元	5614元	5941元	11989元	11693元	8500.67元

工资表

处理后

本案例的编程思路是将每个人所有的工资求和，再除以月份数，按此思路自定义一个平均函数即可。

```
# 自定义平均函数
def average(lst):
    num=sum(lst)/len(lst) # 平均处理。
    avg=float('{:.2f}'.format(num)) # 格式化平均值。
    return avg # 返回平均值。

# 数据平均处理
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb=xlrd.open_workbook('./28-functions-1.xls');ws=wb.sheet_by_name('工资表')
# 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('工资表') # 复制工作簿，以及读取副本工作簿下的工作表。
for row_num in range(1,ws.nrows): # 循环行号。
    salary_list=ws.row_values(row_num)[1:-1] # 获取1到12月份的工资列表。
    nws.write(row_num,13,average(salary_list)) # 调用平均函数average对salary_list列表求平均。
nwb.save('/28-functions-2.xls') # 保存副本工作簿。
```

- 第 2～5 行代码自定义平均函数average(lst)。其中，lst 为必选参数，指定要进行平均计算的列表或元组。
- 第 13 行代码 nws.write(row_num,13,average(salary_list))，其中 average(salary_list) 表示使用 average 函数对每行的工资数据进行平均，然后将结果写入 N 列对应的单元格。

18.2 必选参数的写法及应用

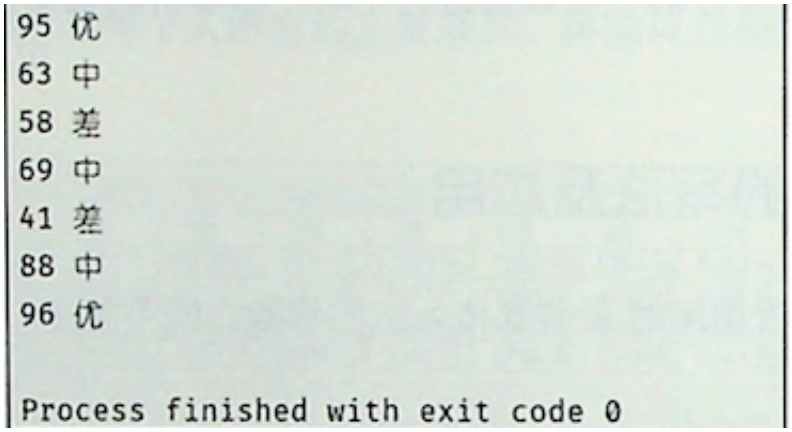
必选参数就是函数调用时必须要传入值的参数，既不能多，也不能少。必选参数也可以叫作位置参数。

18.2.1 必选参数（位置参数）

下面是一个小案例，根据固定的分数确定等级，代码如下所示。

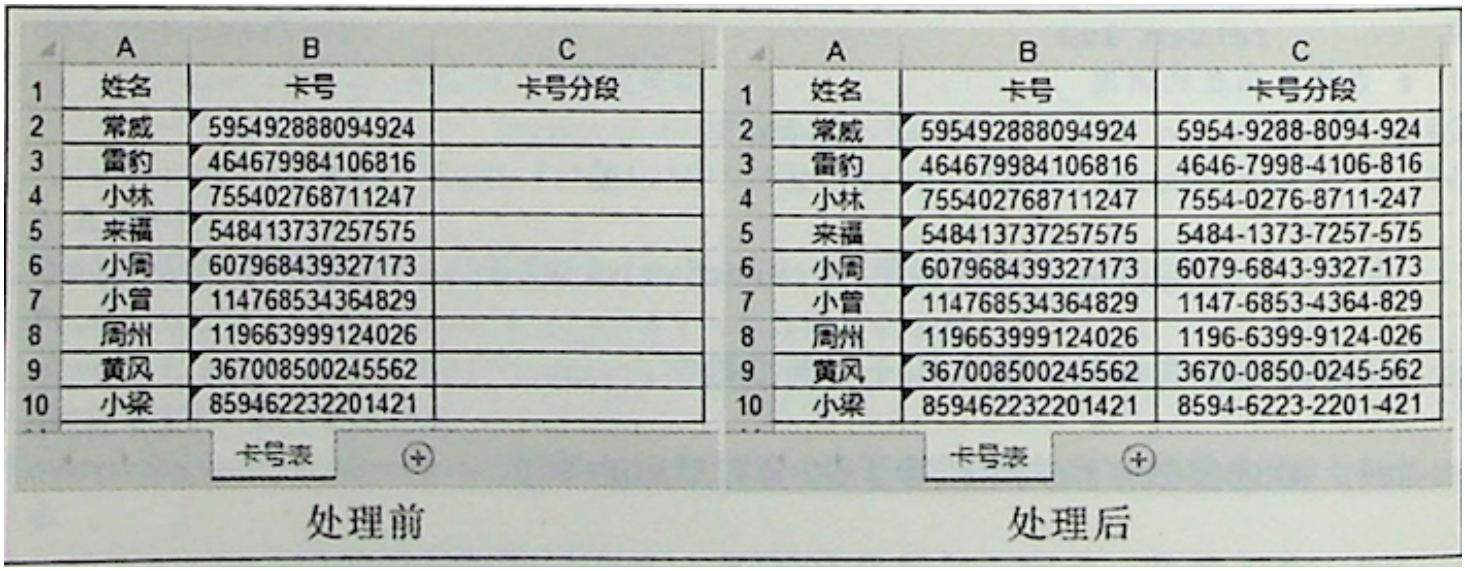
```
1  # 自定义函数。
2  def level(number,lv1,lv2,lv3):
3      if number>=90:
4          return lv1
5      elif number>=60:
6          return lv2
7      elif number>=0:
8          return lv3
9  # 自定义函数的调用。
10 for score in [95,63,58,69,41,88,96]:
11     print(score,level(score,'优','中','差'))
```

- 第 2～8 行代码自定义等级函数level(number,lv1,lv2,lv3)，该函数的参数说明如下。
    - number:必选参数，输入用于判断的数字。
    - lv1:必选参数，当大于或等于 90 分时对应的等级。
    - lv2:必选参数，当大于或等于 60 分且小于 90 分时对应的等级。
    - lv3:必选参数，当大于或等于 0 分且小于 60 分时对应的等级。
- 第 10 行和第 11 行代码将 [95,63,58,69,41,88,96] 列表中的元素循环赋值给 score 变量，score 变量作为level 函数的第 1 个参数，将第 2、3、4 个参数分别写入“优”“中”“差” 3 个等级。最后运行 print(score,level(score,'优','中','差')) 后，返回的结果如图所示。



18.2.2 必选参数应用案例：给号码分段

本案例给“卡号表”工作表 B 列的卡号做分段处理，每 4 位分一段，如果不够 4 位，也要分成一段，如图所示。



处理前

	A	B	C
1	姓名	卡号	卡号分段
2	常威	595492888094924	
3	雷豹	464679984106816	
4	小林	755402768711247	
5	来福	548413737257575	
6	小周	607968439327173	
7	小曾	114768534364829	
8	周州	119663999124026	
9	黄风	367008500245562	
10	小梁	859462232201421	

卡号表

处理后

	A	B	C
1	姓名	卡号	卡号分段
2	常威	595492888094924	5954-9288-8094-924
3	雷豹	464679984106816	4646-7998-4106-816
4	小林	755402768711247	7554-0276-8711-247
5	来福	548413737257575	5484-1373-7257-575
6	小周	607968439327173	6079-6843-9327-173
7	小曾	114768534364829	1147-6853-4364-829
8	周州	119663999124026	1196-6399-9124-026
9	黄风	367008500245562	3670-0850-0245-562
10	小梁	859462232201421	8594-6223-2201-421

卡号表

本案例的编程思路是定义一个分段截取函数，可以让用户指定每段长度，也可以指定分段之间的分隔符，然后将B列中的卡号放到自定义函数中运行，最后将处理好的结果写入C列单元格。

```
# 自定义函数
def intercept(s,num,delimiter):
    s1 = str(s) # 将要分段的对象转换为字符串类型。
    lst = [s1[n:n+num] for n in range(0,len(s1),num)] # 对数据进行分段处理。
    s2 = delimiter.join(lst) # 合并分段的列表。
    return s2 # 将合并结果返回函数。

# 自定义函数应用
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb = xlrd.open_workbook('./29-functions-1.xls');ws=wb.sheet_by_name('卡号表')
# 读取工作簿和工作表。
nwb = copy(wb);nws=nwb.get_sheet('卡号表') # 复制工作簿，以及读取副本工作簿下的工作表。
for row_num in range(1,ws.nrows): # 循环行号。
    val = ws.cell_value(row_num,1) # 读取B列单元格值。
    nws.write(row_num,2,intercept(val,4,'-')) # 截取指定长度的字符进行分段，再写入单元格。
nwb.save('./29-functions-2.xls') # 保存副本工作簿。
```

- 第 2～6 行代码自定义切片函数 intercept(s,num,delimiter)，该函数的参数说明如下。
  - s: 必选参数，指定要分的段数。
  - num: 必选参数，表示每段的长度。
  - delimiter: 必选参数，表示每段之间的分隔符。
- 第 13 行代码 nws.write(row_num,2,intercept(val,4,'-'))，其中 intercept(val,4,'-') 部分使用自定义的intercept 函数，看一下此函数的实参，val 表示 B 列的每个卡号，每 4 个字符为一段，将"-"作为分隔符进行分段，最后将处理好的结果写入 C 列对应的单元格。

18.3 可选参数的写法及应用

可选参数表示在调用函数时可传可不传该参数的值。

- 在定义可选参数时，要设置一个值，这个值叫作**默认值**，所以可选参数也叫作**默认参数**。
- 在调用函数时，如果可选参数**没有传入值**，则使用可选参数的**默认值**。

在自定义函数时，既有必选参数，又有可选参数，要先定义必选参数，再定义可选参数。在调用函数时也一样，要先传必选参数，再传可选参数。

18.3.1 可选参数（默认参数）

比如，现在要自定义一个类似于 Excel 中的 MID 函数的函数，在对序列对象做切片时，可以指定位置和截取长度。案例代码如下所示。

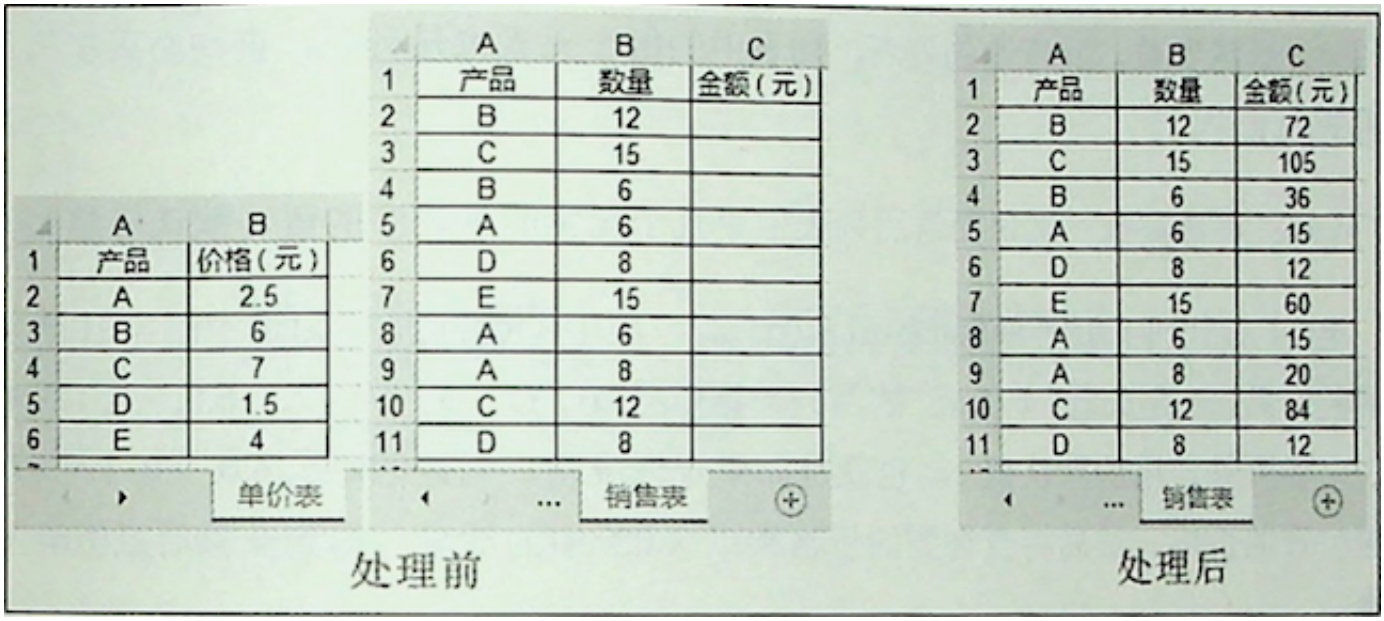
```
1  # 自定义函数。
2  def mid(iterable,start = 0,length = 1):
3      return iterable[start:start+length]
4  # 自定义函数调用。
5  print(mid('abcdefgh'))
6  print(mid('abcdefgh',2))
7  print(mid('abcdefgh',2,4))
```

- 第 2～3 行代码自定义函数 mid(iterable，start＝0，length＝1)，该函数的参数说明如下。
  - iterable:必选参数，提供可以被切片的对象，比如字符串、列表、元组。
  - start:可选参数，切片的起始位置，默认值是0，也就是从开始的位置切片。
  - length:可选参数，切片的长度，默认值是1。
- 第 5～7 行代码调用mid函数做测试。
  - 第5行代码 print(mid('abcdefgh'))，mid函数只提供了第1个参数，第2个参数和第3个参数都使用默认值，等价于print(mid('abcdefgh',0,1))，返回结果为 'a'。
  - 第6行代码 print(mid('abcdefgh',2))，mid函数提供了第1个参数和第2个参数，第3个参数使用默认值，等价于print(mid('abcdefgh',2,1))，返回结果为 'c'。
  - 第7行代码 print(mid('abcdefgh',2,4))，mid函数提供了全部参数，返回结果为 'cdef'。

18.3.2 可选参数应用案例：模拟 vlookup 函数的应用

本案例根据“销售表”工作表中 A 列的产品名称，到“单价表”工作表中查询对应的产品价格，再用查询到的产品价格乘以产品数量得到产品金额，将产品金额写入“销售表”工作表的 C 列单元格，如图所示。

本案例的编程思路是自定义一个类似于 Excel 工作表函数 vlookup 的函数。



	A	B
1	产品	价格(元)
2	A	2.5
3	B	6
4	C	7
5	D	1.5
6	E	4

单价表

	A	B	C
1	产品	数量	金额(元)
2	B	12	
3	C	15	
4	B	6	
5	A	6	
6	D	8	
7	E	15	
8	A	6	
9	A	8	
10	C	12	
11	D	8	

销售表

处理前

	A	B	C
1	产品	数量	金额(元)
2	B	12	72
3	C	15	105
4	B	6	36
5	A	6	15
6	D	8	12
7	E	15	60
8	A	6	15
9	A	8	20
10	C	12	84
11	D	8	12

销售表

处理后

```python
# 自定义vlookup函数
def vlookup(val,lst,num=1):
    r = lst[0].index(val) # 对lst列表第1个元素使用index函数进行查找。得到返回值赋值给r变量。
    v = lst[num][r] # 再使用num和r变量确定截取位置。
    return v # 将截取的值返回给函数
# 自定义函数应用
import xlrd # 导入读取xls文件的库。
from xlutils.copy import copy # 导入复制工作簿函数。
wb = xlrd.open_workbook('./30-functions-1.xls') # 读取工作簿。
ws_price = wb.sheet_by_name('单价表') # 读取工作簿下的工作表。
ws_Sale = wb.sheet_by_name('销售表') # 读取工作簿下的工作表。
nwb = copy(wb);nws=nwb.get_sheet('销售表') # 复制工作簿，以及读取副本工作簿下的工作表。
lst = [ws_price.col_values(0)[1:],ws_price.col_values(1)[1:]] # 读取'单价表'下的产品列和单价列。
for row_num in range(1,ws_Sale.nrows): # 循环'销售表'中的行号。
    val1 = ws_Sale.cell_value(row_num,0) # 读取'销售表'A列的品名。
    val2 = ws_Sale.cell_value(row_num,1) # 读取'销售表'B列的数量。
    val3 = vlookup(val1,lst)*val2 # 将val1中的品名去lst列表查找，返回对应的单价，再乘以val2的数量，得到金额。
    nws.write(row_num,2,val3) # 将计算出来的金额写入'销售表'的C列单元格。
nwb.save('./30-functions-2.xls') # 保存副本工作簿。
```

- 第 2～5 行代码自定义查询函数 vlookup(val,lst,num=1)，该函数的参数说明如下。
  - val:必选参数,要查询的值。
  - lst:必选参数,要查询的列表,列表中的每个元素也是列表,查询必须在第0个元素的列表中。
  - num:可选参数,确认要返回列表中第几个元素的列表中的值,默认值是1。
- 第 17 行代码 val3＝vlookup(val1,lst)＊val2,
  - 其中 vlookup(vall,lst)中的 val1 表示要查询的产品,

- lst 是放置“单价表”的两列数据 [['A','B','C','D','E'],[2.5,6.0,7.0,1.5,4.0]],
- 第 3 个参数是使用的默认值1,也就是如果查询成功,则返回［2.5,6.0,7.0,1.5,4.0］中对应位置的价格。
- 最后将查询到的价格乘以 val2 中的数量,得到金额后赋值给 val3 变量。

18.4 关键字参数的写法及应用

在函数调用时，通过“参数＝值”的形式传入实参的方式叫作关键字参数，可以让函数更加清晰、易于使用，同时参数的书写可以不按顺序。

18.4.1 关键字参数

关键字参数分为普通关键字参数和命名关键字参数。普通关键字参数可以用“参数＝值”的方式传入实参，也可以按参数位置的先后顺序直接传入实参。命名关键字参数必须用“参数＝值”的方式传入实参。

下面看看普通关键字参数与命名关键字参数的不同写法与传参方式，代码如下所 示。

```
# 自定义函数
def counter1(iterable,min,max):# 普通关键字参数。
    lst=[v for v in iterable if v>=min and v<=max]
    return len(lst)
def counter2(iterable,*,min,max):# 命名关键字参数。
    lst=[v for v in iterable if v>=min and v<=max]
    return len(lst)
# 调用自定义函数
print(counter1([2,3,8,3,4,5,9],3,8)) # 按位置传递实参。
print(counter1(max = 8,min = 3,iterable = [2,3,8,3,4,5,9])) # 按关键字参数传递实参。
print(counter1([2,3,8,3,4,5,9],3,max = 8)) # 部分按位置传递实参，部分按关键字参数传递实参。
print(counter2([2,3,8,3,4,5,9],3,8)) # 按位置传递实参会出错。
print(counter2(max = 8,min = 3,iterable = [2,3,8,3,4,5,9])) # 按关键字参数传递实参。
print(counter2([2,3,8,3,4,5,9],max = 8,min = 3)) # 部分按位置传递实参，部分按关键字参数传递实参。
```

- 第 2～7 行代码自定义条件计数函数 counter1(iterable,min,max)和counter2(iterable,＊,min,max),它们的参数都相同,参数说明如下。
  - iterable:必选参数,提供要判断的序列数字。
  - min:必选参数,定义范围最小值。
  - max:必选参数,定义范围最大值。

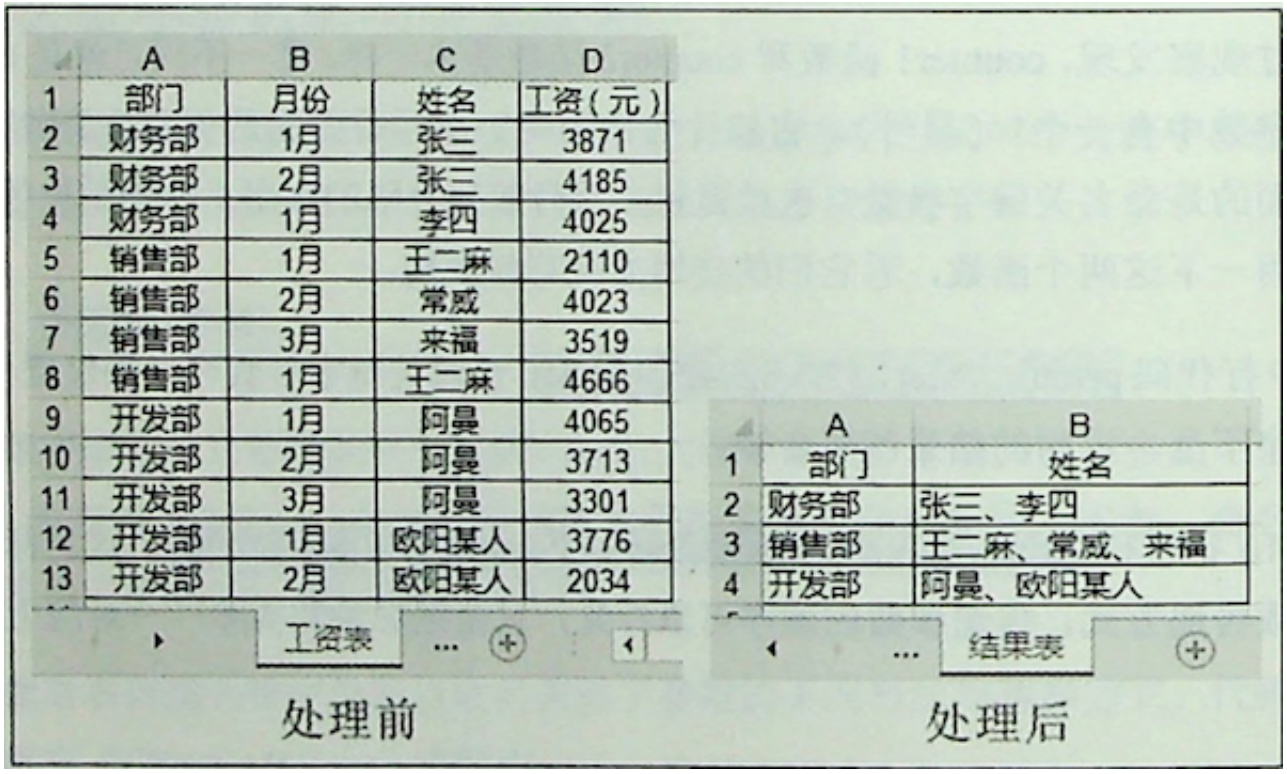
通过观察发现,counter1 函数和 counter2 函数基本一样,唯一不同点就是 counter2 函数的参数

中有一个 * (星号),它起什么作用呢?
星号前面的是普通的关键字参数,星号后面的是命名关键字参数。也就是说,星号起到分隔的作用,便于用户区分。接下来调用一下这两个函数,看它们的使用方法有何区别。

- 第 9 行代码 print(counter1([2,3,8,3,4,5,9],3,8)),普通关键字参数可以按位置传参数,只要顺序不乱,返回的结果就不会错。
- 第 10 行代码 print(counter1(max＝8,min＝3,iterable＝[2,3,8,3,4,5,9]),这是标准的关键参数传递方式,传递参数的顺序可以打乱,只要把形参和实参一一对应上就可以了。
- 第 11 行代码 print(counter1([2,3,8,3,4,5,9],3,max＝8)),这是位置参数与关键字参数混合传递方式,同时包含这两种形式,一定是先位置参数,后关键字参数。前两个参数是按照位置传递参数的,第3个参数是使用关键字参数传递参数的。
- 第 12 行代码 print(counter2([2,3,8,3,4,5,9],3,8)),因为 counter2 函数中有命名关键字参数,所以**这种位置传参方式会出错**。min 和 max 两个参数必须按“参数＝值”的方式来书写,前面的iterable参数可以不按“参数＝值”的方式来书写。
- 第 13 行代码 print(counter2(max＝8,min＝3,iterable＝2,3,8,3,4,5,9])),counter2 函数的所有参数都按“参数＝值”的方式来书写,这种方式也能正常运行。
- 第14行代码 print(counter2([2,3,8,3,4,5,9],max＝8,min＝3)),counter2 函数按混合传参方式来书写也是可以的,注意位置传参方式要写在最前面。

18.4.2 关键字参数应用案例：分类合并字符串

本案例以“工资表”工作表的“部门”列为基准，合并“姓名”列，如图所示。本案例的编程思路是创建一个字典，将指定列的数据作为键，将指定的另一列的数据作为值，循环装入字典中，再对字典中每个键对应的值去重，然后根据指定的分隔符合并，最后将结果返回给自定义的函数即可，也就是把整个数据处理放在自定义函数中。



处理前

	A	B	C	D
1	部门	月份	姓名	工资(元)
2	财务部	1月	张三	3871
3	财务部	2月	张三	4185
4	财务部	1月	李四	4025
5	销售部	1月	王二麻	2110
6	销售部	2月	常威	4023
7	销售部	3月	来福	3519
8	销售部	1月	王二麻	4666
9	开发部	1月	阿曼	4065
10	开发部	2月	阿曼	3713
11	开发部	3月	阿曼	3301
12	开发部	1月	欧阳某人	3776
13	开发部	2月	欧阳某人	2034

工资表

处理后

	A	B
1	部门	姓名
2	财务部	张三、李四
3	销售部	王二麻、常威、来福
4	开发部	阿曼、欧阳某人

结果表

```
# 自定义combine函数。
def combine(range,join_range,delimiter=' '):
    dic={} # 初始化dic变量为空字典。
    for x,y in zip(range,join_range): # 将range和join_range接收的值使用zip函数做组合转换，并赋值给 x,y 变量。
        if not x in dic: # 如果x在dic字典中不存在。
            dic[x]=[y] # 则以x为键，y为值，创建键值，注意y是写在列表中的。
        else: # 否则。
            dic[x].append(y) # 如果x键存在，则将y添加到对应的值列表中。
    l=[(k,delimiter.join([str(i) for i in dict.fromkeys(i)])) for k,i in dic.items()] # 将键对应值列表去重合并。
    return l # l变量是列表，列表中的元素是元组类型，元组中分别存储是dic字典的键，及dic字典对应值合并后的字符串。
# 自定义函数应用
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导计工作簿复制函数。
wb=xlrd.open_workbook('./31-functions-1.xls');ws=wb.sheet_by_name('工资表')
# 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('结果表') # 复制工作簿，以及读取副本工作簿下的工作表。
row_num=0 # 初始化row_num变量为。
for x,y in combine(delimiter='、',range=ws.col_values(0),join_range=ws.col_values(2)):
# 调用combine函数。
    nws.write(row_num,0,x) # 将range参数中的值去重后写入'结果表'工作表A列。
    nws.write(row_num,1,y) # 将join_range参数中的值去重合并再写入'结果表'工作表B列。
    row_num +=1 # 累加row_num变量。
nwb.save('./31-functions-2.xls') # 保存副本工作簿。
```

- 第 2～10 行代码,自定义合并函数 combine(range,join_range,delimiter=''),该函数的参数说明

如下。

  - range:必选参数,指定基准列。
  - join_range:必选参数,指定合并列。
  - delimiter:可选参数,对列进行合并时指定的分隔符,默认值是空格。

- 第 17 行代码 for x,y in combine(delimiter='、',range＝ws.col_values(0),join_range＝ws.col_values(2))：,调用了combine 函数。

  - delimiter 参数指定的分隔符是'、';
  - range 参数指定的基准列是ws.col_values(0),也就是“工资表”的A列;
  - join_range 参数指定的合并列是ws.col_values(2),也就是“工资表”的B列。
  - combine函数的3个参数顺序因为使用的是关键字参数方式,所以顺序可以打乱。
  - combine 函数的返回值是列表,列表中的元素是元组,元组中包含了部门和对应的姓名,结果为［('部门','姓名'),('财务部','张三、李四'),('销售部','王二麻、常威、来福'),('开发部','阿曼、欧阳某人')］。
  - 最后将列表中的值循环写入“结果表”工作表即可。

18.5 不定长参数的写法及应用

不定长参数是指定义的函数参数个数不确定，可以将不定长参数收纳在指定的容器中。

不定长参数分为以下两种。

- `*args`:以一个星号开头，然后接 arg 变量，这种参数接收的值存储在元组中，即 args 是一个元组类型。
- `**kwargs`:以两个星号开头，然后接 kwargs 变量，这种参数接收的值存储在字典中，即 kwargs 是一个字典类型。

args 与 kwargs 变量名称不是固定的，只是约定俗成的一种命名方式，用户可以修改成其他名称。不定长参数必须放在必选参数之后。

`*args` 和 `**kwargs` 接收的数据类型不限，完全根据需求而定。

18.5.1 不定长参数1

下面是一个小案例，代码如下所示。

```
# 自定义函数
def subtotal(iterable,*args):
    print(args) # 在屏幕打印args中的内容。
    return [fun(iterable) for fun in args] # 根据args接收的函数返回处理结果。
# 调用自定义函数
```

```
6 print(subtotal([7,8,9,10],max,min,sum))
```

- 第 2～4 行代码自定义汇总函数 subtotal(iterable,*args)，该函数的参数说明如下。
  - iterable:必选参数，提供要判断的序列数字。
  - *args:可选参数，指定汇总函数。
- 第 6 行代码 print(subtotal([7,8,9,10],max,min,sum))，
  - iterable变量的值是 [7,8,9,10] 列表，
  - 汇总函数是max、min、sum，这 3 个函数都将存储在args变量中。
  - 运行第 3 行代码 print(args)，屏幕上的输出结果为（<built-in function max>，<built-infunction min>，<built-in function sum>），可以看到元组的每个元素是函数对象。
  - 用每个函数去处理 [7,8,9,10] 即可，此行代码只是为了便于理解而写的，可以省略不写。
  - 最后调用 subtotal 函数，返回的结果为 [10,7,34]。

18.5.2 不定长参数2

下面是一个小案例，代码如下所示。

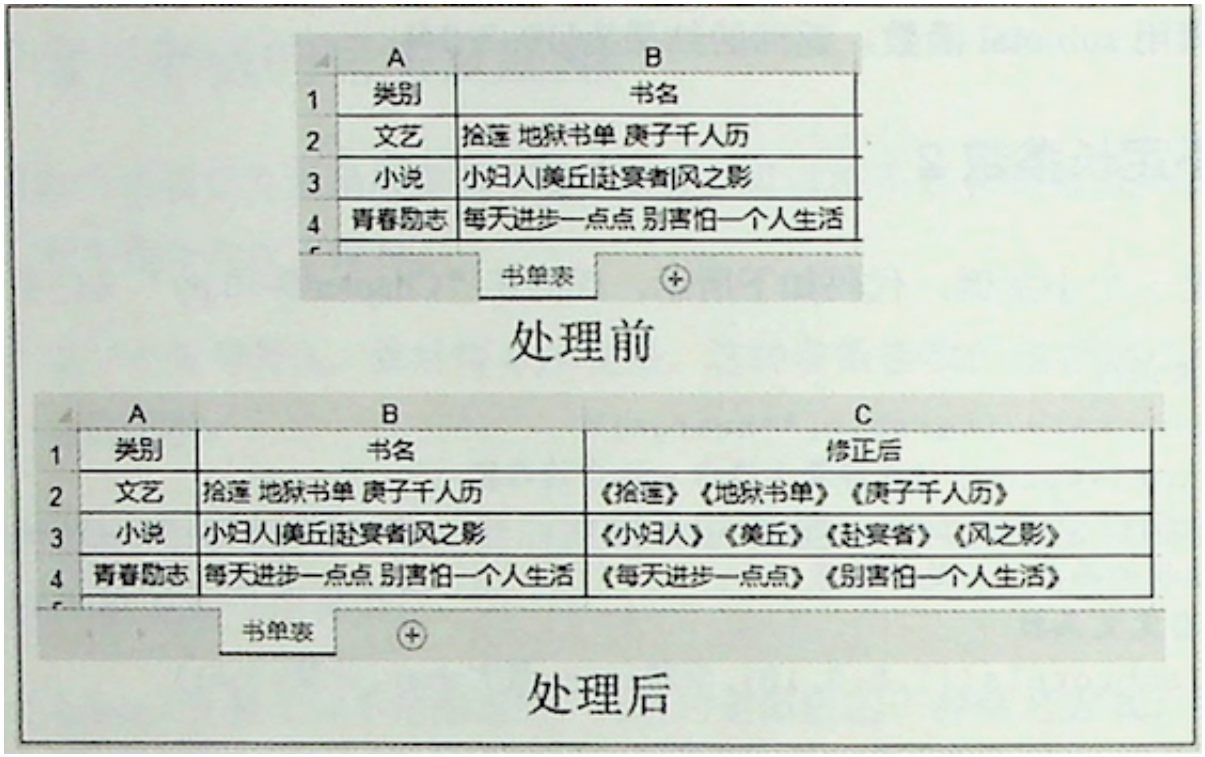
```
1 # 自定义函数
2 def subtotals(iterable,**kwargs):
3     print(kwargs) #在屏幕打印args中的内容。
4     return [(key,item(iterable)) for key,item in kwargs.items()] # 根据args
5 接收的函数返回处理结果。
6 # 调用自定义函数
  print(subtotals([7,8,9,10],求和=sum,最大=max,计数=len))
```

- 第2～4行代码自定义汇总函数 subtotals(iterable,**kwargs)。
  - iterable:必选参数,提供要判断的序列数字。
  - **kwargs:可选参数,按字典的键值方式指定汇总函数。
- 第6行代码 print(subtotals([7,8,9,10],求和＝sum,最大＝max,计数＝len)),
  - iterable变量的值是[7,8,9,10]列表,
  - 汇总函数的指定方式是求和＝sum,最大＝max,计数＝len,这3对键值都将存储在 kwargs 变量中。
- 运行第3行代码print(kwargs),屏幕上的输出结果为{ ' 求和 ' ： <built-in function sum>, ' 最大 ' ： <built-in function max>, ' 计数： <built-in function len>},可以看到字典中的键是汇总名称,值是对应的函数名称,用值的每个函数去处理 [7,8,9,10] 即可。此行代码是为了便于理解而写的,可以省略不写。

- 最后调用 subtotals 函数,返回的结果为[('求和',34),('最大',10),('计数',4)]。

18.5.3 不定长参数应用案例：替换函数增强版

在本案例中，“书单表”工作表的 B 列是书名，书名之间使用空格 (' ')和竖线 ('|') 进行分隔，现在要用规范的书名号来分隔，修改前后的效果如图所示。



本案例的编程思路是定义一个可以替换指定多个旧字符串为统一新字符串的函数，然后调用该函数处理 B 列的书名即可。

```
# 自定义replaces函数。
def replaces(string,new,*old):
    for o in old: # 循环old参数中的元素赋值给o变量。
        string = string.replace(o,new) # 将o变量中的值，替换为new变量中的值，最后再赋值给string参数。
    return string # 循环替换完成后，将string中的值返回给replaces函数。
# 自定义函数应用
import xlrd # 导入xls文件读取库。
from xlutils.copy import copy # 导计工作簿复制函数。
wb=xlrd.open_workbook('./32-books-1.xls');ws=wb.sheet_by_name('书单表') # 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('书单表') # 复制工作簿，以及读取副本工作簿下的工作表。
for row_num in range(1,ws.nrows): # 循环'书单表'的行号，赋值给row_num。
    val1=ws.cell_value(row_num,1) # 获取B列单元格的书名，并赋值给val变量。
    val2='《'+replaces(val1,'》《',' ','|')+'》' #将' '和'|'替换为'》、《', 再补齐两端书名号，赋值给 val2 变量。
    nws.write(row_num,2,val2) # 将处理好后的val2变量写入C列单元格。
nwb.save('./32-books-2.xls') # 保存副本工作簿。
```

- 第 2～5 行代码，自定义替换函数 replaces(string,new,*old)，参数说明如下。
  - string:必选参数，原字符串。
  - new:必选参数，指定要替换的新字符串。
  - *old:可选参数，指定替换的旧字符串，可以指定多个。
- 第13行代码 val2='《'+replaces(val1,'》《',',','|')+'》'，调用 replaces 函数，string 参数是 vall 变量，也就是 B 列单元格的书名。new 参数是“》《”，表示将要替换的新字符串，old参数获取的是（，T），表示将空格（ ' ' ）和竖线（ " ）都替换为“》《”。
- 最后将替换结果赋值给val2变量。

18.6 匿名函数的写法及应用

匿名函数指无须定义标识符的函数或子程序。

Python 中用 lambda 关键字定义匿名函数，只用表达式而无须声明。

使用匿名函数有一个好处，就是匿名函数没有名字，不必担心函数名冲突。此外，匿名函数也是一个函数对象，可以把匿名函数赋值给一个变量，这样变量名就相当于函数名，可以利用变量来调用该函数。

18.6.1 匿名函数的语法结构

在 Python 中，lambda 是一个关键字，用来引入表达式的语法。相比 def 函数，lambda 是单一的表达式，而不是语句块。

在 lambda 中只能封装有限的业务逻辑，lambda定义的匿名函数纯粹是为了编写简单函数而设计的，def 函数则专注于处理更大的业务。

匿名函数的语法结构如图所示。



下面看看匿名函数的定义和调用，代码如下所示。

```
print(lambda x:x*10) # 定义匿名函数。
print((lambda x:x*10)(20)) # 定义及调用匿名函数。
fun=lambda x:x*10 # 匿名函数赋值给变量。
print(fun(20)) # 将变量名当做函数名调。
```

18.6.2 匿名函数的常见书写方式

匿名函数与 def 定义的普通函数一样，可以有不同类型的参数，案例代码如下所示。

```
fun1=lambda x,y:x+y # 必选参数的匿名函数。
print(fun1(10,20)) # 调用fun1函数。
fun2=lambda x,y=20:x+y # 可选参数的匿名函数。
print(fun2(10)) # 调用fun2函数。
fun3=lambda l,*fun:[f(l) for f in fun] # 不定长参数的匿名函数1。
print(fun3([3,4,5],sum,max,min)) # 调用fun3函数。
fun4=lambda l,**fun:[k+':'+str(i(l)) for k,i in fun.items()] # 不定长参数的匿名函数2。
print(fun4([3,4,5],求和=sum,最大=max,最小=min)) # 调用fun4函数。
```

- 第 1 行代码 fun1 = lambda x,y:x＋y，必选参数是 x 和 y，处理表达式是将两个参数相加，再将匿名函数赋值给 fun1变量。
- 第 2 行代码 print(funl(10,20))，给 fun1变量传入参数并输出，结果为 30。
- 第 3 行代码 fun2=lambda x,y＝20:x＋y，x 是必选参数，y 是可选参数，默认值是 20，处理表达式是将两个参数相加，再将匿名函数赋值给fun2变量。
- 第 4 行代码 print(fun2(10))，x 参数接收的是 10，而y参数使用默认值 20，最后 fun2 的输出结果为 20.
- 第 5 行代码 fun3＝lambda l,*fun:[f(l) for f in fun],l 是必选参数,*fun 是不定长参数,用元组存储数据,处理表达式是用 fun 元组接收到的函数去处理 l 参数中的值,再将匿名函数赋值给 fun3 变量。
- 第 6 行代码 print(fun3([3,4,5],sum,max,min)),l 参数接收的是 [3,4,5],而 fun 参数接收的是(＜built-in function sum＞,＜built-in function max＞,＜built-in function min＞),最后 fun3 的输出结果为 [12,5,3]。
- 第 7 行代码 fun4＝lambda l,**fun:[k＋ ' : ' ＋str(i(l))for k,i in fun.items()],l 是必选参数,**fun 是不定长参数,用字典存储数据。处理表达式是用 fun 字典接收到的函数去处理1参数中的值,再将匿名函数赋值给 fun4 变量。
- 第 8 行代码 print(fun4([3,4,5],求和＝sum,最大＝max,最小＝min)),l 参数接收的是 [3,4,5],而 fun参数接收的是 ｛ ' 求和： ＜built-in function sum＞, ' 最大： ＜built-in function max＞, ' 最小 ' ： ＜built-in function min＞｝ ,最后fun4的输出结果为[' 求和： 12, ' 最大： 5 ' , ' 最小： 3 ']。

18.6.3 匿名函数应用案例：根据身份证号判断性别

本案例根据“身份证表”工作表A列单元格的身份证号第 17 位数字判断性别，奇数为男，偶数为女，然

后将结果写入 B 列单元格，如图所示。



	A	B
1	身份证号	性别
2	110101********1689	
3	110101********3095	
4	110101********5234	
5	110101********5939	
6	110101********4005	
7	220102********9130	
8	220102********5893	
9	220102********4751	
10	220102********7952	
11	220102********0421	
12		

身份证表

处理前

	A	B
1	身份证号	性别
2	110101********1689	女
3	110101********3095	男
4	110101********5234	男
5	110101********5939	男
6	110101********4005	女
7	220102********9130	男
8	220102********5893	男
9	220102********4751	男
10	220102********7952	男
11	220102********0421	女
12		

身份证表

处理后

本案例的编程思路是使用匿名函数定义一个判断性别的函数，然后调用即可。

```
import xlrd # 导入读取xls文件的库。
from xlutils.copy import copy # 导入工作簿复制函数。
wb=xlrd.open_workbook('./33-ids-1.xls');ws=wb.sheet_by_name('身份证表') # 读取工作簿和工作表。
nwb=copy(wb);nws=nwb.get_sheet('身份证表') # 复制工作簿，并且读取副本工作簿中的工作表。
for row_num in range(1,ws.nrows): # 循环'身份证表'工作表的行号。
    card=lambda id:'男' if int(id[-2])%2==1 else '女' # 自定义根据身份证号判断性别的匿名函数。
    val=ws.cell_value(row_num, 0) # 获取A列的身份证号，并赋值给val变量。
    nws.write(row_num,1,card(val)) # 将val做为匿名函数card的参数，将判断出的性别写入B列单元格。
nwb.save('./33-ids-2.xls') # 保存副本工作簿。
```

18.7 自定义函数存放在 .py 文件中

自定义函数可以被单独存放在一个 **.py** 文件中，这样其他 **.py** 文件也可以调用，实现自定义函数的复用。

下面介绍如何调用指定 **.py** 文件中的自定义函数。我们将之前学习的一部分自定义函数放在 **fun.py** 文件中，然后在另外的 **.py** 文件中调用，代码如下所示。

```
# 平均函数
def average(lst):
    num=sum(lst)/len(lst) # 平均处理。
    avg=float('{:.2f}'.format(num)) # 格式化平均值。
    return avg # 返回平均值。

```

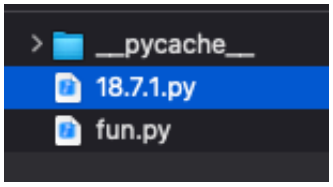
```
# 分段函数
def intercept(s,num,delimiter):
    s1=str(s) # 将要分段的对象转换为字符串类型。
    lst=[s1[n:n+num] for n in range(0,len(s1),num)] # 对数据进行分段处理。
    s2=delimiter.join(lst) # 合并分段的列表。
    return s2 # 将合并结果返回函数。

# 模仿vlookup
def vlookup(val,lst,num=1):
    r=lst[0].index(val) # 对lst列表第1个元素使用index函数进行查找。得到返回值赋值给r变量。
    v=lst[num][r] # 再使用num和r变量确定截取位置。
    return v # 将截取的值返回给函数

# 分类合并字符串
def combine(range,join_range,delimiter=' '):
    dic={} # 初始化dic变量为空字典。
    for x,y in zip(range,join_range): # 将range和join_range接收的值使用zip函数做组合转换，并赋值给x,y变量。
        if not x in dic: # 如果x在dic字典中不存在。
            dic[x]=[y] # 则以x为键，y为值，创建键值，注意y是写在列表中的。
        else: # 否则。
            dic[x].append(y) # 如果x键存在，则将y添加到对应的值列表中。
    l=[(k,delimiter.join([str(i) for i in dict.fromkeys(i)])) for k,i in dic.items()] # 将键对应值列表去重合并。
    return l # l变量是列表，列表中的元素是元组类型，元组中分别存储是dic字典的键，及dic字典对应值合并后的字符串。
```

18.7.1 函数定义在单独 .py 文件中

18-7-1.py 文件与 **fun.py** 文件存放在同一个文件夹中，如图所示。



fun.py 文件中存放的是自定义函数，要求在 **18-7-1.py** 文件中调用 **fun.py** 文件中的函数。

顺便说一句，在图中还有一个“**pycache**”文件夹，是在导入 **fun.py** 文件时产生的文件夹，目的是让程序启动更快一点，当用户的脚本更改时，它将被重新编译，如果删除文件或删除整个文件夹并再次运行程序，它将重新出现，一般忽略即可。

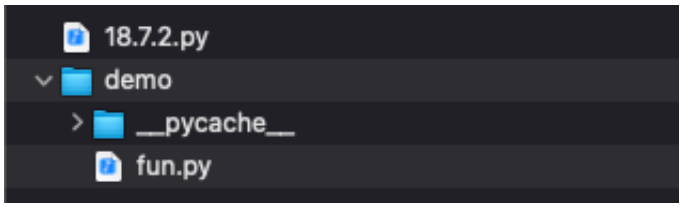
```
import fun #导入函数文件名。
print(fun.average([1,2,3,4,5,6])) #调用fun文件下的average函数。
print(fun.intercept('5120124575',4,'-')) #调用fun文件下的intercept函数。

from fun import * #导入函数文件名下的所有函数。
print(intercept('4523465745',4,'、')) #直接调用函数
```

```
7
8  from fun import average #导入函数文件名下指定的函数。
9  print(average([4,2,7])) #直接调用函数
```

- 第 1 行代码 import fun，直接导入函数文件名，注意不要写扩展名 `.py` 。
- 第 2 行代码和第3行代码调用函数，fun.average 表示调用 fun 文件中的 average 函数。同理，fun.Intercept表示调用 fun 文件中的 intercept 函数。
- 第 5 行代码 from fun import * ，表示导入 fun 件中的所有函数。
- 第 6 行代码直接调用 intercept 函数。
- 第 8 行代码 from fun import average，表示只导入 fun 文件中的 average 函数。
- 第 9 行代码就可以调用 average 函数了，但也只能调用 average 函数，因为只导入了 average 函数。

18.7.2 函数定义在文件夹中



调用不同文件夹中的函数的案例代码如下。

```
1  from demo.fun import * # 导入文件夹下指定的fun.py文件下的所有函数。
2  print(average([1,2,3,4,5,6])) # 直接调用函数
3  print(intercept('5120124575',4,'-')) # 直接调用函数
4
5  from demo.fun import average # 导入文件夹下指定的fun.py文件下的所有函数。
6  print(average([43,52,12])) # 直接调用函数
```

- 第 1 行代码 from demo.fun import *，其中 demo.fun 表示 demo 文件夹中的 fun.py 文件，导入该文件中的所有函数。
- 第 2 行代码和第3行代码直接调用 fun.py 文件中的 average 函数和 intercept 函数。
- 第 5 行代码 from demo.fun import average，表示导入demo 文件夹中 fun.py 文件的 average 函数。
- 第 6 行代码直接调用average函数，但也只能调用average函数，因为只导入了该函数。

在 Python 中，**包**和**库**是两个常见的概念，它们经常用来组织和管理代码。以下是它们的定义和操作方式：

---

19. Python 的包和库

19.1 Python 库的定义

- 包是一个 **Python 模块的集合**，用来组织 Python 模块（文件）结构化。
- 包的本质是一个带有 `__init__.py` 文件的目录。
- 通过包，你可以创建模块的层次结构，用于更清晰地组织和复用代码。

19.2 库的示例结构

```
my_package/
    __init__.py
    module1.py
    module2.py
    subpackage/
        __init__.py
        module3.py
```

- `__init__.py` 文件：
  - 该文件可以为空，但它标志着该目录是一个包。
  - 可以在其中定义包的初始化逻辑（例如导入模块、定义包的变量等）。

19.3 Python 中的库定义

- 库是一个更宽泛的概念，指一组有用的功能或工具的集合，通常由一个或多个模块或包组成。
- 在 Python 中，库可以是标准库（内置库，如 `os`、`math`）、第三方库（如 `NumPy`、`Pandas`）或自定义库。

19.4 包与库的关系

- **包**是用来组织代码的工具，是库的一部分。
- 一个库可以由多个包组成。

19.5 安装第三方库

- 使用 `pip` 安装库。

```
pip install 库名
```

例如，安装 `requests`：

```
pip install requests
```

- 查看已安装的库：

```
pip list
```

- 升级库：

```
pip install --upgrade 库名
```

19.6 导入包或库

- 导入整个模块：

```
import os  # 导入标准库 os
import my_package.module1  # 导入自定义包的模块
```

- 使用别名简化调用：

```
import numpy as np
import pandas as pd
```

- 导入特定功能：

```
from math import sqrt  # 从 math 库导入 sqrt 函数
from my_package.module1 import my_function  # 从模块中导入函数
```

19.7 创建自定义包

1. 创建包的目录结构：

```
my_package/
    __init__.py
    module1.py
    module2.py
```

2. 编写代码：

- `module1.py`：

```
def hello():
    return "Hello from module1"
```

- `module2.py`：

```
def add(a, b):
    return a + b
```

3. 使用自定义包：
   - 添加到系统路径：

```
import sys
sys.path.append('/path/to/my_package')
```

   - 导入模块并使用：

```
from my_package.module1 import hello
print(hello())  # 输出: Hello from module1
```

19.8 安装本地库（开发阶段使用）

- 使用 `setup.py` 文件定义库，并安装：

```
my_project/
    my_package/
        __init__.py
        module1.py
    setup.py
```

- `setup.py` 示例：

```
from setuptools import setup, find_packages

setup(
    name="my_package",
    version="0.1",
    packages=find_packages(),
)
```

- 安装到本地开发环境：

```
pip install -e .
```

19.9 常见库操作

- **查找库的路径**：

```
import numpy
print(numpy.__file__)
```

- **卸载库**：

```
pip uninstall 库名
```

- **查看库文档**：

```
help(模块名)
```

总结来说，**包是代码的组织单位**，**库是功能的集合**，它们在 Python 中相辅相成。通过合理地组织包和使用库，你可以构建更清晰、复用性更高的项目。

阿里云天池 Python 训练营 - Part4

20. 面向对象编程基础

活在当下的程序员应该都听过 " 面向对象编程 " 一词，也经常有人问能不能用一句话解释下什么是 " 面向对象编程 "，我们先来看看比较正式的说法。

" 把一组数据结构和处理它们的方法组成对象（object），把相同行为的对象归纳为类（class），通过类的封装（encapsulation）隐藏内部细节，通过继承（inheritance）实现类的特化（specialization）和泛化（generalization），通过多态（polymorphism）实现基于对象类型的动态分派。"

> ** 说明：** 以上的内容来自于网络，不代表作者本人的观点和看法，与作者本人立场无关，相

> 关责任不由作者承担。

之前我们说过 "** 程序是指令的集合 **"，我们在程序中书写的语句在执行时会变成一条或多条指令然后由 CPU 去执行。当然为了简化程序的设计，我们引入了函数的概念，把相对独立且经常重复使用的代码放置到函数中，在需要使用这些功能的时候只要调用函数即可；如果一个函数的功能过于复杂和臃肿，我们又可以进一步将函数继续切分为子函数来降低系统的复杂性。但是说了这么多，不知道大家是否发现，所谓编程就是程序员按照计算机的工作方式控制计算机完成各种任务。但是，计算机的工作方式与正常人类的思维模式是不同的，如果编程就必须得抛弃人类正常的思维方式去迎合计算机，编程的乐趣就少了很多，" 每个人都应该学习编程 " 这样的豪言壮语就只能说说而已。当然，这些还不是最重要的，最重要的是当我们需要开发一个复杂的系统时，代码的复杂性会让开发和维护工作都变得举步维艰，所以在上世纪 60 年代末期，"软件危机"、"软件工程" 等一系列的概念开始在行业中出现。

当然，程序员圈子内的人都知道，现实中并没有解决上面所说的这些问题的 "银弹"，真正让软件开发者看到希望的是上世纪 70 年代诞生的 Smalltalk 编程语言中引入的面向对象的编程思想（面向对象编程的雏形可以追溯到更早期的 Simula 语言）。按照这种编程理念，程序中的数据和操作数据的函数是一个逻辑上的整体，我们称之为“对象”，而我们解决问题的方式就是创建出需要的对象并向对象发出各种各样的消息，多个对象的协同工作最终可以让我们构造出复杂的系统来解决现实中的问题。

> ** 说明：** 当然面向对象也不是解决软件开发中所有问题的最后的“银弹”，所以今天的高级程序设计语言几乎都提供了对多种编程范式的支持，Python 也不例外。

20.1 类和对象

简单的说，类是对象的蓝图和模板，而对象是类的实例。这个解释虽然有点像用概念在解释概念，但是从这句话我们至少可以看出，类是抽象的概念，而对象是具体的东西。在面向对象编程的世界中，一切皆为对象，对象都有属性和行为，每个对象都是独一无二的，而且对象一定属于某个类（型）。当我们把一大堆拥有共同特征的对象的静态特征（属性）和动态特征（行为）都抽取出来后，就可以定义出一个叫做“类”的东西。

20.2 定义类

在 Python 中可以使用 `class` 关键字定义类，然后在类中通过之前学习过的函数来定义方法，这样就可以将对象的动态特征描述出来，代码如下所示。

```
class Student(object):

    # __init__ 是一个特殊方法用于在创建对象时进行初始化操作
    # 通过这个方法我们可以为学生对象绑定 name 和 age 两个属性
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def study(self, course_name):
```

```
        print('%s 正在学习 %s.' % (self.name, course_name))

    # PEP 8 要求标识符的名字用全小写多个单词用下划线连接
    # 但是部分程序员和公司更倾向于使用驼峰命名法(驼峰标识)
    def watch_movie(self):
        if self.age < 18:
            print('%s 只能观看《熊出没》.' % self.name)
        else:
            print('%s 正在观看岛国爱情大电影.' % self.name)
```

> ** 说明：** 写在类中的函数，我们通常称之为（对象的）方法，这些方法就是对象可以接收的消息。

20.3 创建和使用对象

当我们定义好一个类之后，可以通过下面的方式来创建对象并给对象发消息。

```
def main():
    # 创建学生对象并指定姓名和年龄
    stu1 = Student('骆昊', 38)
    # 给对象发 study 消息
    stu1.study('Python 程序设计')
    # 给对象发 watch_av 消息
    stu1.watch_movie()stu2 = Student('王大锤', 15)
    stu2.study('思想品德')
    stu2.watch_movie()


if __name__ == '__main__':
    main()
```

20.4 访问可见性问题

对于上面的代码，有 C++、Java、C# 等编程经验的程序员可能会问，我们给 `Student` 对象绑定的 `name` 和 `age` 属性到底具有怎样的访问权限（也称为可见性）。因为在很多面向对象编程语言中，我们通常会将对象的属性设置为私有的（private）或受保护的（protected），简单的说就是不允许外界访问，而对象的方法通常都是公开的（public），因为公开的方法就是对象能够接受的消息。在 Python 中，属性和方法的访问权限只有两种，也就是公开的和私有的，如果希望属性是私有的，在给属性命名时可以用两个下划线作为开头，下面的代码可以验证这一点。

```
class Test:

    def __init__(self, foo):
        self.__foo = foo

    def __bar(self):
        print(self.__foo)
```

```
        print('__bar')


def main():
    test = Test('hello')
    # AttributeError: 'Test' object has no attribute '__bar'
    test.__bar()
    # AttributeError: 'Test' object has no attribute '__foo'
    print(test.__foo)


if __name__ == "__main__":
    main()
```

但是，Python 并没有从语法上严格保证私有属性或方法的私密性，它只是给私有的属性和方法换了一个名字来妨碍对它们的访问，事实上如果你知道更换名字的规则仍然可以访问到它们，下面的代码就可以验证这一点。之所以这样设定，可以用这样一句名言加以解释，就是 "**We are all consenting adults here**"。因为绝大多数程序员都认为开放比封闭要好，而且程序员要自己为自己的行为负责。

```
class Test:

    def __init__(self, foo):
        self.__foo = foo

    def __bar(self):
        print(self.__foo)
        print('__bar')


def main():
    test = Test('hello')
    test._Test__bar()print(test._Test__foo)


if __name__ == "__main__":
    main()
```

在实际开发中，我们并不建议将属性设置为私有的，因为这会导致子类无法访问（后面会讲到）。所以大多数 Python 程序员会遵循一种命名惯例就是让属性名以单下划线开头来表示属性是受保护的，本类之外的代码在访问这样的属性时应该要保持慎重。这种做法并不是语法上的规则，单下划线开头的属性和方法外界仍然是可以访问的，所以更多的时候它是一种暗示或隐喻。

20.5 面向对象的支柱

面向对象有三大支柱：
封装、继承和多态。
后面两个概念在下一个章节中进行详细的说明，这里我们先说一下什么是封装。

我自己对封装的理解是 " 隐藏一切可以隐藏的实现细节，只向外界暴露（提供）简单的编程接口 "。我们在类中定义的方法其实就是把数据和对数据的操作封装起来了，在我们创建了对象之后，只需要给对象发送一个消息（调用方法）就可以执行方法中的代码，也就是说我们只需要知道方法的名字和传入的参数（方法的外部视图），而不需要知道方法内部的实现细节（方法的内部视图）。

20.6 练习

练习 1：定义一个类描述数字时钟。

参考答案：

```
from time import sleep


class Clock(object):
    """ 数字时钟 """

    def __init__(self, hour=0, minute=0, second=0):
        """ 初始化方法

        :param hour: 时
        :param minute: 分
        :param second: 秒
        """
        self._hour = hour
        self._minute = minute
        self._second = second

    def run(self):
        """ 走字 """
        self._second += 1
        if self._second == 60:
            self._second = 0
            self._minute += 1
            if self._minute == 60:
                self._minute = 0
                self._hour += 1
                if self._hour == 24:
                    self._hour = 0

    def show(self):
        """ 显示时间 """
        return '%02d:%02d:%02d' % \
               (self._hour, self._minute, self._second)


def main():
    clock = Clock(23, 59, 58)
    while True:
```

```
        print(clock.show())
        sleep(1)
        clock.run()


if __name__ == '__main__':
    main()
```

练习 2：定义一个类描述平面上的点并提供移动点和计算到另一个点距离的方法。

参考答案：

```
from math import sqrt


class Point(object):

    def __init__(self, x=0, y=0):
        """ 初始化方法

        :param x: 横坐标
        :param y: 纵坐标
        """
        self.x = x
        self.y = y

    def move_to(self, x, y):
        """ 移动到指定位置

        :param x: 新的横坐标
        "param y: 新的纵坐标"""
        self.x = x
        self.y = y

    def move_by(self, dx, dy):
        """ 移动指定的增量

        :param dx: 横坐标的增量
        "param dy: 纵坐标的增量"""
        self.x += dx
        self.y += dy

    def distance_to(self, other):
        """ 计算与另一个点的距离

        :param other: 另一个点
        """
        dx = self.x - other.x
        dy = self.y - other.y
        return sqrt(dx ** 2 + dy ** 2)

```

```
    def __str__(self):
        return '(%s, %s)' % (str(self.x), str(self.y))


def main():
    p1 = Point(3, 5)
    p2 = Point()print(p1)
    print(p2)
    p2.move_by(-1, 2)
    print(p2)
    print(p1.distance_to(p2))


if __name__ == '__main__':
    main()
```

> ** 说明：** 本章中的插图来自于 Grady Booch 等著作的《面向对象分析与设计》一书，该书是讲解面向对象编程的经典著作，有兴趣的读者可以购买和阅读这本书来了解更多的面向对象的相关知识。

21. 面向对象进阶

在前面的章节我们已经了解了面向对象的入门知识，知道了如何定义类，如何创建对象以及如何给对象发消息。为了能够更好的使用面向对象编程思想进行程序开发，我们还需要对Python中的面向对象编程进行更为深入的了解。

21.1 @property装饰器

之前我们讨论过Python中属性和方法访问权限的问题，虽然我们不建议将属性设置为私有的，但是如果直接将属性暴露给外界也是有问题的，比如我们没有办法检查赋给属性的值是否有效。我们之前的建议是将属性命名以单下划线开头，通过这种方式来暗示属性是受保护的，不建议外界直接访问，那么如果想访问属性可以通过属性的getter（访问器）和setter（修改器）方法进行对应的操作。如果要做到这点，就可以考虑使用@property包装器来包装getter和setter方法，使得对属性的访问既安全又方便，代码如下所示。

```
class Person(object):

    def __init__(self, name, age):
        self._name = name
        self._age = age

    # 访问器 - getter方法
    @property
    def name(self):
        return self._name

    # 访问器 - getter方法
```

```
    @property
    def age(self):
        return self._age

    # 修改器 - setter方法
    @age.setter
    def age(self, age):
        self._age = age

    def play(self):
        if self._age <= 16:
            print('%s正在玩飞行棋.' % self._name)
        else:
            print('%s正在玩斗地主.' % self._name)


def main():
    person = Person('王大锤', 12)
    person.play()
    person.age = 22
    person.play()
    # person.name = '白元芳'  # AttributeError: can't set attribute


if __name__ == '__main__':
    main()
```

21.2 __slots__魔法

我们讲到这里，不知道大家是否已经意识到，Python是一门动态语言。通常，动态语言允许我们在程序运行时给对象绑定新的属性或方法，当然也可以对已经绑定的属性和方法进行解绑定。但是如果我们需要限定自定义类型的对象只能绑定某些属性，可以通过在类中定义__slots__变量来进行限定。需要注意的是__slots__的限定只对当前类的对象生效，对子类并不起任何作用。

```
class Person(object):

    # 限定Person对象只能绑定_name, _age和_gender属性
    __slots__ = ('_name', '_age', '_gender')

    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def name(self):
        return self._name

    @property
    def age(self):
```

```
        return self._age

    @age.setter
    def age(self, age):
        self._age = age

    def play(self):
        if self._age <= 16:
            print('%s正在玩飞行棋.' % self._name)
        else:
            print('%s正在玩斗地主.' % self._name)


def main():
    person = Person('王大锤', 22)
    person.play()
    person._gender = '男'
    # AttributeError: 'Person' object has no attribute '_is_gay'
    # person._is_gay = True
```

21.3 静态方法和类方法

之前，我们在类中定义的方法都是对象方法，也就是说这些方法都是发送给对象的消息。实际上，我们写在类中的方法并不需要都是对象方法，例如我们定义一个“三角形”类，通过传入三条边长来构造三角形，并提供计算周长和面积的方法，但是传入的三条边长未必能构造出三角形对象，因此我们可以先写一个方法来验证三条边长是否可以构成三角形，这个方法很显然就不是对象方法，因为在调用这个方法时三角形对象尚未创建出来（因为都不知道三条边能不能构成三角形），所以这个方法是属于三角形类而并不属于三角形对象的。我们可以使用静态方法来解决这类问题，代码如下所示。

```
from math import sqrt


class Triangle(object):

    def __init__(self, a, b, c):
        self._a = a
        self._b = b
        self._c = c

    @staticmethod
    def is_valid(a, b, c):
        return a + b > c and b + c > a and a + c > b

    def perimeter(self):
        return self._a + self._b + self._c

    def area(self):
        half = self.perimeter() / 2
        return sqrt(half * (half - self._a) *
```

```
                    (half - self._b) * (half - self._c))


def main():
    a, b, c = 3, 4, 5
    # 静态方法和类方法都是通过给类发消息来调用的
    if Triangle.is_valid(a, b, c):
        t = Triangle(a, b, c)
        print(t.perimeter())
        # 也可以通过给类发消息来调用对象方法但是要传入接收消息的对象作为参数
        # print(Triangle.perimeter(t))
        print(t.area())
        # print(Triangle.area(t))
    else:
        print('无法构成三角形.')


if __name__ == '__main__':
    main()
```

和静态方法比较类似，Python还可以在类中定义类方法，类方法的第一个参数约定名为cls，它代表的是当前类相关的信息的对象（类本身也是一个对象，有的地方也称之为类的元数据对象），通过这个参数我们可以获取和类相关的信息并且可以创建出类的对象，代码如下所示。

```
from time import time, localtime, sleep


class Clock(object):
    """数字时钟"""

    def __init__(self, hour=0, minute=0, second=0):
        self._hour = hour
        self._minute = minute
        self._second = second

    @classmethod
    def now(cls):
        ctime = localtime(time())
        return cls(ctime.tm_hour, ctime.tm_min, ctime.tm_sec)

    def run(self):
        """走字"""
        self._second += 1
        if self._second == 60:
            self._second = 0
            self._minute += 1
            if self._minute == 60:
                self._minute = 0
                self._hour += 1
                if self._hour == 24:
                    self._hour = 0
```

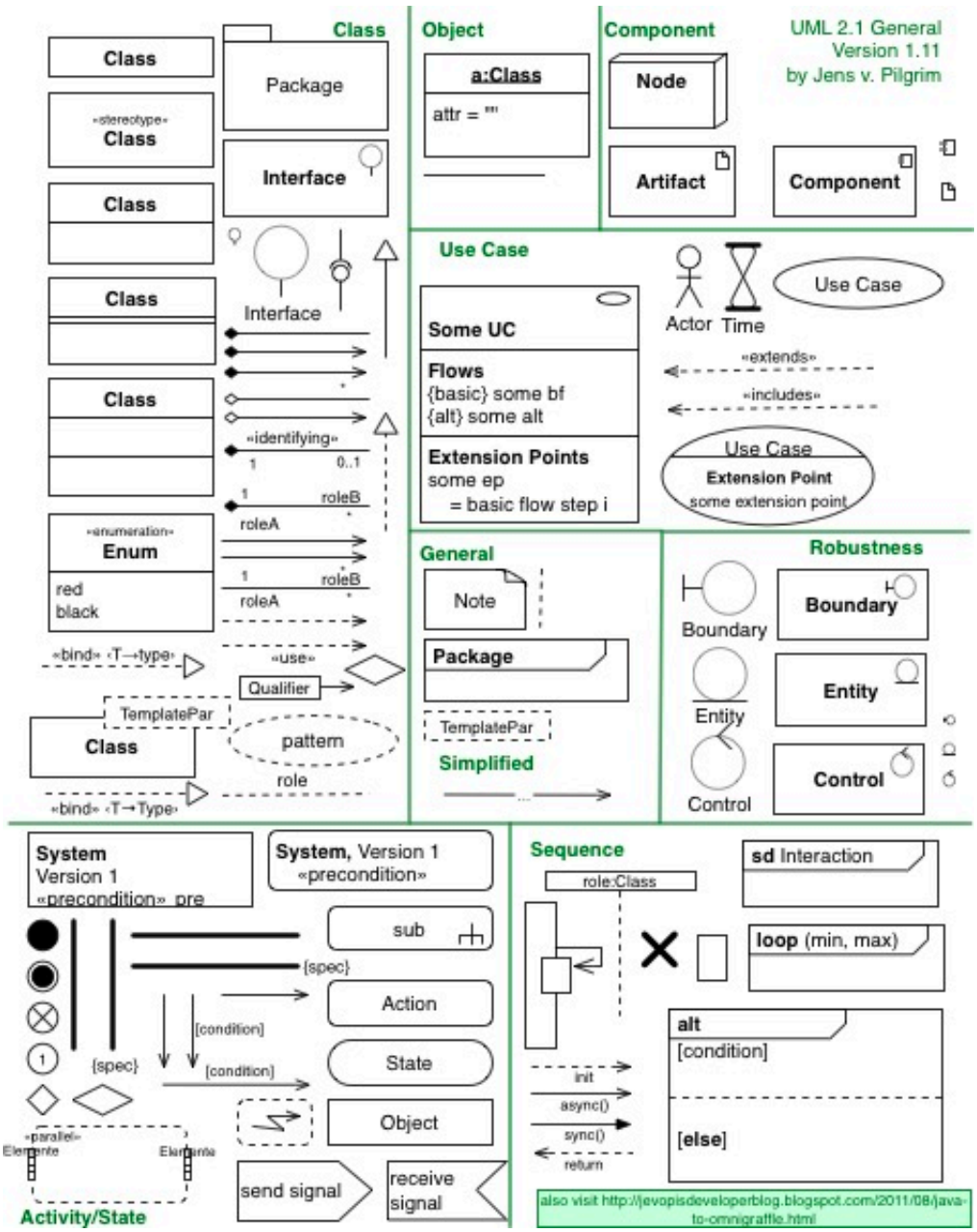
```
28
29      def show(self):
30          """显示时间"""
31          return '%02d:%02d:%02d' % \
32                 (self._hour, self._minute, self._second)
33
34
35  def main():
36      # 通过类方法创建对象并获取系统时间
37      clock = Clock.now()
38      while True:
39          print(clock.show())
40          sleep(1)
41          clock.run()
42
43
44  if __name__ == '__main__':
45      main()
```

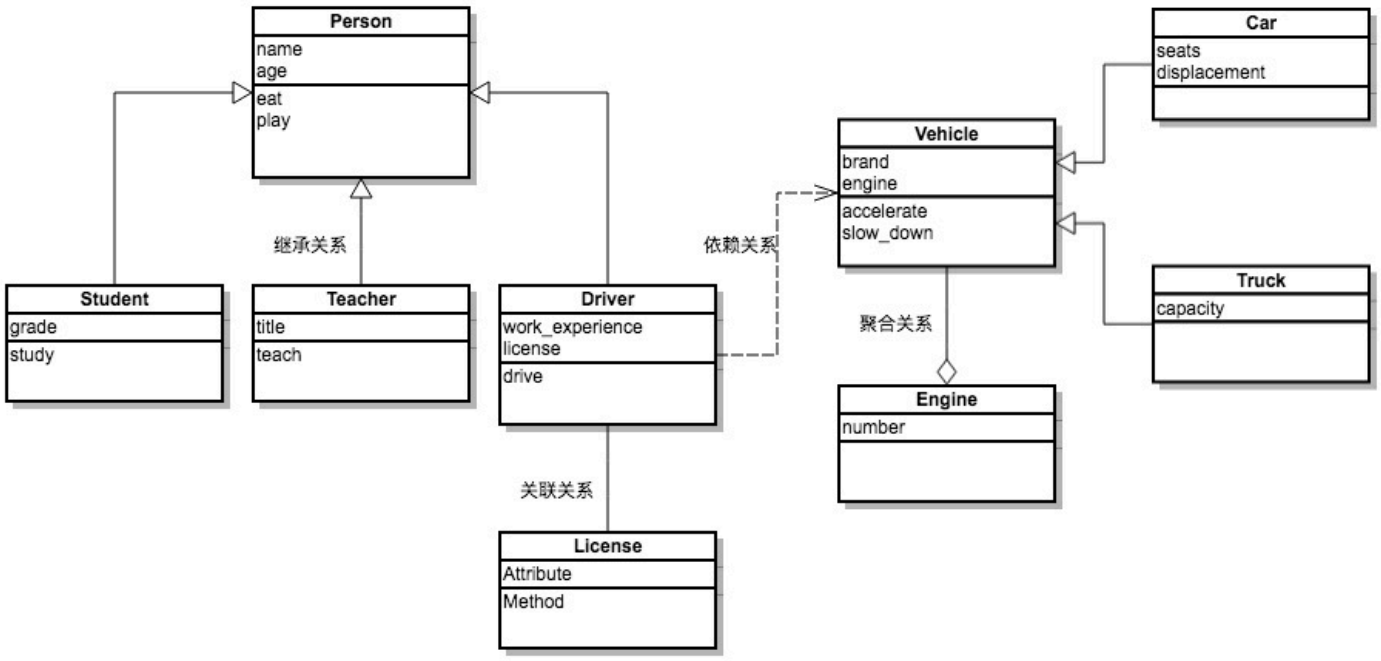
21.4 类之间的关系

简单的说，类和类之间的关系有三种：is-a、has-a和use-a关系。

- is-a关系也叫继承或泛化，比如学生和人的关系、手机和电子产品的关系都属于继承关系。
- has-a关系通常称之为关联，比如部门和员工的关系，汽车和引擎的关系都属于关联关系；关联关系如果是整体和部分的关联，那么我们称之为聚合关系；如果整体进一步负责了部分的生命周期（整体和部分是不可分割的，同时同在也同时消亡），那么这种就是最强的关联关系，我们称之为合成关系。
- use-a关系通常称之为依赖，比如司机有一个驾驶的行为（方法），其中（的参数）使用到了汽车，那么司机和汽车的关系就是依赖关系。

我们可以使用一种叫做UML（统一建模语言）的东西来进行面向对象建模，其中一项重要的工作就是把类和类之间的关系用标准化的图形符号描述出来。关于UML我们在这里不做详细的介绍，有兴趣的读者可以自行阅读《UML面向对象设计基础》一书。





利用类之间的这些关系，我们可以在已有类的基础上来完成某些操作，也可以在已有类的基础上创建

新的类，这些都是实现代码复用的重要手段。复用现有的代码不仅可以减少开发的工作量，也有利于代码的管理和维护，这是我们在日常工作中都会使用到的技术手段。

21.5 继承和多态

刚才我们提到了，可以在已有类的基础上创建新类，这其中的一种做法就是让一个类从另一个类那里将属性和方法直接继承下来，从而减少重复代码的编写。提供继承信息的我们称之为父类，也叫超类或基类；得到继承信息的我们称之为子类，也叫派生类或衍生类。子类除了继承父类提供的属性和方法，还可以定义自己特有的属性和方法，所以子类比父类拥有的更多的能力，在实际开发中，我们经常会用子类对象去替换掉一个父类对象，这是面向对象编程中一个常见的行为，对应的原则称之为里氏替换原则。下面我们先看一个继承的例子。

```
class Person(object):
    """人"""

    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def name(self):
        return self._name

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, age):
        self._age = age

    def play(self):
        print('%s正在愉快的玩耍.' % self._name)

    def watch_av(self):
        if self._age >= 18:
            print('%s正在观看爱情动作片.' % self._name)
        else:
            print('%s只能观看《熊出没》.' % self._name)


class Student(Person):
    """学生"""

    def __init__(self, name, age, grade):
        super().__init__(name, age)
        self._grade = grade

    @property
```

```
    def grade(self):
        return self._grade

    @grade.setter
    def grade(self, grade):
        self._grade = grade

    def study(self, course):
        print('%s的%s正在学习%s.' % (self._grade, self._name, course))


class Teacher(Person):
    """老师"""

    def __init__(self, name, age, title):
        super().__init__(name, age)
        self._title = title

    @property
    def title(self):
        return self._title

    @title.setter
    def title(self, title):
        self._title = title

    def teach(self, course):
        print('%s%s正在讲%s.' % (self._name, self._title, course))


def main():
    stu = Student('王大锤', 15, '初三')
    stu.study('数学')
    stu.watch_av()
    t = Teacher('骆昊', 38, '砖家')
    t.teach('Python程序设计')
    t.watch_av()


if __name__ == '__main__':
    main()
```

子类在继承了父类的方法后，可以对父类已有的方法给出新的实现版本，这个动作称之为方法重写（override）。通过方法重写我们可以让父类的同一个行为在子类中拥有不同的实现版本，当我们调用这个经过子类重写的方法时，不同的子类对象会表现出不同的行为，这个就是多态（poly-morphism）。

```
from abc import ABCMeta, abstractmethod


class Pet(object, metaclass=ABCMeta):
```

```
    """宠物"""

    def __init__(self, nickname):
        self._nickname = nickname

    @abstractmethod
    def make_voice(self):
        """发出声音"""
        pass


class Dog(Pet):
    """狗"""

    def make_voice(self):
        print('%s: 汪汪汪...' % self._nickname)


class Cat(Pet):
    """猫"""

    def make_voice(self):
        print('%s: 喵...喵...' % self._nickname)


def main():
    pets = [Dog('旺财'), Cat('凯蒂'), Dog('大黄')]
    for pet in pets:
        pet.make_voice()


if __name__ == '__main__':
    main()
```

在上面的代码中，我们将`Pet`类处理成了一个抽象类，所谓抽象类就是不能够创建对象的类，这种类的存在就是专门为了让其他类去继承它。Python从语法层面并没有像Java或C#那样提供对抽象类的支持，但是我们可以通过`abc`模块的`ABCMeta`元类和`abstractmethod`包装器来达到抽象类的效果，如果一个类中存在抽象方法那么这个类就不能够实例化（创建对象）。上面的代码中，`Dog`和`Cat`两个子类分别对`Pet`类中的`make_voice`抽象方法进行了重写并给出了不同的实现版本，当我们在`main`函数中调用该方法时，这个方法就表现出了多态行为（同样的方法做了不同的事情）。

22. 综合案例

案例1：奥特曼打小怪兽。

```
from abc import ABCMeta, abstractmethod
from random import randint, randrange

```

```python

class Fighter(object, metaclass=ABCMeta):
    """战斗者"""

    # 通过__slots__魔法限定对象可以绑定的成员变量
    __slots__ = ('_name', '_hp')

    def __init__(self, name, hp):
        """初始化方法

        :param name: 名字
        :param hp: 生命值
        """
        self._name = name
        self._hp = hp

    @property
    def name(self):
        return self._name

    @property
    def hp(self):
        return self._hp

    @hp.setter
    def hp(self, hp):
        self._hp = hp if hp >= 0 else 0

    @property
    def alive(self):
        return self._hp > 0

    @abstractmethod
    def attack(self, other):
        """攻击

        :param other: 被攻击的对象
        """
        pass


class Ultraman(Fighter):
    """奥特曼"""

    __slots__ = ('_name', '_hp', '_mp')

    def __init__(self, name, hp, mp):
        """初始化方法

        :param name: 名字
        :param hp: 生命值
        :param mp: 魔法值
        """
```

```
        super().__init__(name, hp)
        self._mp = mp

    def attack(self, other):
        other.hp -= randint(15, 25)

    def huge_attack(self, other):
        """究极必杀技(打掉对方至少50点或四分之三的血)

        :param other: 被攻击的对象

        :return: 使用成功返回True否则返回False
        """
        if self._mp >= 50:
            self._mp -= 50
            injury = other.hp * 3 // 4
            injury = injury if injury >= 50 else 50
            other.hp -= injury
            return True
        else:
            self.attack(other)
            return False

    def magic_attack(self, others):
        """魔法攻击

        :param others: 被攻击的群体

        :return: 使用魔法成功返回True否则返回False
        """
        if self._mp >= 20:
            self._mp -= 20
            for temp in others:
                if temp.alive:
                    temp.hp -= randint(10, 15)
            return True
        else:
            return False

    def resume(self):
        """恢复魔法值"""
        incr_point = randint(1, 10)
        self._mp += incr_point
        return incr_point

    def __str__(self):
        return '~~~%s奥特曼~~~\n' % self._name + \
            '生命值: %d\n' % self._hp + \
            '魔法值: %d\n' % self._mp


class Monster(Fighter):
    """小怪兽"""
```

```
110
111     __slots__ = ('_name', '_hp')
112
113     def attack(self, other):
114         other.hp -= randint(10, 20)
115
116     def __str__(self):
117         return '~~~%s小怪兽~~~\n' % self._name + \
118             '生命值: %d\n' % self._hp
119
120
121 def is_any_alive(monsters):
122     """判断有没有小怪兽是活着的"""
123     for monster in monsters:
124         if monster.alive > 0:
125             return True
126     return False
127
128
129 def select_alive_one(monsters):
130     """选中一只活着的小怪兽"""
131     monsters_len = len(monsters)
132     while True:
133         index = randrange(monsters_len)
134         monster = monsters[index]
135         if monster.alive > 0:
136             return monster
137
138
139 def display_info(ultraman, monsters):
140     """显示奥特曼和小怪兽的信息"""
141     print(ultraman)
142     for monster in monsters:
143         print(monster, end='')
144
145
146 def main():
147     u = Ultraman('骆昊', 1000, 120)
148     m1 = Monster('狄仁杰', 250)
149     m2 = Monster('白元芳', 500)
150     m3 = Monster('王大锤', 750)
151     ms = [m1, m2, m3]
152     fight_round = 1
153     while u.alive and is_any_alive(ms):
154         print('========第%02d回合========' % fight_round)
155         m = select_alive_one(ms)  # 选中一只小怪兽
156         skill = randint(1, 10)   # 通过随机数选择使用哪种技能
157         if skill <= 6:  # 60%的概率使用普通攻击
158             print('%s使用普通攻击打了%s.' % (u.name, m.name))
159             u.attack(m)
160             print('%s的魔法值恢复了%d点.' % (u.name, u.resume()))
161         elif skill <= 9:  # 30%的概率使用魔法攻击(可能因魔法值不足而失败)
162             if u.magic_attack(ms):
```

```
                print('%s使用了魔法攻击.' % u.name)
            else:
                print('%s使用魔法失败.' % u.name)
        else:  # 10%的概率使用究极必杀技(如果魔法值不足则使用普通攻击)
            if u.huge_attack(m):
                print('%s使用究极必杀技虐了%s.' % (u.name, m.name))
            else:
                print('%s使用普通攻击打了%s.' % (u.name, m.name))
                print('%s的魔法值恢复了%d点.' % (u.name, u.resume()))
        if m.alive > 0:  # 如果选中的小怪兽没有死就回击奥特曼
            print('%s回击了%s.' % (m.name, u.name))
            m.attack(u)
        display_info(u, ms)  # 每个回合结束后显示奥特曼和小怪兽的信息
        fight_round += 1
    print('\n========战斗结束!========\n')
    if u.alive > 0:
        print('%s奥特曼胜利!' % u.name)
    else:
        print('小怪兽胜利!')


if __name__ == '__main__':
    main()
```

案例2：扑克游戏。

```
import random


class Card(object):
    """一张牌"""

    def __init__(self, suite, face):
        self._suite = suite
        self._face = face

    @property
    def face(self):
        return self._face

    @property
    def suite(self):
        return self._suite

    def __str__(self):
        if self._face == 1:
            face_str = 'A'
        elif self._face == 11:
            face_str = 'J'
        elif self._face == 12:
```

```
            face_str = 'Q'
        elif self._face == 13:
            face_str = 'K'
        else:
            face_str = str(self._face)
        return '%s%s' % (self._suite, face_str)

    def __repr__(self):
        return self.__str__()


class Poker(object):
    """一副牌"""

    def __init__(self):
        self._cards = [Card(suite, face)
                       for suite in '♠♥♣♦'
                       for face in range(1, 14)]
        self._current = 0

    @property
    def cards(self):
        return self._cards

    def shuffle(self):
        """洗牌(随机乱序)"""
        self._current = 0
        random.shuffle(self._cards)

    @property
    def next(self):
        """发牌"""
        card = self._cards[self._current]
        self._current += 1
        return card

    @property
    def has_next(self):
        """还有没有牌"""
        return self._current < len(self._cards)


class Player(object):
    """玩家"""

    def __init__(self, name):
        self._name = name
        self._cards_on_hand = []

    @property
    def name(self):
        return self._name

```

```
    @property
    def cards_on_hand(self):
        return self._cards_on_hand

    def get(self, card):
        """摸牌"""
        self._cards_on_hand.append(card)

    def arrange(self, card_key):
        """玩家整理手上的牌"""
        self._cards_on_hand.sort(key=card_key)


# 排序规则-先根据花色再根据点数排序
def get_key(card):
    return (card.suite, card.face)


def main():
    p = Poker()
    p.shuffle()
    players = [Player('东邪'), Player('西毒'), Player('南帝'), Player('北丐')]
    for _ in range(13):
        for player in players:
            player.get(p.next)
    for player in players:
        print(player.name + ':', end=' ')
        player.arrange(get_key)
        print(player.cards_on_hand)


if __name__ == '__main__':
    main()
```

说明： 大家可以自己尝试在上面代码的基础上写一个简单的扑克游戏，例如21点(Black Jack)，游戏的规则可以自己在网上找一找。

案例3：工资结算系统。

```
"""
某公司有三种类型的员工 分别是部门经理、程序员和销售员
需要设计一个工资结算系统 根据提供的员工信息来计算月薪
部门经理的月薪是每月固定15000元
程序员的月薪按本月工作时间计算 每小时150元
销售员的月薪是1200元的底薪加上销售额5%的提成
"""
from abc import ABCMeta, abstractmethod


class Employee(object, metaclass=ABCMeta):
    """员工"""
```

```
12
13      def __init__(self, name):
14          """
15          初始化方法
16
17          :param name: 姓名
18          """
19          self._name = name
20
21      @property
22      def name(self):
23          return self._name
24
25      @abstractmethod
26      def get_salary(self):
27          """
28          获得月薪
29
30          :return: 月薪
31          """
32          pass
33
34
35  class Manager(Employee):
36      """部门经理"""
37
38      def get_salary(self):
39          return 15000.0
40
41
42  class Programmer(Employee):
43      """程序员"""
44
45      def __init__(self, name, working_hour=0):
46          super().__init__(name)
47          self._working_hour = working_hour
48
49      @property
50      def working_hour(self):
51          return self._working_hour
52
53      @working_hour.setter
54      def working_hour(self, working_hour):
55          self._working_hour = working_hour if working_hour > 0 else 0
56
57      def get_salary(self):
58          return 150.0 * self._working_hour
59
60
61  class Salesman(Employee):
62      """销售员"""
63
64      def __init__(self, name, sales=0):
```

```
        super().__init__(name)
        self._sales = sales

    @property
    def sales(self):
        return self._sales

    @sales.setter
    def sales(self, sales):
        self._sales = sales if sales > 0 else 0

    def get_salary(self):
        return 1200.0 + self._sales * 0.05


def main():
    emps = [
        Manager('刘备'), Programmer('诸葛亮'),
        Manager('曹操'), Salesman('荀彧'),
        Salesman('吕布'), Programmer('张辽'),
        Programmer('赵云')
    ]
    for emp in emps:
        if isinstance(emp, Programmer):
            emp.working_hour = int(input('请输入%s本月工作时间: ' % emp.name))
        elif isinstance(emp, Salesman):
            emp.sales = float(input('请输入%s本月销售额: ' % emp.name))
        # 同样是接收get_salary这个消息但是不同的员工表现出了不同的行为(多态)
        print('%s本月工资为: ￥%s元' %
              (emp.name, emp.get_salary()))


if __name__ == '__main__':
    main()
```